

ĐẠI HỌC BÁCH KHOA HÀ NỘI

ĐỒ ÁN TỐT NGHIỆP

Xây dựng tính năng bám bắt đối tượng cho ROV

TRẦN NGUYỄN CHIẾN

chien.tn220018@sis.hust.edu.vn

Chương trình đào tạo: Khoa học máy tính

Giảng viên hướng dẫn: TS. Ngô Lam Trung

Khoa: Khoa học máy tính

Trường: Công nghệ thông tin và Truyền thông

HÀ NỘI, 06/2026

ĐẠI HỌC BÁCH KHOA HÀ NỘI

ĐỒ ÁN TỐT NGHIỆP

Thiết kế xây dựng công nghệ thực tế ảo và ứng dụng

TRẦN NGUYỄN CHIẾN

chien.tn220018@sis.hust.edu.vn

Chương trình đào tạo: Khoa học máy tính

Giảng viên hướng dẫn: PGS. TS. Ngô Lam Trung

Chữ kí GVHD

Khoa: Khoa học máy tính

Trường: Công nghệ Thông tin và Truyền thông

HÀ NỘI, 06/2022

LỜI CẢM ƠN

Lời đầu tiên, em xin chân thành cảm ơn các thầy cô khoa Khoa học máy tính, đặc biệt là thầy Ngô Lam Trung đã tận tâm định hướng và đồng hành cùng em trong suốt quá trình thực hiện đồ án.

Con xin cảm ơn gia đình đã luôn là điểm tựa bình yên và vững chãi nhất. Cảm ơn cùng những người bạn kề vai sát cánh vì đã luôn thấu hiểu, kiên nhẫn và động viên mình qua những khoảng thời gian áp lực nhất.

Cuối cùng, tôi xin gửi lời cảm ơn đến chính bản thân mình. Cảm ơn vì đã luôn giữ vững kỷ luật, không thỏa hiệp với khó khăn và nỗ lực đến cùng để mang lại một kết quả trọn vẹn nhất cho cột mốc quan trọng này.

Xin chân thành cảm ơn tất cả mọi người!

TÓM TẮT NỘI DUNG ĐỒ ÁN

Trong các nhiệm vụ khảo sát, quan sát và thao tác dưới nước, ROV thường được điều khiển thủ công thông qua hình ảnh camera truyền về từ phương tiện. Tuy nhiên, môi trường dưới nước có nhiều yếu tố gây khó khăn như ánh sáng yếu, nhiễu ảnh, dòng chảy, độ trễ truyền hình ảnh và đặc biệt là không thể sử dụng GPS để định vị chính xác. Vì vậy, bài toán bám theo và tiếp cận mục tiêu dưới nước là một vấn đề quan trọng, giúp giảm tải cho người vận hành và nâng cao độ chính xác khi thực hiện nhiệm vụ. Hiện nay có nhiều hướng tiếp cận cho bài toán này như điều khiển hoàn toàn thủ công, sử dụng cảm biến khoảng cách, dùng mô hình học sâu để phát hiện mục tiêu hoặc dùng các thuật toán thị giác truyền thống để theo dõi chuyển động trong ảnh. Tuy nhiên, các phương pháp học sâu thường yêu cầu dữ liệu huấn luyện và tài nguyên tính toán lớn, còn điều khiển thủ công phụ thuộc nhiều vào kinh nghiệm người lái. Trong đồ án này, em lựa chọn hướng tiếp cận sử dụng Optical Flow kết hợp với bộ điều khiển PID do phương pháp này nhẹ, có thể chạy thời gian thực và phù hợp với hệ thống ROV hiện có. Giải pháp được xây dựng gồm ba phần chính: module truyền hình ảnh MJPEG từ camera ROV, module theo dõi mục tiêu và giao diện chọn mục tiêu trên máy host, và module điều khiển ROV trên phương tiện. Sau khi người dùng chọn mục tiêu, hệ thống theo dõi sự dịch chuyển và thay đổi kích thước của mục tiêu giữa các frame, chuẩn hóa thành các đại lượng như sai lệch ngang, sai lệch tỷ lệ kích thước và độ tin cậy, sau đó gửi tới bộ điều khiển qua UDP. Bộ điều khiển PID sử dụng các tín hiệu này để sinh lệnh tiến/lùi, ngang, kết hợp với giữ hướng và giữ độ sâu, rồi chuyển thành lệnh cho các thruster thông qua ma trận mixing. Đóng góp chính của đồ án là xây dựng được một pipeline Track and Catch hoàn chỉnh cho ROV dưới nước, tích hợp thị giác máy tính với điều khiển thời gian thực, có cơ chế kiểm tra độ tin cậy, giới hạn lệnh và ưu tiên can thiệp thủ công. Kết quả đạt được là hệ thống có khả năng chọn, theo dõi và hỗ trợ ROV bám giữ mục tiêu theo vị trí ban đầu trong điều kiện thử nghiệm.

Sinh viên thực hiện
(Ký và ghi rõ họ tên)

ABSTRACT

In underwater inspection, observation, and manipulation tasks, Remotely Operated Vehicles (ROVs) are commonly controlled manually based on visual feedback from onboard cameras. However, underwater environments pose several challenges, including poor lighting, image noise, water currents, visual latency, and the unavailability of GPS-based positioning. These factors make target tracking and target approach difficult, especially when precise relative motion is required. Existing approaches include fully manual control, distance-sensor-based control, deep-learning-based object detection, and traditional computer vision methods for motion tracking. While deep learning methods can provide strong recognition capability, they often require large training datasets and considerable computational resources. Manual control, on the other hand, depends heavily on the operator's experience and may become unstable under latency or disturbance. In this graduation project, an Optical Flow-based tracking approach combined with PID control is selected because it is lightweight, suitable for real-time operation, and compatible with the existing ROV platform. The proposed system consists of three main modules: an MJPEG camera streaming module on the ROV, a host-side tracking and target selection module, and an onboard ROV control module. After the operator selects a target on the web interface, the tracking module estimates the target's displacement and scale variation across consecutive frames. These values are normalized into vision signals, including horizontal displacement, scale error, and tracking confidence, and are transmitted to the controller via UDP. The controller then uses PID algorithms to generate forward/backward and lateral motion commands, while also integrating heading hold and depth hold functions. The final thruster commands are computed through a mixing matrix and sent to the ROV's thrusters. The main contribution of this project is the development of a complete Track and Catch pipeline that integrates real-time computer vision with ROV motion control. The implemented system enables target selection, visual tracking, confidence-based validation, command limitation, manual override, and target-relative position holding under experimental conditions.

MỤC LỤC

CHƯƠNG 1. GIỚI THIỆU ĐỀ TÀI.....	1
1.1 Đặt vấn đề.....	1
1.1.1 Giới thiệu ROV và nhu cầu ứng dụng	1
1.1.2 Nhiệm vụ đề tài	1
1.1.3 Chia nhiệm vụ	1
1.1.4 Cấu trúc báo cáo	1
CHƯƠNG 2. MÔ TẢ PHẦN CỨNG	2
2.1 Cơ khí	2
2.2 Động lực - Động cơ	2
2.3 Sensor	2
2.4 Cơ cấu điều khiển	2
2.4.1 Động cơ	2
2.4.2 Đền.....	2
2.4.3 Tay cầm	2
2.5 Bộ điều khiển	2
2.5.1 Máy tính nhúng.....	2
2.5.2 Driver board	2
2.6 Truyền thông (Ethernet + PLC).....	2
CHƯƠNG 3. THIẾT KẾ HỆ THỐNG PHẦN MỀM.....	3
3.1 Tổng quan kiến trúc phần mềm hệ thống ROV	3
3.2 Kiến trúc tổng thể hệ thống	4
3.3 Bảng đặc tả chức năng	4
3.3.1 Điều khiển bằng tay	4
3.3.2 Điều khiển tự động.....	5

3.3.3	Hiển thị dữ liệu thời gian thực từ ROV	7
3.3.4	Lưu trữ dữ liệu từ ROV	7
3.3.5	Lưu trữ dữ liệu lên web.....	8
3.4	Thiết kế module.....	9
3.4.1	Thiết kế module firmware trên ROV	9
3.4.2	Thiết kế module phần mềm GCS	13
3.4.3	Kiến trúc tổng thể phần mềm GCS	14
3.4.4	Nhóm module truyền thông.....	14
3.4.5	Nhóm module điều khiển.....	16
3.4.6	Nhóm module xử lý dữ liệu thời gian thực	17
3.4.7	Nhóm module ghi dữ liệu nhiệm vụ	17
3.4.8	Nhóm module quản lý hệ thống.....	19
3.4.9	Nhóm module giao diện người dùng	19
3.5	Quan hệ giữa các module	19
CHƯƠNG 4. XÂY DỰNG PHẦN MỀM TRÊN ROV		21
4.1	Kiến trúc tổng thể và luồng dữ liệu.....	21
4.2	Module cảm biến.....	21
4.2.1	Đo độ sâu và nhiệt độ nước (MS5837)	21
4.2.2	IMU ICM20602 (gia tốc, con quay, từ kế) + pitch,roll,yaw qua Extended Kalman filter.....	22
4.2.3	Đo nhiệt độ và độ ẩm khoang kín (Si7021).....	23
4.2.4	Module giám sát năng lượng qua ADC ADS1115 (Điện áp, Dòng điện, SOC)	24
4.3	Module truyền thông.....	26
4.3.1	Kết nối GCS(ping_server).....	26
4.3.2	Nhận lệnh điều khiển(control_loop).....	26

4.4 Moudle điều khiển	26
4.4.1 Điều khiển thủ công - Mixing matrix và điều khiển động cơ	26
4.4.2 Điều khiển tự động.....	26
4.5 Module cơ cấu chấp hành	26
4.5.1 Điều khiển ESC và cơ cấu chấp hành qua PCA9685	26
4.5.2 Điều khiển đèn chiếu sáng	28
4.5.3 Điều khiển độ ngả camera	28
4.5.4 Điều khiển tay gấp (Gripper).....	28
4.6 Module telemetry	28
4.6.1 Đóng gói và phát dữ liệu về cho GCS.....	28
4.7 Moule cấu hình động cơ	28
CHƯƠNG 5. GCS	29
5.1 Cơ sở lý thuyết	29
5.1.1 Tổng quan chức năng Track and Catch trong hệ thống ROV	29
5.1.2 Cơ sở về truyền luồng hình ảnh MJPEG	29
5.1.3 Cơ sở về theo dõi mục tiêu bằng Optical Flow	29
5.1.4 Chuẩn hóa sai lệch ảnh cho điều khiển	29
5.1.5 Cơ sở điều khiển PID cho Track and Catch	29
5.1.6 Cơ sở điều khiển ROV và mixing matrix	29
5.1.7 Cơ sở truyền thông và tích hợp hệ thống.....	29
5.1.8 Cơ chế an toàn khi vận hành.....	29
CHƯƠNG 6. ĐIỀU KHIỂN TỰ ĐỘNG	30
6.1 Cơ sở lý thuyết	30
6.1.1 Cơ sở lý thuyết điều khiển chuyển động ROV	30
6.1.2 Ánh xạ động cơ logic sang kênh PWM thực tế	37
6.1.3 Thuật toán Optical Flow	39

CHƯƠNG 7. QUẢN LÝ THÔNG TIN VÀ PHÂN TÍCH DỮ LIỆU	62
7.1 Tổng quan phân hệ.....	62
7.1.1 Bài toán đặt ra	62
7.1.2 Biểu đồ use case tổng quát.....	63
7.1.3 Yêu cầu phi chức năng.....	66
7.2 Kiến trúc và hệ sinh thái công nghệ.....	67
7.2.1 Kiến trúc tổng quan.....	67
7.2.2 Hệ sinh thái công nghệ	69
7.2.3 Thiết kế cơ sở dữ liệu	70
7.2.4 Mô hình triển khai đề xuất	71
7.3 Phân quyền và xác thực	72
7.3.1 Mô hình phân quyền RBAC	72
7.3.2 Cơ chế xác thực JWT và Google OAuth	73
7.3.3 Vòng đời Project và Trip	74
7.3.4 Vòng đời phân tích Media	75
7.4 Hiện thực các chức năng cốt lõi.....	76
7.4.1 Dashboard và thống kê tổng quan	76
7.4.2 Quản lý Project và Trip.....	77
7.4.3 Giao diện Cockpit TripDetailPage	78
7.4.4 Upload đa nguồn dữ liệu thực địa	79
7.4.5 Thông báo realtime và Audit Log	79
7.4.6 Sơ đồ tuần tự các luồng nghiệp vụ chính	81
7.5 Giải pháp đồng bộ hóa Video và Dữ liệu cảm biến.....	83
7.5.1 Bài toán đặt ra	83
7.5.2 Giải pháp hiện thực	83
7.5.3 Kết quả đạt được	84

7.6 Kiến trúc xử lý bất đồng bộ tích hợp YOLOv8.....	84
7.6.1 Bài toán đặt ra	84
7.6.2 Giải pháp hiện thực	84
7.6.3 AI Analyze Popover – Cấu hình phân tích	85
7.6.4 Kết quả đạt được	86
7.7 Hệ thống trích xuất bằng chứng (Evidence System)	86
7.7.1 Bài toán đặt ra	86
7.7.2 Giải pháp hiện thực	87
7.7.3 Kết quả đạt được	87
7.8 Phát hiện bất thường cảm biến tự động (Z-Score)	88
7.8.1 Bài toán đặt ra	88
7.8.2 Giải pháp hiện thực	88
7.8.3 Kết quả đạt được	88
7.9 Tóm tắt nhiệm vụ bằng Mô hình ngôn ngữ lớn (Gemini)	88
7.9.1 Bài toán đặt ra	88
7.9.2 Giải pháp hiện thực	89
7.9.3 Kết quả đạt được	89
7.10 Kiểm thử hệ thống	90
7.10.1 Kiểm thử chức năng (Functional Test)	90
7.10.2 Kiểm thử tải (Load Test)	91
7.10.3 Kiểm thử hiệu năng Lighthouse	91
CHƯƠNG 8. KẾT QUẢ THỰC NGHIỆM.....	93
CHƯƠNG 9. KẾT LUẬN	94
9.1 Kết luận	94
9.2 Hướng phát triển.....	94
TÀI LIỆU THAM KHẢO.....	97

PHỤ LỤC.....	98
A. HƯỚNG DẪN VIẾT ĐỒ ÁN TỐT NGHIỆP	98
A.1 Ngành học.....	99
A.2 Đánh dấu (bullet) và đánh số (numering)	99
A.3 Cách thêm bảng	100
A.4 Chèn hình ảnh	100
A.5 Tài liệu tham khảo	100
A.6 Cách viết phương trình và công thức toán học.....	101
A.7 Qui cách đóng quyển.....	101
B. ĐẶC TẢ USE CASE.....	103
B.1 UC-01: Tải lên và đồng bộ dữ liệu cảm biến	103
B.2 UC-02: Kích hoạt phân tích nhận diện vật thể bằng YOLOv8.....	103
B.3 UC-03: Trích xuất và phân tích bằng chứng	104
B.4 UC-04: Tạo báo cáo tóm tắt bằng ngôn ngữ tự nhiên	105

DANH MỤC HÌNH VẼ

Hình 7.1	Biểu đồ Use Case tổng quát phân hệ quản lý thông tin ROV . .	63
Hình 7.2	Biểu đồ Use Case phân rã – Phân hệ Upload và Quản lý Dữ liệu Thực địa	64
Hình 7.3	Biểu đồ Use Case phân rã – Phân hệ AI Phân tích	65
Hình 7.4	Biểu đồ hoạt động – Quy trình phân tích dữ liệu nhiệm vụ ROV	66
Hình 7.5	Kiến trúc tổng quan phân hệ quản lý thông tin ROV	68
Hình 7.6	Sơ đồ phụ thuộc module phía Backend	68
Hình 7.7	Sơ đồ ER cơ sở dữ liệu phân hệ quản lý thông tin ROV	70
Hình 7.8	Mô hình triển khai đề xuất trên VPS sử dụng Docker Compose	72
Hình 7.9	Màn hình đăng nhập – hỗ trợ cả Email/Password và Google OAuth 2.0	74
Hình 7.10	Sơ đồ trạng thái vòng đời Chuyển khảo sát (Project)	74
Hình 7.11	Sơ đồ trạng thái vòng đời Lướt lặn (Trip)	75
Hình 7.12	Dashboard – KPI cards và biểu đồ thống kê tổng quan (dark mode)	76
Hình 7.13	Trang danh sách Projects – filter, search và export	77
Hình 7.14	ProjectDetailPage – TripCard mở rộng với DATA FILES và AI Summary	77
Hình 7.15	Trang danh sách Trips – filter theo status và date range	78
Hình 7.16	TripDetailPage Cockpit – layout 3 cột, đồng bộ video–biểu đồ thời gian thực	78
Hình 7.17	Tab biểu đồ Environment – dot đỏ đánh dấu điểm bất thường Z-Score ($ z > 2,5$)	79
Hình 7.18	Notification bell dropdown – thông báo realtime qua SSE . . .	80
Hình 7.19	Trang quản lý người dùng (Admin) – bulk operations và role management	81
Hình 7.20	Audit Log – nhật ký hoạt động admin theo timeline	81
Hình 7.21	Sơ đồ tuần tự – Luồng tải lên file CSV cảm biến	82
Hình 7.22	Sơ đồ tuần tự – Luồng phân tích YOLO bất đồng bộ qua Bull Queue	83
Hình 7.23	Bounding box YOLO overlay trên video ROV – đồng bộ per-frame theo <code>frameTime</code>	85
Hình 7.24	AI Analyze Popover – chọn model và điều chỉnh ngưỡng confidence trước khi phân tích	86

Hình 7.25 Evidence Panel và EvidenceViewer – PHOTO/CLIP thumbnails với bounding box overlay	87
Hình 7.26 AI Summary song ngữ – được tổng hợp bởi Gemini 2.5 Flash sau khi Project hoàn tất	89
Hình 7.27 Kết quả kiểm thử chức năng – 37/37 ca đạt (100%)	90
Hình A.1 Internet vạn vật	100
Hình A.2 Qui cách đóng quyển đồ án	102
Hình A.3 Qui cách đóng quyển đồ án	102

DANH MỤC BẢNG BIỂU

Bảng 1.1	Caption	1
Bảng 3.1	Đặc tả chức năng điều khiển bằng tay	5
Bảng 3.2	Đặc tả chức năng giữ hướng	5
Bảng 3.3	Đặc tả chức năng giữ độ sâu	6
Bảng 3.4	Đặc tả chức năng giữ vị trí theo DVL	6
Bảng 3.5	Đặc tả chức năng giữ vị trí theo Vision	7
Bảng 3.6	Đặc tả chức năng hiển thị ROV	7
Bảng 3.7	Đặc tả chức năng lưu trữ dữ liệu từ ROV	8
Bảng 3.8	Đặc tả chức năng lưu trữ dữ liệu	8
Bảng 3.9	Các module cảm biến trong phần mềm ROV	10
Bảng 3.10	Các module điều khiển trong phần mềm ROV	11
Bảng 3.11	Các cơ cấu chấp hành và thông số PWM điều khiển	11
Bảng 3.12	Các module Camera trong phần mềm ROV	12
Bảng 3.13	Các module truyền thông trong phần mềm ROV	12
Bảng 3.14	Các hàm telemetry và kênh gửi dữ liệu	13
Bảng 3.15	Module cấu hình và driver chấp hành	13
Bảng 3.16	Chức năng của MavlinkService	15
Bảng 3.17	Chức năng của CameraService	15
Bảng 3.18	Chức năng của SonarService	16
Bảng 3.19	Chức năng của DVLSERVICE	16
Bảng 3.20	Chức năng của JoystickService	17
Bảng 3.21	Chức năng của TelemetryService	17
Bảng 3.22	Chức năng CameraRecorderService	18
Bảng 7.1	So sánh các giải pháp quản lý dữ liệu ROV hiện có	62
Bảng 7.2	Hệ sinh thái công nghệ phân hệ quản lý thông tin	69
Bảng 7.3	Mô tả các collection chính trong MongoDB	71
Bảng 7.4	Ma trận phân quyền RBAC của phân hệ quản lý thông tin	73
Bảng 7.5	Các trạng thái vòng đời phân tích Media (analysisStatus) . . .	75
Bảng 7.6	Kết quả kiểm thử tải – 20 concurrent $\times$ 3 rounds (60 re- quests/endpoint)	91
Bảng 7.7	Kết quả kiểm thử Lighthouse – Desktop mode	91
Bảng A.1	Table to test captions and labels.	100
Bảng B.1	Đặc tả Use Case UC-01: Tải lên và đồng bộ dữ liệu cảm biến .	103

Bảng B.2	Đặc tả Use Case UC-02: Phân tích nhận diện vật thể bằng AI .	103
Bảng B.3	Đặc tả Use Case UC-03: Trích xuất bằng chứng (Evidence System)	104
Bảng B.4	Đặc tả Use Case UC-04: Tóm tắt nhiệm vụ bằng AI (AI Summary)	105

CHƯƠNG 1. GIỚI THIỆU ĐỀ TÀI

1.1 Đặt vấn đề

1.1.1 Giới thiệu ROV và nhu cầu ứng dụng

1.1.2 Nhiệm vụ đề tài

- Không làm phần cứng, xây dựng pm điều khiển (Firmware, GCS, Cloud), thử nghiệm

1.1.3 Chia nhiệm vụ

STT	Họ và tên	MSSV	Nhiệm vụ
1	Nguyễn Mạnh Cường	20225268	
2	Trần Nguyên Chiến		
3	Phan Lê Hải Đăng	20225273	

Bảng 1.1: Caption

1.1.4 Cấu trúc báo cáo

CHƯƠNG 2. MÔ TẢ PHẦN CỨNG

2.1 Cơ khí

Thông thường, khảo sát chi tiết về hiện trạng và yêu cầu của phần mềm sẽ được lấy từ ba nguồn chính, đó là (i) người dùng/khách hàng, (ii) các hệ thống đã có, (iii) và các ứng dụng tương tự. Sinh viên cần tiến hành phân tích, so sánh, đánh giá chi tiết ưu nhược điểm của các sản phẩm/nguyên cứu hiện có. Sinh viên có thể lập bảng so sánh nếu cần thiết. Kết hợp với khảo sát người dùng/khách hàng (nếu có), sinh viên nêu và mô tả sơ lược các tính năng phần mềm quan trọng cần phát triển.

2.2 Động lực - Động cơ

2.3 Sensor

2.4 Cơ cấu điều khiển

2.4.1 Động cơ

2.4.2 Đèn

2.4.3 Tay cầm

2.5 Bộ điều khiển

2.5.1 Máy tính nhúng

2.5.2 Driver board

2.6 Truyền thông (Ethernet + PLC)

CHƯƠNG 3. THIẾT KẾ HỆ THỐNG PHẦN MỀM

Vẽ biểu đồ vào chương này, ref:

3.1 Tổng quan kiến trúc phần mềm hệ thống ROV

Hệ thống tàu ngầm điều khiển từ xa (ROV – Remotely Operated Vehicle) được thiết kế theo mô hình phân tán, bao gồm hai thành phần phần mềm chính: firmware nhúng chạy trực tiếp trên ROV và phần mềm điều khiển mặt đất Ground Control Station (GCS).

Firmware nhúng được triển khai trên nền tảng tính toán nhúng QCS8550, chạy hệ điều hành Linux và được phát triển chủ yếu bằng ngôn ngữ Python. Thành phần này chịu trách nhiệm giao tiếp với các cảm biến và cơ cấu chấp hành gắn trực tiếp trên ROV, thực hiện các thuật toán điều khiển thời gian thực, duy trì trạng thái ổn định của phương tiện dưới nước và truyền dữ liệu telemetry về trạm điều khiển.

Phần mềm Ground Control Station (GCS) được phát triển trên nền tảng desktop sử dụng ngôn ngữ C và framework WPF. GCS đóng vai trò trung tâm điều khiển, cho phép người vận hành giám sát trạng thái ROV theo thời gian thực, điều khiển bằng thiết bị Joystick, tiếp nhận và hiển thị dữ liệu video, lưu trữ dữ liệu nhiệm vụ, đồng thời hỗ trợ phân tích dữ liệu sau quá trình vận hành.

Kiến trúc hệ thống sử dụng kết nối Ethernet thông qua dây tether nối giữa ROV và trạm điều khiển. Trong hệ thống tồn tại ba luồng dữ liệu độc lập hoạt động song song.

Luồng thứ nhất là kênh điều khiển và telemetry giữa firmware trên ROV và GCS sử dụng giao thức MAVLink truyền qua UDP. Kênh này đảm nhiệm việc truyền lệnh điều khiển từ GCS đến ROV và truyền trạng thái cảm biến từ ROV ngược về GCS.

Luồng thứ hai là dữ liệu video từ camera gắn trên ROV. Camera kết nối trực tiếp với board QCS8550, hình ảnh được mã hóa JPEG và truyền liên tục về GCS dưới dạng luồng MJPEG thông qua giao thức TCP nhằm đảm bảo độ trễ thấp trong quá trình quan sát.

Luồng thứ ba bao gồm các thiết bị cảm biến độc lập như sonar quét hình ảnh và cảm biến định vị vận tốc DVL. Các thiết bị này giao tiếp mạng độc lập và truyền dữ liệu trực tiếp đến GCS mà không đi qua firmware điều khiển chính trên board QCS8550.

Kiến trúc phân tách này giúp giảm tải cho bộ xử lý nhúng trung tâm, đồng thời

cho phép các thiết bị chuyên dụng như sonar và DVL hoạt động song song với hệ thống điều khiển mà không ảnh hưởng đến vòng lặp điều khiển thời gian thực của ROV.

3.2 Kiến trúc tổng thể hệ thống

Hệ thống phần mềm được tổ chức thành ba thành phần độc lập bao gồm:

- Firmware nhúng trên ROV
- Phần mềm Ground Control Station
- Hệ thống lưu trữ và đồng bộ dữ liệu sau nhiệm vụ

Firmware nhúng chịu trách nhiệm điều khiển phương tiện và xử lý phần cứng gắn trực tiếp trên ROV. Ground Control Station đảm nhiệm giám sát, điều khiển từ xa, ghi nhận dữ liệu nhiệm vụ và hiển thị giao diện người dùng. Sau khi nhiệm vụ kết thúc, dữ liệu thu thập được có thể được lưu trữ cục bộ hoặc đồng bộ lên hệ thống web server phục vụ quản lý và phân tích từ xa.

3.3 Bảng đặc tả chức năng

Hệ thống phần mềm được thiết kế nhằm đáp ứng bảy nhóm chức năng chính, tương ứng với các nhu cầu vận hành thực tế trong môi trường dưới nước.

- Điều khiển bằng tay
- Điều khiển bán tự động
- Thu thập và xử lý dữ liệu cảm biến
- Hiển thị dữ liệu thời gian thực
- Lưu trữ dữ liệu nhiệm vụ
- Đồng bộ dữ liệu lên hệ thống web
- Giám sát trạng thái và đảm bảo an toàn hệ thống

3.3.1 Điều khiển bằng tay

Người vận hành điều khiển trực tiếp phương tiện thông qua thiết bị Joystick kết nối với Ground Control Station. Dữ liệu điều khiển được gửi liên tục đến firmware ROV thông qua giao thức MAVLink.

Chức năng	Đầu vào	Kết quả
Di chuyển 6 bậc tự do	Joystick axes (forward, lateral, ascend, yaw)	ROV di chuyển đúng hướng với thời gian phản hồi nhỏ hơn 200 ms
Điều khiển đèn	Nút bật tắt, tăng giảm độ sáng	Đèn thay đổi trạng thái theo lệnh điều khiển
Điều khiển camera	Nút ngẩng, hạ, đưa về vị trí giữa	Servo camera thay đổi góc nghiêng tương ứng
Điều khiển gripper	Nút đóng mở	Cơ cấu gripper thực hiện thao tác gấp hoặc nhả vật thể
Điều chỉnh công suất	Slider Joystick	Hệ số <i>ESC_GAIN</i> thay đổi, hiển thị trên GCS

Bảng 3.1: Đặc tả chức năng điều khiển bằng tay

3.3.2 Điều khiển tự động

Hệ thống hỗ trợ các chế độ điều khiển hỗ trợ nhằm giảm tải thao tác cho người vận hành trong môi trường có dòng chảy hoặc khi cần duy trì trạng thái ổn định trong thời gian dài.

a, Giữ hướng (Hold Heading)

Mô tả	Duy trì góc heading (yaw) của ROV tại giá trị tại thời điểm kích hoạt, bù nhiễu dòng nước hoặc lực không đối xứng từ tether.
Đầu vào	Lệnh kích hoạt từ Joystick hoặc giao diện GCS (mode = 1). Góc yaw hiện tại từ bộ lọc EKF.
Xử lý	Bộ điều khiển PID tính sai số góc và hiệu chỉnh thành phần yaw trong vector điều khiển.
Kết quả	ROV giữ nguyên hướng khi không có lệnh xoay từ Joystick; người vận hành vẫn điều khiển tự do các trục còn lại.

Bảng 3.2: Đặc tả chức năng giữ hướng

b, Giữ độ sâu (Hold Depth)

Mô tả	Duy trì độ sâu của ROV tại giá trị tại thời điểm kích hoạt, bù lực nổi và nhiễu dòng nước theo chiều thẳng đứng.
Đầu vào	Lệnh kích hoạt (mode = 4). Độ sâu hiện tại từ cảm biến áp suất MS5837.
Xử lý	Bộ điều khiển PID tính sai số độ sâu và hiệu chỉnh thành phần ascend trong vector điều khiển.
Kết quả	ROV duy trì độ sâu ổn định; người vận hành điều khiển tự do các trục forward, lateral, yaw.

Bảng 3.3: Đặc tả chức năng giữ độ sâu

c, Giữ vị trí theo DVL

Mô tả	Duy trì vị trí ngang (X, Y) của ROV dựa trên dữ liệu vận tốc từ cảm biến DVL (Doppler Velocity Log), bù trôi dạt do dòng nước.
Đầu vào	Lệnh kích hoạt từ GCS. Vận tốc tức thời từ DVL, tích phân tính vị trí tương đối.
Xử lý	Bộ điều khiển PID vị trí tính sai lệch X, Y và hiệu chỉnh thành phần forward, lateral trong vector điều khiển.
Kết quả	ROV giữ nguyên vị trí ngang trong môi trường có dòng chảy.
Trạng thái	Đang phát triển.

Bảng 3.4: Đặc tả chức năng giữ vị trí theo DVL

d, Giữ vị trí theo Vision

Mô tả	Theo dõi và bám theo mục tiêu xác định trong khung hình camera bằng thuật toán Optical Flow, điều khiển ROV giữ mục tiêu ở trung tâm khung hình (Track and Catch).
Đầu vào	Lệnh kích hoạt và vùng chọn mục tiêu từ GCS. Luồng hình ảnh MJPEG từ camera ROV.
Xử lý	Optical Flow tính vector dịch chuyển của mục tiêu giữa các khung hình, chuẩn hóa sai lệch ảnh thành lệnh điều khiển forward/lateral/yaw.
Kết quả	ROV tự động di chuyển để giữ mục tiêu ở trung tâm khung hình.
Trạng thái	Đang phát triển (Chương 6).

Bảng 3.5: Đặc tả chức năng giữ vị trí theo Vision

3.3.3 Hiển thị dữ liệu thời gian thực từ ROV

Ground Control Station tiếp nhận nhiều luồng dữ liệu song song và hiển thị trực tiếp trên giao diện điều khiển phục vụ quá trình vận hành.

Chức năng	Mô tả
Xem video trực tiếp	Nhận và hiển thị luồng MJPEG từ camera IP trên ROV với độ trễ thấp trên giao diện GCS.
Hiển thị Sonar	Nhận và hiển thị dữ liệu quét Sonar dưới dạng hình ảnh sector scan theo thời gian thực.
Hiển thị DVL	Nhận và hiển thị dữ liệu từ cảm biến DVL bao gồm vận tốc tương đối, độ cao so với đáy và các thông tin định vị hỗ trợ điều hướng ROV theo thời gian thực.
Hiển thị log vận hành	Thu thập và hiển thị trực quan các thông số vận hành do ROV gửi về theo thời gian thực như góc định hướng (roll, pitch, yaw), chế độ điều khiển, trạng thái đèn, camera, trạng thái pin và các dữ liệu giám sát khác nhằm phục vụ theo dõi và phân tích.

Bảng 3.6: Đặc tả chức năng hiển thị ROV

3.3.4 Lưu trữ dữ liệu từ ROV

Trong suốt quá trình vận hành, toàn bộ dữ liệu được ghi lại phục vụ việc xem lại và phân tích sau nhiệm vụ.

Chức năng	Mô tả
Lưu trữ video	Thu nhận và lưu lại luồng video MJPEG từ camera gắn trên ROV dưới dạng tệp video nhằm phục vụ việc xem lại và phân tích sau quá trình vận hành.
Lưu trữ dữ liệu Sonar	Ghi nhận và lưu trữ dữ liệu quét Sonar trong suốt quá trình hoạt động của ROV, phục vụ tái hiện hình ảnh môi trường và phân tích dữ liệu sau nhiệm vụ.
Lưu trữ dữ liệu DVL	Thu thập và lưu trữ dữ liệu từ cảm biến DVL bao gồm vận tốc tương đối, độ cao so với đáy và các thông tin hỗ trợ định vị nhằm phục vụ phân tích quá trình điều hướng của ROV.
Lưu trữ log vận hành	Thu thập và lưu trữ các thông số vận hành do ROV gửi về theo thời gian thực như góc định hướng (roll, pitch, yaw), chế độ điều khiển, trạng thái đèn, camera, trạng thái pin và các dữ liệu giám sát khác nhằm phục vụ theo dõi và phân tích sau nhiệm vụ.
Trực quan hóa dữ liệu sau lưu trữ	Hỗ trợ tái tạo và hiển thị trực quan dữ liệu Sonar và dữ liệu từ cảm biến DVL sau khi quá trình ghi kết thúc, giúp người vận hành phân tích môi trường, đánh giá quỹ đạo di chuyển và xem lại dữ liệu thu thập được sau nhiệm vụ.

Bảng 3.7: Đặc tả chức năng lưu trữ dữ liệu từ ROV

3.3.5 Lưu trữ dữ liệu lên web

Sau khi ROV về bờ, Operator tải dữ liệu thủ công lên hệ thống web quản lý thông tin. Hệ thống web không nhận dữ liệu trực tiếp trong khi ROV hoạt động ngoài thực địa do không có kết nối mạng. Bảng ?? mô tả các nhóm chức năng chính của hệ thống.

Nhóm chức năng	Mô tả
Tải lên dữ liệu	Operator upload file CSV cảm biến, DVL JSON, sonar binary và video/ảnh lên REST API. Dữ liệu số lưu vào MongoDB Atlas; video/ảnh lưu trên AWS S3 thông qua presigned URL.
Quản lý ROV, Project, Trip	CRUD và theo dõi vòng đời trạng thái (<i>pending</i> → <i>running</i> → <i>done/failed</i>) của ROV, chuyến khảo sát (Project) và lượt lặn (Trip). Phân quyền RBAC ba cấp: viewer, operator, admin.
Xem lại và phân tích dữ liệu	Biểu đồ cảm biến (độ sâu, nhiệt độ, áp suất); bản đồ GPS Leaflet; trình phát sonar; quỹ đạo DVL; đồng bộ video—biểu đồ theo timestamp recordedAt.
Phát hiện bất thường	Tự động tính Z-Score ($ z > 2,5$) cho từng chỉ số cảm biến; highlight điểm bất thường trên biểu đồ.
Nhận diện vật thể AI	Phân tích ảnh/video bằng YOLOv8 qua microservice Python FastAPI; kết quả trả về bất đồng bộ qua Bull Queue và SSE; hiển thị bounding box đồng bộ theo từng frame video.
Tóm tắt nhiệm vụ AI	Tạo báo cáo tóm tắt ngôn ngữ tự nhiên bằng Gemini 2.5 Flash khi Project hoàn tất; lưu vào DB và thông báo tức thì qua SSE.
Hệ thống bằng chứng	Operator chụp frame ảnh hoặc đánh dấu đoạn clip từ video làm bằng chứng; có thể phân tích YOLO riêng từng bằng chứng mà không cần phân tích toàn bộ video.
Thông báo realtime	SSE push thông báo khi Trip hoàn tất, YOLO phân tích xong, AI summary sẵn sàng; không cần reload trang.

Bảng 3.8: Đặc tả chức năng hệ thống web quản lý thông tin ROV

3.4 Thiết kế module

3.4.1 Thiết kế module firmware trên ROV

Firmware nhúng chạy trực tiếp trên board xử lý trung tâm của ROV, chịu trách nhiệm điều khiển thời gian thực, giao tiếp với phần cứng ngoại vi, thu thập dữ liệu cảm biến, xử lý thuật toán điều khiển và truyền dữ liệu telemetry về Ground Control Station.

Firmware được triển khai trên nền tảng QCS8550 chạy hệ điều hành Linux Embedded và được phát triển chủ yếu bằng ngôn ngữ Python. Kiến trúc phần mềm được thiết kế theo mô hình đa luồng nhằm cho phép các tác vụ điều khiển, đọc cảm

biến, truyền dữ liệu và xử lý camera hoạt động song song mà không gây chặn lẫn nhau.

Các module firmware được chia thành bảy nhóm chức năng chính.

- Nhóm module cảm biến
- Nhóm module điều khiển
- Nhóm module cơ cấu chấp hành
- Nhóm module camera
- Nhóm module truyền thông
- Nhóm module telemetry
- Nhóm module cấu hình và an toàn hệ thống

a, Nhóm module cảm biến

Nhóm module cảm biến chịu trách nhiệm giao tiếp trực tiếp với các thiết bị đo lường gắn onboard trên ROV. Mỗi cảm biến được vận hành trên một luồng độc lập nhằm đảm bảo dữ liệu luôn được cập nhật liên tục với tần suất ổn định.

Module	Giao tiếp	Chức năng
MS5837	I2C bus 3	Đo áp suất nước, tính độ sâu và nhiệt độ nước. Cập nhật <code>latest_depth</code> , <code>WaterTemp</code>
EKF_ICM	SPI bus 0	Đọc IMU ICM20602 (gia tốc, con quay, từ kế), chạy bộ lọc Kalman mở rộng tính góc roll/pitch/yaw. Cập nhật <code>latest_roll</code> , <code>latest_pitch</code> , <code>latest_yaw</code>
Si7021	UART	Đo nhiệt độ và độ ẩm trong khoang kín ROV. Cập nhật <code>temp</code> , <code>humid</code>
ADS1115	I2C	Đọc ADC điện áp và dòng điện pin. Phục vụ tính SOC (state of charge)

Bảng 3.9: Các module cảm biến trong phần mềm ROV

b, Nhóm module điều khiển

Đây là thành phần trung tâm của firmware, chịu trách nhiệm tiếp nhận lệnh điều khiển từ Ground Control Station và chuyển đổi thành lệnh điều khiển cụ thể cho

các động cơ đẩy.

Toàn bộ logic điều khiển hoạt động theo mô hình vòng lặp thời gian thực, liên tục nhận dữ liệu cảm biến, tính toán thuật toán ổn định và cập nhật tín hiệu điều khiển.

Module	Chức năng
control_loop	Vòng lặp chính lắng nghe lệnh MAVLink từ GCS (cổng UDP 16001). Phân loại và điều phối đến các hàm xử lý tương ứng. Giám sát heartbeat, tự động chuyển về manual khi mất kết nối.
control_thruster	Nhận vector điều khiển 4 chiều $[fw, lat, asc, yaw]$, áp dụng PID giữ hướng/giữ độ sâu nếu đang kích hoạt, nhân với ma trận mixing 8×4 để tính lực 8 động cơ.
process_command	Xử lý lệnh chuyển chế độ (manual, hold heading, hold depth), chốt setpoint tại thời điểm kích hoạt.
auto_tune_pid	Tự động xác định hệ số PID tối ưu bằng phương pháp relay Ziegler–Nichols, áp dụng cho trục heading hoặc depth.

Bảng 3.10: Các module điều khiển trong phần mềm ROV

c, Nhóm module cơ cấu chấp hành

Toàn bộ cơ cấu chấp hành trên ROV đều giao tiếp thông qua IC PWM PCA9685 kết nối I2C, phát tín hiệu PWM chuẩn RC (50 Hz, 1000–2000 μs) đến từng thiết bị. Nhóm này gồm 4 loại thiết bị với đặc tính điều khiển khác nhau.

Thiết bị	Kênh PCA9685	Kiểu điều khiển	Dải PWM
8 động cơ (ESC)	Theo THRUSTER_MAP	Tốc độ liên tục	1100–1900 μs , trung tính 1500 μs
Đèn chiếu sáng	Kênh 15	Vị trí (độ sáng)	1100 μs (tắt) đến 1900 μs (sáng tối đa)
Camera tilt (servo)	Kênh 2	Vị trí (góc)	1100–1900 μs , tương ứng -45° đến $+45^\circ$
Gripper	Kênh 14	Vị trí (đóng/mở)	1200 μs (đóng), 1700 μs (mở)

Bảng 3.11: Các cơ cấu chấp hành và thông số PWM điều khiển

Điểm chung của tất cả thiết bị là PCA9685 duy trì tín hiệu PWM liên tục sau mỗi lần ghi, nghĩa là phần mềm chỉ cần ghi lại khi có thay đổi trạng thái thay vì phải ghi lặp lại mỗi vòng lặp. Điểm khác biệt giữa động cơ và các thiết bị còn lại là kiểu điều khiển: ESC nhận PWM thay đổi liên tục theo lệnh Joystick (điều khiển tốc độ), trong khi đèn, servo camera và gripper chỉ thay đổi khi người vận hành bấm nút (điều khiển vị trí).

d, Nhóm module camera

Việc sử dụng MJPEG streaming cho phép giảm độ trễ hiển thị và đơn giản hóa quá trình giải mã video phía Ground Control Station.

Module	Cổng	Chức năng
mjpeg_server	TCP 8080	Nhận từng dữ liệu từ camera sau đó gửi từng frame lên cho phần mềm GCS

Bảng 3.12: Các module Camera trong phần mềm ROV

e, Nhóm module truyền thông

Nhóm này quản lý toàn bộ kết nối mạng giữa ROV và GCS, gồm hai chiều độc lập: nhận lệnh và phát telemetry.

Module	Cổng	Chức năng
ping_server	UDP 15000	Lắng nghe ping từ GCS để nhận diện địa chỉ IP động, khởi tạo 5 kênh MAVLink gửi ngược về GCS.
udp_config_listener	UDP 12345	Nhận lệnh cấu hình động cơ (PING, SAVE_CONFIG) từ GCS, tách biệt hoàn toàn khỏi luồng MAVLink.
control_loop	UDP 16001	Nhận gói MANUAL_CONTROL và SET_MODE từ GCS.

Bảng 3.13: Các module truyền thông trong phần mềm ROV

f, Nhóm module telemetry

Nhóm này đóng gói dữ liệu cảm biến và trạng thái hệ thống thành các gói tin MAVLink gửi định kỳ về GCS mỗi 1 giây qua 5 kênh UDP riêng biệt (16001–16005).

Hàm	Kênh	Dữ liệu gửi
send_heartbeat	16001	Trạng thái hoạt động của ROV (MAV_STATE_ACTIVE).
send_sys_status	16002	Điện áp/dòng pin, chế độ hiện tại, và các giá trị NAMED_VALUE_FLOAT: nhiệt độ, độ ẩm, độ sâu, mức đèn, góc camera, công suất, trạng thái chế độ.
send_attitude	16005	Góc roll/pitch/yaw, SOC pin, độ sâu hiện tại.
send_imu	16004	Dữ liệu thô gia tốc kế, con quay hồi chuyển, từ kế.
send_global_position	16003	Tọa độ vị trí (hiện dùng tọa độ tĩnh, dự phòng cho DVL/GPS).

Bảng 3.14: Các hàm telemetry và kênh gửi dữ liệu

g, Nhóm module cấu hình

Module	Chức năng
load_thrust_config	Đọc file <code>thruster_config.json</code> khi khởi động, nạp ánh xạ kênh vật lý PCA9685 vào THRUSTER_MAP. Nếu file không tồn tại, giữ nguyên giá trị mặc định.
PCA9685	Driver IC PWM 16 kênh, giao tiếp I2C. Phục vụ toàn bộ cơ cấu chấp hành: 8 ESC động cơ, đèn, camera tilt, gripper.

Bảng 3.15: Module cấu hình và driver chấp hành

3.4.2 Thiết kế module phần mềm GCS

Ground Control Station (GCS) là phần mềm trung tâm phục vụ điều khiển và giám sát toàn bộ hệ thống ROV trong quá trình vận hành. Phần mềm được triển khai trên máy tính của người vận hành, phát triển bằng ngôn ngữ C sử dụng framework WPF nhằm xây dựng giao diện trực quan và hỗ trợ xử lý đa luồng hiệu quả.

Khác với firmware nhúng trên ROV vốn tập trung vào xử lý thời gian thực, phần mềm GCS đảm nhiệm đồng thời nhiều chức năng phức tạp bao gồm điều khiển phương tiện, hiển thị dữ liệu từ nhiều nguồn độc lập, ghi nhận dữ liệu nhiệm vụ, quản lý kết nối mạng và đồng bộ dữ liệu sau khi vận hành.

Kiến trúc phần mềm được thiết kế theo hướng module hóa, trong đó mỗi thành phần được triển khai dưới dạng một service độc lập. Các service giao tiếp với nhau thông qua cơ chế sự kiện (event-driven architecture) và callback bất đồng bộ nhằm giảm phụ thuộc giữa các module.

Toàn bộ hệ thống phần mềm GCS được chia thành sáu nhóm module chính:

- Nhóm module truyền thông
- Nhóm module điều khiển
- Nhóm module xử lý dữ liệu thời gian thực
- Nhóm module ghi dữ liệu nhiệm vụ
- Nhóm module quản lý hệ thống
- Nhóm module giao diện người dùng

3.4.3 Kiến trúc tổng thể phần mềm GCS

Phần mềm Ground Control Station tiếp nhận đồng thời ba luồng dữ liệu độc lập hoạt động song song trong quá trình vận hành.

Luồng dữ liệu thứ nhất là kênh điều khiển và telemetry giữa GCS và firmware trên ROV sử dụng giao thức MAVLink chạy trên UDP. Kênh này cho phép gửi lệnh điều khiển theo thời gian thực và nhận trạng thái hệ thống từ ROV.

Luồng dữ liệu thứ hai là dữ liệu video từ camera gắn trên ROV. Camera được xử lý trực tiếp bởi board nhúng trên ROV và truyền về GCS thông qua luồng MJPEG chạy trên TCP.

Luồng dữ liệu thứ ba là dữ liệu từ các thiết bị ngoại vi độc lập bao gồm sonar và cảm biến DVL. Các thiết bị này giao tiếp trực tiếp với GCS thông qua kết nối mạng riêng mà không đi qua firmware điều khiển chính.

Phần mềm GCS cần xử lý đồng thời toàn bộ các luồng dữ liệu này, hiển thị trực tiếp trên giao diện, đồng thời ghi lại toàn bộ dữ liệu nhiệm vụ phục vụ phân tích sau vận hành.

3.4.4 Nhóm module truyền thông

Nhóm truyền thông đóng vai trò trung tâm của toàn bộ hệ thống GCS, chịu trách nhiệm thiết lập và duy trì kết nối với các thiết bị ngoài.

a, MavlinkService

MavlinkService là module quan trọng nhất trong hệ thống GCS, đóng vai trò cầu nối duy nhất giữa phần mềm điều khiển và firmware nhúng trên ROV.

Service này chịu trách nhiệm khởi tạo các socket UDP phục vụ gửi và nhận gói MAVLink. Hệ thống sử dụng nhiều cổng UDP độc lập nhằm tách riêng các luồng telemetry.

Module duy trì các luồng nền riêng biệt để nhận dữ liệu song song từ nhiều socket khác nhau, giải mã gói MAVLink và phát sự kiện tới các service khác trong hệ thống.

Chức năng	Mô tả
Khởi tạo UDP sockets	Mở các cổng giao tiếp MAVLink
SendManualControl	Gửi dữ liệu joystick đến firmware ROV
SendSetMode	Chuyển đổi chế độ điều khiển
ReceiveTelemetry	Nhận dữ liệu telemetry từ ROV
PacketDecoder	Giải mã các gói MAVLink nhận được
HeartbeatMonitor	Theo dõi trạng thái kết nối với ROV
EventDispatcher	Phát event tới các service khác

Bảng 3.16: Chức năng của MavlinkService

Các service khác không giao tiếp trực tiếp với ROV mà bắt buộc phải đi thông qua MavlinkService. Cách thiết kế này giúp tập trung toàn bộ logic mạng vào một điểm duy nhất, đơn giản hóa việc bảo trì hệ thống.

b, CameraService

CameraService chịu trách nhiệm tiếp nhận luồng video MJPEG từ camera trên ROV.

Module thiết lập kết nối TCP đến HTTP MJPEG server đang chạy trên board QCS8550, liên tục nhận dữ liệu JPEG, giải mã thành từng khung hình và cập nhật cho giao diện hiển thị video theo thời gian thực.

Do video là luồng dữ liệu có tốc độ cao, module sử dụng một luồng nền riêng biệt để tránh ảnh hưởng đến các service khác.

Kết nối TCP	Kết nối đến MJPEG server trên ROV
Frame Parser	Tách boundary HTTP multipart stream
JPEG Decoder	Giải mã dữ liệu JPEG thành image frame
Frame Event	Phát sự kiện frame mới tới giao diện
Reconnect Handler	Tự động kết nối lại khi camera bị mất tín hiệu

Bảng 3.17: Chức năng của CameraService

c, SonarService

SonarService chịu trách nhiệm giao tiếp trực tiếp với sonar Ping360.

Do sonar không đi qua firmware trên ROV, service này kết nối trực tiếp với thiết bị sonar thông qua giao thức mạng độc lập.

Module liên tục nhận dữ liệu sector scan, giải mã dữ liệu phản hồi và chuyển đổi thành dữ liệu phục vụ hiển thị trực tiếp trên giao diện.

Socket Connection	Thiết lập kết nối trực tiếp với sonar
Packet Decoder	Giải mã packet dữ liệu sonar
Scan Reconstruction	Khôi phục dữ liệu sector scan
Image Renderer	Chuyển dữ liệu sonar thành hình ảnh hiển thị
Frame Event	Phát dữ liệu scan mới cho UI

Bảng 3.18: Chức năng của SonarService

d, DVLService

DVLService chịu trách nhiệm nhận dữ liệu trực tiếp từ cảm biến DVL.

Module liên tục nhận dữ liệu vận tốc tương đối theo các trục, độ cao so với đáy và các thông tin định vị phục vụ giám sát quá trình vận hành.

Connection Manager	Thiết lập kết nối đến thiết bị DVL
Packet Parser	Giải mã dữ liệu vận tốc và altitude
Data Normalization	Chuẩn hóa dữ liệu về cấu trúc nội bộ
Velocity Event	Phát dữ liệu mới cho hệ thống
Status Monitoring	Theo dõi trạng thái hoạt động của DVL

Bảng 3.19: Chức năng của DVLService

3.4.5 Nhóm module điều khiển

a, JoystickService

JoystickService chịu trách nhiệm đọc trạng thái thiết bị Joystick kết nối với máy tính điều khiển.

Module sử dụng DirectInput để nhận trạng thái các trục điều khiển và nút bấm, chuẩn hóa dữ liệu và gửi lệnh điều khiển định kỳ tới MavlinkService.

Dữ liệu joystick được đọc liên tục với chu kỳ khoảng 100 ms nhằm đảm bảo khả năng điều khiển thời gian thực.

Device Detection	Phát hiện joystick khi cắm vào hệ thống
Axis Reading	Đọc dữ liệu các trục điều khiển
Button Mapping	Ánh xạ nút bấm sang chức năng điều khiển
Normalize Input	Chuẩn hóa giá trị điều khiển về dải nội bộ
Send Command	Gửi dữ liệu tới MavlinkService
Reconnect Device	Kết nối lại khi joystick bị ngắt

Bảng 3.20: Chức năng của JoystickService

3.4.6 Nhóm module xử lý dữ liệu thời gian thực

a, TelemetryService

TelemetryService đăng ký các event từ MavlinkService và cập nhật dữ liệu telemetry phục vụ giao diện người dùng.

Service này chuyển đổi dữ liệu MAVLink sang các ViewModel tương ứng của WPF.

Do WPF chỉ cho phép cập nhật UI từ UI thread, module sử dụng `Dispatcher.BeginInvoke` để chuyển dữ liệu từ background thread sang giao diện.

Event Subscription	Đăng ký các sự kiện từ MavlinkService
Telemetry Parser	Phân loại dữ liệu telemetry
ViewModel Update	Cập nhật dữ liệu vào ViewModel
UI Dispatching	Đồng bộ thread với giao diện WPF
PropertyChanged Trigger	Kích hoạt cập nhật UI tự động

Bảng 3.21: Chức năng của TelemetryService

3.4.7 Nhóm module ghi dữ liệu nhiệm vụ

Nhóm recording service chịu trách nhiệm ghi lại toàn bộ dữ liệu phát sinh trong suốt quá trình vận hành phục vụ việc phân tích sau nhiệm vụ.

a, CameraRecorderService

CameraRecorderService chịu trách nhiệm ghi lại luồng video nhận được từ CameraService.

Do video cần được lưu trữ chính xác theo timestamp thực tế, module lưu tạm từng khung hình riêng biệt kèm timestamp tương ứng trước khi thực hiện ghép thành file video hoàn chỉnh.

Hệ thống chia video thành các đoạn có thời lượng cố định, ví dụ mười phút cho mỗi file nhằm tránh tạo ra các file quá lớn trong nhiệm vụ dài.

Sau khi quá trình ghi kết thúc, service thực hiện chuyển đổi dữ liệu thô thành

file MP4 và xóa dữ liệu trung gian.

Frame Queue	Nhận frame từ CameraService
Timestamp Logger	Ghi timestamp cho từng frame
Segment Manager	Chia video thành từng đoạn cố định
Background Converter	Ghép dữ liệu thành MP4 ở background thread
Cleanup Manager	Xóa dữ liệu raw sau khi convert thành công

Bảng 3.22: Chức năng CameraRecorderService

b, SonarRecorder

Module ghi nhận toàn bộ dữ liệu sonar thô phục vụ replay hoặc tái dựng môi trường sau khi nhiệm vụ kết thúc.

Dữ liệu được lưu kèm timestamp đồng bộ với các nguồn dữ liệu khác.

c, DVLRecorder

Module ghi nhận toàn bộ dữ liệu vận tốc, altitude và trạng thái cảm biến DVL theo thời gian thực.

d, TelemetryRecorder

Module ghi lại toàn bộ dữ liệu telemetry nhận được từ firmware ROV bao gồm orientation, depth, battery status và trạng thái điều khiển.

e, MasterRecordingService

MasterRecordingService là module điều phối trung tâm cho toàn bộ quá trình ghi dữ liệu.

Thay vì để từng service hoạt động độc lập, module này chịu trách nhiệm đồng bộ việc bắt đầu và dừng ghi trên tất cả nguồn dữ liệu.

Khi người vận hành bắt đầu mission recording, service này đồng thời kích hoạt:

- CameraRecorderService
- SonarRecorder
- DVLRecorder
- TelemetryRecorder

Toàn bộ dữ liệu được lưu vào cùng một thư mục nhiệm vụ nhằm đảm bảo dễ dàng truy xuất và đồng bộ timestamp giữa các nguồn.

3.4.8 Nhóm module quản lý hệ thống

a, LogService

Module chịu trách nhiệm ghi log hoạt động của toàn bộ hệ thống phục vụ debug và theo dõi trạng thái vận hành.

Các thông tin được lưu bao gồm trạng thái kết nối, lỗi mạng, lỗi thiết bị ngoại vi và các sự kiện bất thường trong quá trình hoạt động.

b, WebUploadService

Sau khi nhiệm vụ kết thúc, dữ liệu đã ghi nhận được nén và tải lên web server.

Module chịu trách nhiệm:

- Upload video
- Upload sonar data
- Upload DVL data
- Upload telemetry logs
- Tạo metadata cho từng mission

Module gọi REST API của hệ thống web quản lý thông tin. Phía web tiếp nhận và lưu trữ: video/ảnh vào AWS S3 thông qua presigned URL; dữ liệu cảm biến, DVL, sonar vào MongoDB Atlas liên kết với Trip tương ứng.

3.4.9 Nhóm module giao diện người dùng

Tầng giao diện được xây dựng bằng WPF theo mô hình MVVM.

Mỗi thành phần dữ liệu trong hệ thống tương ứng với một ViewModel riêng.

Các ViewModel đăng ký dữ liệu từ service tương ứng và tự động cập nhật giao diện thông qua cơ chế `INotifyPropertyChanged`.

Giao diện bao gồm:

- Camera view
- Sonar display
- DVL dashboard
- Telemetry dashboard
- Joystick status monitor
- Recording control panel
- Mission management panel

- System log viewer

3.4.10 Thiết kế module hệ thống quản lý thông tin web

Hệ thống web được tổ chức theo kiến trúc ba tầng: Frontend (React SPA), Backend API (Node.js/Express), và tầng dữ liệu (MongoDB Atlas, Redis, AWS S3). Tác vụ nặng như phân tích AI và nhận diện vật thể được xử lý bất đồng bộ qua Bull Queue, không block HTTP request.

a, Nhóm module Backend API

Mỗi tài nguyên tương ứng với một module Express gồm `model`, `controller`, `service` và `routes`. Tất cả response tuân theo cấu trúc thống nhất `{ success, message, data }`.

Module	Chức năng
<code>auth</code>	Đăng ký, đăng nhập, đăng xuất, refresh token, Google OAuth2. JWT access 15 phút + refresh 7 ngày; Redis blacklist khi logout.
<code>rovs</code>	CRUD ROV; liên kết với projects.
<code>projects</code>	CRUD chuyến khảo sát; filter theo status, ROV, thời gian; enqueue AI summary khi hoàn tất.
<code>trips</code>	CRUD lượt lặn; cascade delete toàn bộ dữ liệu liên quan (sensor, DVL, sonar, media, snapshot, S3) khi xóa.
<code>sensor / dvl / sonar</code>	Upload và truy vấn dữ liệu cảm biến, DVL, sonar theo Trip; hỗ trợ nhiều file per lượt lặn; phát hiện bất thường Z-Score.
<code>media</code>	Upload qua presigned URL → S3; confirm; gallery; enqueue phân tích YOLO.
<code>snapshots</code>	Lưu ảnh frame và clip bằng chứng từ video; phân tích YOLO riêng theo range thời gian.
<code>notifications</code>	SSE stream per user; CRUD thông báo; mark as read.
<code>ai</code>	Worker Gemini 2.5 Flash: tóm tắt Project; lưu <code>aiSummary</code> ; push SSE.
<code>audit</code>	Ghi log thao tác quan trọng (tạo/xóa/đổi quyền); truy vấn dành riêng cho admin.
<code>stats</code>	Aggregation MongoDB song song: tổng hợp Project, Trip, ROV, User; timeline 6 tháng.

Bảng 3.23: Các module Backend API của hệ thống web

b, Nhóm module Frontend

Giao diện xây dựng bằng React 18 + Vite + Tailwind CSS theo kiến trúc SPA. Zustand quản lý trạng thái xác thực; React Query quản lý server state và cache.

Trang / Component	Chức năng
Dashboard	Stat cards + biểu đồ trạng thái Project/Trip/ROV timeline hoạt động 6 tháng.
RovsPage / RovDetailPage	Danh sách, tạo/sửa/xóa ROV; lịch sử Project kết.
ProjectsPage / ProjectDetailPage	Quản lý Project; danh sách Trip con; AI Summary; Media gallery.
TripDetailPage	Layout cockpit 3 cột: map GPS + KPI — video player — gauge điều hướng; biểu đồ cảm biến; sonar player; YOLO bbox overlay; evidence system.
UsersPage	Quản lý người dùng (admin): tìm kiếm, đổi mật khẩu, bật/tắt tài khoản, bulk action.
AuditPage	Xem lịch sử thao tác hệ thống (admin); filter entity.
ProfilePage	Đổi thông tin cá nhân, mật khẩu; ẩn tab đổi mật khẩu với Google user.
NotificationBell	Bell icon + badge đếm chưa đọc; SSE real-time dropdown 10 thông báo gần nhất.

Bảng 3.24: Các trang và component Frontend của hệ thống web

c, Nhóm module xử lý bất đồng bộ

Tác vụ nặng (AI, phân tích video) được enqueue vào Bull Queue thay vì xử lý trong HTTP request, đảm bảo phản hồi tức thì (HTTP 202) và không block giao diện.

Module	Chức năng
<code>ai.worker</code>	Nhận job từ queue <code>ai-summary</code> ; gọi Gemini 2.5 Flash API với prompt tóm tắt Project; lưu kết quả vào <code>project.aiSummary</code> ; push SSE thông báo đến Operator.
<code>media.worker</code>	Nhận job từ queue <code>media-analysis</code> ; gọi YOLO microservice <code>POST /detect</code> ; lưu labels, bbox, frameTime, trackId vào Media document; push SSE khi xong.
<code>snapshot.worker</code>	Tương tự <code>media.worker</code> nhưng xử lý Snapshot (ảnh frame hoặc clip bằng chứng); hỗ trợ range <code>[startTime, endTime]</code> để YOLO chỉ phân tích đoạn video được chọn.
YOLO microservice	FastAPI Python độc lập (port 8000); nhận URL presigned + tên model + ngưỡng confidence; xử lý ảnh hoặc video với frame sampling thích nghi; trả về labels, bbox, frameTime, trackId.

Bảng 3.25: Các module xử lý bất đồng bộ của hệ thống web

3.5 Quan hệ giữa các module

Kiến trúc phần mềm tuân theo nguyên tắc phân tầng một chiều.

Các service nhận dữ liệu thiết bị hoạt động độc lập và không giao tiếp trực tiếp với nhau.

Toàn bộ điều khiển ROV phải đi qua `MavlinkService`.

Toàn bộ recording phải được điều phối bởi `MasterRecordingService`.

Giao diện người dùng không truy cập trực tiếp network layer mà chỉ làm việc thông qua `ViewModel` và service trung gian.

Cách thiết kế này giúp hệ thống dễ dàng mở rộng thêm thiết bị mới trong tương lai mà không làm ảnh hưởng đến kiến trúc tổng thể.

CHƯƠNG 4. XÂY DỰNG PHẦN MỀM TRÊN ROV

Triển khai mọi module

4.1 Kiến trúc tổng thể và luồng dữ liệu

4.2 Module cảm biến

4.2.1 Đo độ sâu và nhiệt độ nước (MS5837)

Cảm biến MS5837-30BA là cảm biến áp suất độ phân giải cao giao tiếp qua I2C bus 3, đo áp suất nước để tính độ sâu và đồng thời đo nhiệt độ nước. Dữ liệu được cập nhật liên tục vào hai biến toàn cục `latest_depth` và `WaterTemp` bởi luồng nền `depth_reader()` chạy ở tần suất 20 Hz.

Nguyên lý đo Cảm biến lưu 6 hệ số hiệu chuẩn C_1-C_6 trong PROM nội tại, được đọc một lần khi khởi tạo và xác minh bằng CRC-4. Mỗi chu kỳ đọc, cảm biến thực hiện hai lần chuyển đổi ADC 24-bit: D_1 cho áp suất và D_2 cho nhiệt độ thô. Từ hai giá trị này, thư viện driver áp dụng bù nhiễu bậc hai (second order compensation) để đạt độ chính xác cao trong dải nhiệt độ thấp điển hình của môi trường dưới nước.

Độ sâu được tính từ áp suất hiệu chuẩn theo công thức:

$$d = \frac{P - P_{atm}}{\rho \cdot g} \quad (4.1)$$

trong đó P là áp suất đo được (Pa), $P_{atm} = 101300 \text{ Pa}$ là áp suất khí quyển chuẩn, $\rho = 1029 \text{ kg/m}^3$ là mật độ nước biển và $g = 9.80665 \text{ m/s}^2$.

Thiết kế tích hợp Mức oversampling được chọn là `OSR_8192` cho độ phân giải tốt nhất ($\approx 0.2 \text{ mbar}$, tương đương $\approx 2 \text{ mm}$ độ sâu) với thời gian chuyển đổi $\approx 18 \text{ ms}$ mỗi lần đọc, vẫn đáp ứng tốt chu kỳ 50 ms của luồng. Mật độ chất lỏng được cấu hình là nước biển (1029 kg/m^3) để phù hợp với môi trường vận hành thực tế.

Nếu cảm biến không khởi tạo được (CRC thất bại hoặc không phản hồi I2C), hệ thống bật `exit_flag` để dừng toàn bộ tiến trình thay vì chạy tiếp với dữ liệu sai — đây là quyết định thiết kế quan trọng vì `latest_depth` được dùng trực tiếp bởi bộ điều khiển PID giữ độ sâu. Trong vòng lặp đọc, cơ chế `try/except` với độ trễ 1 giây cho phép luồng tự phục hồi sau lỗi I2C thoáng qua mà không làm sập tiến trình.

Hai biến đầu ra `latest_depth` và `WaterTemp` được dùng chung bởi ba module: bộ điều khiển PID giữ độ sâu, module telemetry attitude và module telemetry status.

4.2.2 IMU ICM20602 (gia tốc, con quay, từ kế) + pitch,roll,yaw qua Extended Kalman filter

Module này chịu trách nhiệm cung cấp góc tư thế (roll, pitch, yaw) chính xác cho hệ thống điều khiển. Do dữ liệu thô từ cảm biến IMU chứa nhiễu và sai lệch tích lũy theo thời gian, hệ thống sử dụng bộ lọc Kalman mở rộng (Extended Kalman Filter — EKF) để tổng hợp dữ liệu từ ba nguồn cảm biến: gia tốc kế, con quay hồi chuyển và từ kế, cho ra góc tư thế ổn định và liên tục.

Phần cứng. ICM20602 là IMU 6 trục (gia tốc kế 3 trục + con quay hồi chuyển 3 trục) giao tiếp với SoC QCS8550 qua SPI bus 0. Từ kế 3 trục MMC5983 được tích hợp thêm để cung cấp thông tin hướng tuyệt đối (heading), cũng giao tiếp qua SPI. Cả hai cảm biến được đọc đồng thời trong cùng một luồng `heading_reader_ekf()` chạy liên tục.

Hiệu chuẩn. Trước khi vận hành, hệ thống thực hiện hiệu chuẩn con quay hồi chuyển bằng cách lấy trung bình 500 mẫu khi ROV đứng yên để xác định độ lệch zero (gyro bias) theo ba trục. Độ lệch này được trừ đi ở mỗi lần đọc, loại bỏ sai số hệ thống trước khi đưa vào bộ lọc EKF. Từ kế có thể được hiệu chuẩn thêm bằng cách thu thập dữ liệu theo nhiều hướng để xác định offset và scale theo từng trục, bù ảnh hưởng của từ trường cục bộ từ các thiết bị điện tử xung quanh.

Bộ lọc Kalman mở rộng. EKF hoạt động trong không gian quaternion thay vì góc Euler để tránh hiện tượng gimbal lock. Mỗi chu kỳ cập nhật, bộ lọc thực hiện hai bước:

- **Bước dự đoán:** sử dụng dữ liệu con quay hồi chuyển (tốc độ góc) tích phân theo thời gian dt để dự đoán quaternion tư thế tiếp theo.
- **Bước hiệu chỉnh:** sử dụng gia tốc kế (xác định hướng trọng lực) và từ kế (xác định hướng Bắc từ) để hiệu chỉnh sai lệch tích lũy từ bước dự đoán.

Quaternion đầu ra $\mathbf{q} = [w, x, y, z]$ được chuyển đổi sang góc Euler theo hệ tọa độ NED (North-East-Down) thông qua các công thức:

$$\phi = \arctan 2(2(wx + yz), w^2 - x^2 - y^2 + z^2) \quad (4.2)$$

$$\theta = -\arcsin(2(xz - wy)) \quad (4.3)$$

$$\psi = \arctan 2(2(xy + wz), w^2 + x^2 - y^2 - z^2) \quad (4.4)$$

trong đó ϕ, θ, ψ lần lượt là roll, pitch và yaw.

Thiết kế tích hợp. Ba biến toàn cục `latest_roll, latest_pitch, lat-`

`est_yaw` được cập nhật liên tục và dùng chung bởi hai module: bộ điều khiển PID giữ hướng (đọc `latest_yaw` làm phản hồi) và module telemetry attitude (gửi cả ba góc về GCS để hiển thị trên UserControl Compass). Biến `dt` được tính chính xác theo thời gian thực bằng `time.perf_counter()` thay vì giá trị cố định, đảm bảo tần số EKF không bị ảnh hưởng bởi độ trễ hệ thống.

4.2.3 Đo nhiệt độ và độ ẩm khoang kín (Si7021)

Cảm biến Si7021 được đặt bên trong khoang kín của ROV, có nhiệm vụ giám sát nhiệt độ và độ ẩm không khí trong ống nhằm phát hiện sớm hiện tượng ngưng nước — một trong những nguy cơ nghiêm trọng nhất có thể gây hỏng toàn bộ hệ thống điện tử bên trong ROV.

Kiến trúc phần cứng

Si7021 giao tiếp với vi điều khiển Arduino qua I2C. Arduino đóng vai trò trung gian: đọc dữ liệu từ Si7021, đóng gói thành chuỗi văn bản ASCII theo định dạng `nhiệt_độ, độ_ẩm` (ví dụ `28.5, 65.3`) rồi truyền sang SoC QCS8550 qua UART ở tốc độ `9600 baud` trên cổng `/dev/ttyHS3`. Thiết kế này giúp tách biệt hoàn toàn tầng thu thập dữ liệu cảm biến (Arduino + I2C) khỏi tầng xử lý chính (QCS8550), đồng thời tận dụng lợi thế của UART là giao tiếp đơn giản, ít nhiễu và không yêu cầu đồng bộ clock chặt chẽ như I2C trên đường dây dài trong khoang kín.

Thiết kế tích hợp

Driver phía QCS8550 được thiết kế theo mô hình non-blocking: hàm `get_latest_data()` kiểm tra `in_waiting` trước khi đọc, chỉ thực hiện `readline()` khi đã có dữ liệu trong buffer. Nhờ vậy luồng gọi không bị chặn khi UART chưa có dữ liệu mới, phù hợp với kiến trúc đa luồng của hệ thống. Nếu chuỗi nhận được không đúng định dạng (lỗi decode hoặc thiếu trường), hàm trả về `None` thay vì ném ngoại lệ, tránh làm gián đoạn luồng đọc.

Luồng `temp_humid_reader()` gọi `get_latest_data()` liên tục với chu kỳ `10 ms`, cập nhật hai biến toàn cục `temp` và `humid`. Hai biến này được module telemetry gửi định kỳ về GCS qua `NAMED_VALUE_FLOAT` với nhãn `"Temp"` và `"Humid"`, hiển thị trên giao diện `RovHealth` để người vận hành theo dõi trạng thái khoang kín trong suốt nhiệm vụ.

Ngưỡng cảnh báo độ ẩm không được xử lý tự động ở tầng firmware mà được hiển thị trực tiếp lên giao diện GCS, để người vận hành tự quyết định dừng nhiệm vụ khi phát hiện độ ẩm tăng bất thường — tránh việc hệ thống tự dừng động cơ đột ngột ở độ sâu lớn gây nguy hiểm cho ROV.

4.2.4 Module giám sát năng lượng qua ADC ADS1115 (Điện áp, Dòng điện, SOC)

Module này chịu trách nhiệm giám sát thời gian thực các thông số năng lượng cốt lõi của ROV bao gồm: tổng điện áp khối pin (Pack Voltage), dòng điện tiêu thụ toàn hệ thống, điện áp trung bình của từng cell và dung lượng pin còn lại (State of Charge — SOC). Việc này giúp hệ thống kịp thời đưa ra cảnh báo an toàn độc lập hoặc kích hoạt chế độ cứu hộ (failsafe) khi năng lượng cạn kiệt.

Phần cứng và Cấu hình giao tiếp. Bộ chuyển đổi tương tự — số (ADC) 16-bit độ chính xác cao ADS1115 được sử dụng để đọc tín hiệu analog từ các cảm biến nguồn (phân áp điện áp và cảm biến dòng Hall). ADS1115 giao tiếp với SoC QCS8550 qua bus ngoại vi I2C số 4 tại địa chỉ 0×48 . Để tối ưu hóa tài nguyên phần cứng, hệ thống cấu hình bộ chuyển đổi ở chế độ đơn lẻ (Single-ended) với tốc độ lấy mẫu 128 SPS (*Samples Per Second*), cấu hình tầm đo điện áp đầu vào (PGA) tối đa ở mức ± 4.096 V, tương ứng với độ phân giải lý thuyết cho mỗi LSB là:

$$1 \text{ LSB} = \frac{4.096 \text{ V}}{2^{15}} = 125 \mu\text{V} \quad (4.5)$$

Hệ thống sử dụng hai kênh đầu vào đơn lẻ: kênh AIN3 dùng để giám sát điện áp tổng khối pin qua mạch phân áp, kênh AIN2 dùng để giám sát dòng điện tiêu thụ của ROV.

Hiệu chuẩn Điện áp bằng phương pháp tuyến tính hóa 2 điểm. Dữ liệu thô (*raw ADC data*) từ kênh AIN3 mang giá trị đại số nguyên có dấu 16-bit. Do ảnh hưởng của sai số linh kiện từ mạch phân áp cơ khí và điện trở ký sinh, hệ thống thực hiện hiệu chuẩn thực nghiệm bằng phương pháp hồi quy tuyến tính qua 2 điểm đo chuẩn để xác định hệ số góc A_V và độ lệch B_V :

$$V_{pack} = A_V \cdot \text{Raw}_{AIN3} + B_V \quad (4.6)$$

Qua thực nghiệm đo đạc, các thông số tối ưu được xác định và cấu hình trực tiếp vào firmware bao gồm: $A_V = 0.00201448$ và $B_V = -0.31670$. Phương pháp này cho phép bù sai số hệ thống trực tiếp, đảm bảo điện áp khối pin đầu ra đạt độ chính xác cao.

Ước lượng Dung lượng Pin (SOC) bằng phương pháp Nội suy Tuyến tính. Khối nguồn chính của ROV sử dụng cấu trúc pin LiPo 4S (4 cell nối tiếp). Điện áp của một cell đơn lẻ được tính toán gián tiếp bằng công thức:

$$V_{cell} = \frac{V_{pack}}{4.0} \quad (4.7)$$

Dung lượng pin còn lại (SOC %) được ước lượng dựa trên mối quan hệ phi tuyến giữa điện áp nghỉ (hoặc nhẹ tải) và dung lượng pin thông qua bảng tra cứu thực nghiệm (Look-up Table - LUT) gồm 21 điểm đặc tính từ 4.20 V (100%) đến 3.50 V (0%).

Khi giá trị V_{cell} nằm giữa hai điểm mốc (V_{low}, P_{low}) và (V_{high}, P_{high}) trong bảng LUT, thuật toán nội suy tuyến tính (*Linear Interpolation*) được áp dụng để tính toán mìn giá trị phần trăm năng lượng:

$$SOC = P_{low} + \frac{V_{cell} - V_{low}}{V_{high} - V_{low}} \cdot (P_{high} - P_{low}) \quad (4.8)$$

Đối với các trường hợp tới hạn ($V_{cell} \geq 4.20$ V hoặc $V_{cell} \leq 3.50$ V), hàm xử lý tự động bão hòa đầu ra về giá trị 100.0% hoặc 0.0% để triệt tiêu các sai số ngoại biên.

Thiết kế Tích hợp Hệ thống và Đảm bảo An toàn. Luồng giám sát nguồn chạy song song, độc lập với vòng điều khiển động cơ để tránh hiện tượng nghẽn luồng (*thread blocking*) khi truy cập bus I2C. Dữ liệu tính toán bao gồm `v_pack`, `v_cell`, và `soc` được cập nhật liên tục vào bộ nhớ chia sẻ sau mỗi chu kỳ 0.2 giây (5 Hz).

Kiến trúc phần mềm tích hợp module này phục vụ hai nhiệm vụ chính:

- **Telemetry hạ tầng:** Dữ liệu đóng gói thành các bản tin trạng thái gửi về Trạm điều khiển mặt đất (GCS) để hiển thị trực quan đồ họa thông tin năng lượng cho người vận hành.
- **Module Failsafe hệ thống:** Firmware liên tục giám sát điều kiện an toàn: nếu $SOC < 15\%$ hoặc $V_{cell} < 3.6$ V kéo dài quá 3 giây, hệ thống tự động kích hoạt trạng thái cảnh báo nguy hiểm độc lập, ép ROV chuyển sang chế độ tự nổi hoặc phát tín hiệu khẩn cấp về GCS.

4.3 Module truyền thông

4.3.1 Kết nối GCS(ping_server)

4.3.2 Nhận lệnh điều khiển(control_loop)

4.4 Module điều khiển

4.4.1 Điều khiển thủ công - Mixing matrix và điều khiển động cơ

4.4.2 Điều khiển tự động

a, PID giữ hướng

b, PID giữ độ sâu

c, PID tự động giữ ổn định vị trí

d, PID dự ổn định dựa vào AI bám theo vật thể

4.5 Module cơ cấu chấp hành

4.5.1 Điều khiển ESC và cơ cấu chấp hành qua PCA9685

Để điều khiển hệ thống động cơ (thông qua ESC), hệ thống đèn chiếu sáng và các cơ cấu servo (tay gắp, độ ngả camera), ROV sử dụng vi điều khiển mở rộng PWM 16-channel 12-bit PCA9685. Chip này giao tiếp với SoC QCS8550 qua bus I2C số 1 tại địa chỉ 0×40 . Việc sử dụng PCA9685 giúp giải phóng tài nguyên phần cứng của SoC, đảm bảo xung điều khiển động cơ luôn ổn định, không bị méo tiếng hoặc trễ do phân phối đa nhiệm của hệ điều hành.

Kiến trúc phần cứng và Quản lý GPIO. Hệ thống tích hợp chân cho phép xuất đầu ra Output Enable ($\overline{OE}$) của PCA9685 với chân số 8 thuộc chip GPIO ngoại vi /dev/gpiochip1. Quá trình quản lý pin được thực hiện thông qua thư viện gpiod hiện đại (V2 API) thay vì hệ thống sysfs cũ đã bị loại bỏ:

- **Khởi tạo cấu hình:** Chân $\overline{OE}$ được thiết lập ở chế độ `Direction.OUTPUT` với giá trị mặc định là `Value.ACTIVE` (Mức cao) để tạm thời vô hiệu hóa toàn bộ đầu ra PWM trong quá trình khởi động hệ thống, tránh xung nhiễu kích hoạt động cơ bất ngờ.
- **Kích hoạt hệ thống:** Hàm `output_enable()` sẽ kéo chân $\overline{OE}$ xuống mức thấp (`Value.INACTIVE`), cho phép các chân PWM xuất tín hiệu điều khiển đến các ESC và Servo.

Nguyên lý phát xung và Cấu hình tần số. PCA9685 hoạt động dựa trên một bộ đếm nội bộ 12-bit chạy liên tục từ 0 đến 4095 ($0 \times \text{FFF}$). Mỗi kênh có hai cặp thanh ghi 16-bit xác định thời điểm lật trạng thái: `LEDn_ON` (thời điểm xung lên mức cao) và `LEDn_OFF` (thời điểm xung xuống mức thấp). Trong kiến trúc phần mềm

này, hệ thống cố định giá trị ON bằng 0 và chỉ thay đổi giá trị OFF (tương ứng với giá trị đại số `raw` từ 0 đến 4095) để điều chỉnh chu kỳ nhiệm vụ (*Duty Cycle*).

Tần số xung đầu ra (f) được thiết lập thông qua thanh ghi bộ tiền chia tần số (Prescaler) dựa trên xung thạch anh nội bộ hoặc ngoại vi (`extclk`). Mỗi quan hệ giữa tần số mục tiêu và giá trị thanh ghi được định nghĩa theo công thức từ datasheet:

$$\text{prescaler} = \text{round} \left(\frac{\text{extclk}}{4096 \cdot f} \right) - 1 \quad (4.9)$$

Ngược lại, tần số thực tế của hệ thống được tính toán lại để cập nhật chu kỳ thời gian thực (`period_us`) phục vụ cho việc chuyển đổi đơn vị:

$$f = \frac{\text{extclk}}{4096 \cdot (\text{prescaler} + 1)} \quad (4.10)$$

Khi thay đổi tần số cấu hình (ví dụ đặt tần số 50 Hz cho ESC và Servo), chip bắt buộc phải được đưa về chế độ ngủ (`MODEL_SLEEP`) kết hợp bật chế độ tự động tăng địa chỉ thanh ghi (`MODEL_AI - Auto-Increment`) nhằm tối ưu hóa băng thông truyền tải dữ liệu hàng loạt I2C.

Mô hình ánh xạ dữ liệu (Data Abstraction Layers). Để đơn giản hóa việc lập trình ở các tầng điều khiển phía trên (như vòng lặp PID hay ma trận Mixing), phần mềm xây dựng lớp trừu tượng truy cập thông qua các thuộc tính Python (`@property`). Người lập trình có thể tương tác với 16 kênh đầu ra dưới dạng mảng (List) thông qua 3 không gian đơn vị:

1. **Raw Access (`raw`):** Truy cập trực tiếp vào giá trị số nguyên 12-bit ($0 \rightarrow 4095$) nạp vào thanh ghi.
2. **Duty Access (`duty`):** Điều khiển theo tỷ lệ phần trăm chu kỳ nhiệm vụ từ 0.0 đến 1.0. Công thức chuyển đổi tuyến tính:

$$\text{raw} = \text{round}(\text{duty} \cdot 4095) \quad (4.11)$$

3. **PWM Access (`pwm`):** Điều khiển trực tiếp theo độ rộng xung thời gian thực (Microseconds - μs), đặc biệt quan trọng đối với tín hiệu điều khiển ESC chuẩn RC (thường từ $1000 \mu s$ đến $2000 \mu s$, điểm chết trung tính tại $1500 \mu s$). Giá trị thanh ghi được tính bằng:

$$\text{raw} = \text{round} \left(4095 \cdot \frac{\text{pwm_us}}{\text{period_us}} \right) \quad (4.12)$$

Thiết kế tối ưu hóa Bus I2C và An toàn hệ thống. Do việc điều khiển đồng thời nhiều động cơ yêu cầu tần suất cập nhật cao, nếu ghi dữ liệu riêng lẻ từng kênh sẽ gây nghẽn bus I2C. Firmware giải quyết vấn đề này bằng cách tận dụng cờ `MODEL_AI`.

Hàm `channels_set_raw()` đóng gói toàn bộ giá trị ON (0,0) và OFF của cả 16 kênh vào một mảng byte duy nhất và thực hiện ghi một lần (*Block Write*) bắt đầu từ thanh ghi gốc `REG_LED0_ON_L` (0x06). Sau mỗi lệnh ghi, hệ thống thực hiện đọc ngược lại trạng thái thanh ghi (*Read-back Verification*) để đối chiếu dữ liệu; nếu phát hiện sai lệch do nhiễu đường truyền, một ngoại lệ sẽ được ném ra để hệ thống xử lý failsafe kịp thời.

Khi kết thúc tiến trình hoặc hệ thống gặp sự cố nghiêm trọng, hàm giải phóng `release_gpio()` sẽ được kích hoạt nhằm cấu hình lại chân $\overline{OE}$ về trạng thái đầu vào (`Direction.INPUT`) tách biệt trở kháng, giải phóng tài nguyên dòng lệnh và đóng an toàn kết nối xử lý chip nhằm đưa toàn bộ hệ thống động cơ về trạng thái dừng hoàn toàn.

4.5.2 Điều khiển đèn chiếu sáng

4.5.3 Điều khiển độ ngả camera

4.5.4 Điều khiển tay gấp (Gripper)

4.6 Module telemetry

4.6.1 Đóng gói và phát dữ liệu về cho GCS

4.7 Module cấu hình động cơ

a, UDP config listener

b, Load, Save THRUSTER_MAP

CHƯƠNG 5. GCS

5.1 Cơ sở lý thuyết

5.1.1 Tổng quan chức năng Track and Catch trong hệ thống ROV

- Vị trí của module trong hệ thống lớn - Luồng Camera -> Tracker -> Vision Packet -> Controller -> Thruster

5.1.2 Cơ sở về truyền luồng hình ảnh MJPEG

- HTTP multipart MJPEG - Camera server trên ROV - Agent đọc stream, tách JPEG frame, decode bằng OpenCV - Ưu/nhược điểm: đơn giản, dễ tích hợp, nhưng có độ trễ và không đảm bảo đồng bộ tuyệt đối

5.1.3 Cơ sở về theo dõi mục tiêu bằng Optical Flow

- Bài toán tracking mục tiêu do người dùng chọn - Feature detection - Lucas-Kanade Optical Flow - Pyramidal LK - Lọc outlier và ước lượng chuyển động robust - Ước lượng scale và confidence - Reacquire khi tracking yếu

5.1.4 Chuẩn hóa sai lệch ảnh cho điều khiển

- $dx_{normalized}$
- $dy_{normalized}$
- $scale_{error_{normalized}}$
- Vì sao scale có thể dùng làm xấp xỉ khoảng cách tương đối - Phân biệt: đưa mục tiêu về vị trí lúc chọn, không nhất thiết về tâm ảnh

5.1.5 Cơ sở điều khiển PID cho Track and Catch

- PID rời rạc - PID lateral - PID forward/range - Deadband, saturation, slew-rate limit, derivative filtering, anti-windup - Reset PID khi mất target/manual override

5.1.6 Cơ sở điều khiển ROV và mixing matrix

- Vector điều khiển [forward, lateral, ascend, yaw] - Ma trận mixing 8x4 - Chuyển thrust sang PWM - Vai trò PCA9685, ESC, neutral PWM

5.1.7 Cơ sở truyền thông và tích hợp hệ thống

- HTTP giữa host và agent - UDP vision packet giữa agent và controller - MAVLink/ $SETMODE$ để bật/tắt mode - Đa luồng và shared state

5.1.8 Cơ chế an toàn khi vận hành

- Confidence threshold - Vision timeout - Manual override - Hold heading khi hold vision - Disarm/neutral thrust - Giới hạn lệnh điều khiển

CHƯƠNG 6. ĐIỀU KHIỂN TỰ ĐỘNG

6.1 Cơ sở lý thuyết

6.1.1 Cơ sở lý thuyết điều khiển chuyển động ROV

Trong hệ thống ROV, bộ điều khiển không tác động trực tiếp tới từng động cơ đẩy mà thay vào đó, hệ thống sinh ra một vector lệnh điều khiển biểu diễn chuyển động của ROV. Ma trận trộn (mixing matrix) được sử dụng để phân bổ lệnh này xuống từng động cơ đẩy. Cách tiếp cận này giúp tách biệt giữa bài toán điều khiển chuyển động ở mức logic và bài toán điều khiển phần cứng ở mức cơ cấu chấp hành.

Đối với ROV trong đồ án này, chuyển động chính được xét trên bốn trục điều khiển gồm: tiến/lùi, dịch ngang trái/phải, lên/xuống và quay quanh trục đứng. Do đó, vector điều khiển cấp cao được biểu diễn dưới dạng:

$$\mathbf{u} = \begin{bmatrix} u_f \\ u_l \\ u_z \\ u_{\psi} \end{bmatrix} \quad (6.1)$$

Trong đó:

- u_f là lệnh điều khiển chuyển động tiến/lùi theo phương dọc thân ROV;
- u_l là lệnh điều khiển chuyển động dịch ngang trái/phải;
- u_z là lệnh điều khiển chuyển động lên/xuống theo phương thẳng đứng;
- u_{ψ} là lệnh điều khiển quay quanh trục đứng, hay còn gọi là điều khiển yaw.

Trong chương trình điều khiển, vector này tương ứng với bốn thành phần:

$$\mathbf{u} = \begin{bmatrix} forward \\ lateral \\ ascend \\ yaw \end{bmatrix} \quad (6.2)$$

Các giá trị trong vector điều khiển được chuẩn hóa trong khoảng xấp xỉ $[-1, 1]$. Giá trị dương và âm thể hiện hai chiều chuyển động ngược nhau trên cùng một trục. Ví dụ, $u_f > 0$ và $u_f < 0$ lần lượt biểu diễn hai hướng chuyển động ngược nhau trên phương tiến/lùi; tương tự, u_l , u_z và u_{ψ} cũng biểu diễn hai chiều điều khiển tương

ứng trên các trục dịch ngang, lên/xuống và quay yaw.

a, Nguyên lý phân bổ lực đẩy bằng ma trận trộn

ROV sử dụng 4 động cơ đẩy ngang tại 4 góc trên để điều khiển độ sâu, 4 động cơ dưới xoay góc 45° để di chuyển tiến/lùi/trái/phải. Mỗi động cơ khi hoạt động sẽ tạo ra một lực đẩy theo hướng lắp đặt của nó. Tổng hợp lực và mô-men từ tất cả các động cơ sẽ tạo ra chuyển động thực tế của ROV.

Vì người điều khiển chỉ đưa ra lệnh ở mức chuyển động tổng quát, hệ thống cần một cơ chế để chuyển đổi lệnh điều khiển cấp cao thành lệnh cho từng động cơ. Cơ chế này được gọi là phân bổ lực đẩy, hay control allocation. Trong đồ án, việc phân bổ lực đẩy được thực hiện bằng ma trận trộn.

Quan hệ giữa vector lệnh điều khiển cấp cao và vector lệnh động cơ được biểu diễn như sau:

$$\mathbf{t} = M\mathbf{u} \quad (6.3)$$

Trong đó:

- $\mathbf{u}$ là vector lệnh điều khiển cấp cao;
- M là ma trận trộn;
- $\mathbf{t}$ là vector lệnh lực đẩy chuẩn hóa của các động cơ.

Với ROV sử dụng 8 động cơ đẩy, vector lệnh động cơ có dạng:

$$\mathbf{t} = \begin{bmatrix} t_1 \\ t_2 \\ t_3 \\ t_4 \\ t_5 \\ t_6 \\ t_7 \\ t_8 \end{bmatrix} \quad (6.4)$$

Trong đó t_i là lệnh điều khiển chuẩn hóa gửi tới động cơ đẩy thứ i . Mỗi giá trị t_i thể hiện mức công suất và chiều quay tương ứng của động cơ. Giá trị $t_i = 0$ tương ứng với trạng thái trung tính, $t_i > 0$ và $t_i < 0$ tương ứng với hai chiều tạo lực ngược nhau.

Ma trận trộn có kích thước 8×4 , vì hệ thống có 8 động cơ đẩy và 4 trục điều

khuyến cấp cao. Mỗi hàng của ma trận tương ứng với một động cơ, còn mỗi cột tương ứng với một trục chuyển động. Với cấu hình trong đồ án, ma trận trộn được biểu diễn như sau:

$$M = \begin{bmatrix} 1 & -1 & 0 & -0.7071 \\ -1 & -1 & 0 & -0.7071 \\ 1 & 1 & 0 & -0.7071 \\ 1 & -1 & 0 & 0.7071 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & -1 & 0 \\ 0 & 0 & -1 & 0 \\ 0 & 0 & 1 & 0 \end{bmatrix} \quad (6.5)$$

Do đó:

$$\begin{bmatrix} t_1 \\ t_2 \\ t_3 \\ t_4 \\ t_5 \\ t_6 \\ t_7 \\ t_8 \end{bmatrix} = \begin{bmatrix} 1 & -1 & 0 & -0.7071 \\ -1 & -1 & 0 & -0.7071 \\ 1 & 1 & 0 & -0.7071 \\ 1 & -1 & 0 & 0.7071 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & -1 & 0 \\ 0 & 0 & -1 & 0 \\ 0 & 0 & 1 & 0 \end{bmatrix} \begin{bmatrix} u_f \\ u_l \\ u_z \\ u_\psi \end{bmatrix} \quad (6.6)$$

Từ phép nhân ma trận trên, lệnh của từng động cơ được tính như sau:

$$t_1 = u_f - u_l - 0.7071u_\psi \quad (6.7)$$

$$t_2 = -u_f - u_l - 0.7071u_\psi \quad (6.8)$$

$$t_3 = u_f + u_l - 0.7071u_\psi \quad (6.9)$$

$$t_4 = u_f - u_l + 0.7071u_\psi \quad (6.10)$$

$$t_5 = u_z \quad (6.11)$$

$$t_6 = -u_z \quad (6.12)$$

$$t_7 = -u_z \quad (6.13)$$

$$t_8 = u_z \quad (6.14)$$

Các phương trình trên cho thấy lệnh của từng động cơ là tổ hợp tuyến tính của các thành phần điều khiển cấp cao. Điều này có nghĩa là một động cơ có thể đồng thời tham gia tạo ra nhiều loại chuyển động khác nhau. Chẳng hạn, các động cơ ngang vừa có thể tham gia tạo chuyển động tiến/lùi, vừa tạo chuyển động dịch ngang, vừa tạo mô-men quay yaw.

b, Ý nghĩa vật lý của các cột trong ma trận trộn

Mỗi cột trong ma trận trộn biểu diễn mức đóng góp của các động cơ đối với một trục chuyển động cụ thể.

Cột thứ nhất tương ứng với lệnh tiến/lùi u_f :

$$M_f = \begin{bmatrix} 1 \\ -1 \\ 1 \\ 1 \\ 0 \\ 0 \\ 0 \\ 0 \end{bmatrix} \quad (6.15)$$

Cột này cho thấy chuyển động tiến/lùi được tạo ra bởi nhóm động cơ ngang. Các động cơ thẳng đứng không tham gia vào chuyển động tiến/lùi nên hệ số của chúng bằng 0.

Cột thứ hai tương ứng với lệnh dịch ngang u_l :

$$M_l = \begin{bmatrix} -1 \\ -1 \\ 1 \\ -1 \\ 0 \\ 0 \\ 0 \\ 0 \end{bmatrix} \quad (6.16)$$

Tương tự như chuyển động tiến/lùi, chuyển động dịch ngang cũng được tạo bởi các động cơ ngang. Tổ hợp dấu khác nhau giữa các động cơ giúp tạo ra lực tổng theo phương ngang thân ROV.

Cột thứ ba tương ứng với lệnh lên/xuống u_z :

$$M_z = \begin{bmatrix} 0 \\ 0 \\ 0 \\ 0 \\ 1 \\ -1 \\ -1 \\ 1 \end{bmatrix} \quad (6.17)$$

Cột này cho thấy chuyển động thẳng đứng chỉ được tạo bởi các động cơ thẳng đứng. Các động cơ ngang không tham gia vào chuyển động lên/xuống nên hệ số của chúng bằng 0.

Cột thứ tư tương ứng với lệnh quay yaw u_ψ :

$$M_\psi = \begin{bmatrix} -0.7071 \\ -0.7071 \\ -0.7071 \\ 0.7071 \\ 0 \\ 0 \\ 0 \\ 0 \end{bmatrix} \quad (6.18)$$

Chuyển động yaw được tạo bởi mô-men xoắn quanh trục đứng. Các động cơ ngang tạo ra các lực có phương và vị trí phù hợp để sinh mô-men quay cho ROV. Hệ số 0.7071 xấp xỉ $\sin(45^\circ)$, thường xuất hiện trong các cấu hình động cơ đặt chéo góc 45° . Khi động cơ được đặt chéo, lực đẩy của nó có thể được phân rã thành các thành phần theo phương dọc và phương ngang. Với góc 45° , hai thành phần này có độ lớn bằng nhau và bằng $\frac{\sqrt{2}}{2}$ lần lực tổng.

c, Tổng hợp nhiều lệnh chuyển động

Ma trận trộn cho phép ROV tổng hợp nhiều lệnh chuyển động cùng lúc. Do quan hệ giữa vector điều khiển và lệnh động cơ là tuyến tính, lệnh cuối cùng của mỗi động cơ là tổng đóng góp từ nhiều thành phần điều khiển.

Ví dụ, khi ROV vừa tiến vừa quay yaw, vector điều khiển có dạng:

$$\mathbf{u} = \begin{bmatrix} u_f \\ 0 \\ 0 \\ u_\psi \end{bmatrix} \quad (6.19)$$

Khi đó:

$$\mathbf{t} = M \begin{bmatrix} u_f \\ 0 \\ 0 \\ u_\psi \end{bmatrix} = u_f M_f + u_\psi M_\psi \quad (6.20)$$

Điều này có nghĩa là mỗi động cơ ngang sẽ nhận một lệnh tổng hợp từ yêu cầu tiến/lùi và yêu cầu quay yaw. Chẳng hạn, với động cơ thứ nhất:

$$t_1 = u_f - 0.7071u_\psi \quad (6.21)$$

Nếu vừa có lệnh tiến, vừa có lệnh quay, động cơ thứ nhất sẽ nhận giá trị là tổng đại số của hai thành phần này. Nhờ đó, ROV có thể thực hiện các chuyển động phức hợp như vừa tiến vừa quay, vừa dịch ngang vừa giữ độ sâu, hoặc vừa bám mục tiêu vừa ổn định hướng.

d, Chuẩn hóa và giới hạn lệnh động cơ

Sau khi tính được vector lệnh động cơ $\mathbf{t}$, mỗi giá trị t_i cần được giới hạn trong khoảng cho phép trước khi chuyển thành tín hiệu điều khiển thực tế. Trong hệ thống này, lệnh động cơ được chuẩn hóa trong khoảng $[-1, 1]$:

$$t_i^{clip} = clip(t_i, -1, 1) \quad (6.22)$$

Trong đó hàm $clip$ được định nghĩa như sau:

$$clip(t_i, -1, 1) = \begin{cases} -1, & t_i < -1 \\ t_i, & -1 \leq t_i \leq 1 \\ 1, & t_i > 1 \end{cases} \quad (6.23)$$

Việc giới hạn này giúp tránh trường hợp lệnh điều khiển vượt quá khả năng đáp ứng của động cơ hoặc ESC. Nếu không có bước giới hạn, khi nhiều lệnh chuyển động được cộng lại, một số giá trị t_i có thể vượt quá biên cho phép, gây ra lệnh điều khiển không hợp lệ hoặc làm hệ thống hoạt động không ổn định.

e, Chuyển đổi lệnh động cơ sang tín hiệu PWM

Các động cơ đẩy của ROV được điều khiển thông qua ESC. ESC nhận tín hiệu PWM để xác định chiều quay và mức công suất của động cơ. Sau khi có lệnh động cơ chuẩn hóa t_i^{clip} , hệ thống chuyển đổi lệnh này sang xung PWM theo công thức:

$$PWM_i = PWM_{neutral} + t_i^{clip} \cdot G_{ESC} \cdot P_{max} \quad (6.24)$$

Trong đó:

- PWM_i là giá trị xung PWM gửi tới ESC của động cơ thứ i ;
- $PWM_{neutral}$ là giá trị PWM trung tính, tại đó động cơ không tạo lực đẩy;
- G_{ESC} là hệ số khuếch đại giới hạn biên điều khiển của ESC;
- P_{max} là biên độ PWM tối đa được sử dụng trong điều khiển;
- t_i^{clip} là lệnh động cơ đã được giới hạn trong khoảng $[-1, 1]$.

Trong hệ thống, giá trị trung tính của PWM là:

$$PWM_{neutral} = 1500 \quad (6.25)$$

Khi $t_i^{clip} = 0$, ta có:

$$PWM_i = 1500 \quad (6.26)$$

Khi đó động cơ ở trạng thái trung tính. Nếu $t_i^{clip} > 0$, PWM lớn hơn 1500 và

động cơ quay theo một chiều. Nếu $t_i^{clip} < 0$, PWM nhỏ hơn 1500 và động cơ quay theo chiều ngược lại.

Với hệ số khuếch đại ESC là G_{ESC} và biên PWM cực đại P_{max} , khoảng PWM thực tế được xác định bởi:

$$PWM_i \in [PWM_{neutral} - G_{ESC}P_{max}, PWM_{neutral} + G_{ESC}P_{max}] \quad (6.27)$$

Ví dụ, nếu $PWM_{neutral} = 1500$, $G_{ESC} = 0.5$ và $P_{max} = 400$, ta có:

$$PWM_i \in [1300, 1700] \quad (6.28)$$

Khoảng này đảm bảo động cơ có thể tạo lực theo cả hai chiều nhưng vẫn được giới hạn trong biên điều khiển an toàn.

6.1.2 Ánh xạ động cơ logic sang kênh PWM thực tế

Trong phần mềm điều khiển, các động cơ được đánh số logic từ T_1 đến T_8 . Tuy nhiên, các động cơ này không nhất thiết được nối với các kênh PWM có cùng thứ tự trên mạch điều khiển. Vì vậy, hệ thống cần một bảng ánh xạ giữa thứ tự động cơ trong mô hình điều khiển và kênh PWM thực tế.

Bảng ánh xạ có dạng:

$$\begin{bmatrix} T_1 & T_2 & T_3 & T_4 & T_5 & T_6 & T_7 & T_8 \end{bmatrix} \rightarrow \begin{bmatrix} 9 & 10 & 4 & 6 & 7 & 8 & 5 & 11 \end{bmatrix} \quad (6.29)$$

Điều này có nghĩa là lệnh của động cơ logic T_1 được gửi tới kênh PWM số 9, lệnh của T_2 được gửi tới kênh PWM số 10, lệnh của T_3 được gửi tới kênh PWM số 4, và tương tự cho các động cơ còn lại. Việc sử dụng bảng ánh xạ giúp phần mềm điều khiển không phụ thuộc trực tiếp vào cách đi dây phần cứng, đồng thời giúp dễ hiệu chỉnh khi thay đổi kết nối động cơ.

a, Mối quan hệ giữa bộ điều khiển PID và ma trận trộn

Bộ điều khiển PID không trực tiếp điều khiển từng động cơ riêng lẻ. Thay vào đó, PID được sử dụng để sinh ra các thành phần trong vector điều khiển cấp cao u . Ví dụ, một bộ điều khiển giữ hướng có thể sinh ra thành phần u_ψ , một bộ điều khiển giữ độ sâu có thể sinh ra thành phần u_z , còn một bộ điều khiển bám mục tiêu bằng thị giác có thể sinh ra các thành phần u_f và u_l .

Sau khi các thành phần điều khiển cấp cao được xác định, chúng đều được đưa qua cùng một ma trận trộn để tạo ra lệnh cho từng động cơ:

$$\mathbf{u} = \begin{bmatrix} u_f \\ u_l \\ u_z \\ u_\psi \end{bmatrix} \Rightarrow \mathbf{t} = M\mathbf{u} \Rightarrow PWM_i = PWM_{neutral} + clip(t_i, -1, 1)G_{ESC}P_{max} \quad (6.30)$$

Như vậy, có thể xem bộ điều khiển PID là khối tạo lệnh chuyển động, còn ma trận trộn là khối phân bổ lệnh chuyển động xuống các cơ cấu chấp hành. Sự tách biệt này giúp hệ thống có cấu trúc rõ ràng: các thuật toán điều khiển chỉ cần quan tâm đến việc ROV cần di chuyển theo hướng nào, trong khi ma trận trộn chịu trách nhiệm xác định từng động cơ phải hoạt động ra sao để tạo ra chuyển động đó.

b, Ý nghĩa đối với bài toán điều khiển ROV trong đồ án

Trong bài toán của đồ án, ROV cần thực hiện các chuyển động ổn định và có khả năng phối hợp nhiều trục điều khiển cùng lúc. Ví dụ, khi bám theo mục tiêu, ROV có thể cần tiến/lùi để giữ khoảng cách, dịch ngang để đưa mục tiêu về vị trí mong muốn trong khung hình, đồng thời giữ độ sâu và hướng quay ổn định. Nếu điều khiển trực tiếp từng động cơ, việc phối hợp các chuyển động này sẽ phức tạp và khó mở rộng.

Việc sử dụng vector điều khiển cấp cao kết hợp với ma trận trộn giải quyết vấn đề này bằng cách chuẩn hóa giao diện giữa thuật toán điều khiển và phần cứng động cơ. Mọi thuật toán điều khiển đều chỉ cần sinh ra lệnh theo bốn trục cơ bản gồm tiến/lùi, dịch ngang, lên/xuống và yaw. Sau đó, ma trận trộn tự động chuyển đổi các lệnh này thành lệnh tương ứng cho 8 động cơ đẩy.

Cách tiếp cận này có ba ưu điểm chính. Thứ nhất, nó giúp đơn giản hóa quá trình thiết kế thuật toán điều khiển, vì bộ điều khiển không cần trực tiếp tính toán lệnh cho từng động cơ. Thứ hai, nó cho phép kết hợp nhiều chế độ điều khiển khác nhau trong cùng một hệ thống, chẳng hạn điều khiển thủ công, giữ hướng, giữ độ sâu và bám mục tiêu bằng thị giác. Thứ ba, nó giúp hệ thống dễ bảo trì và hiệu chỉnh, vì khi thay đổi cấu hình động cơ hoặc thứ tự kết nối phần cứng, có thể chỉ cần điều chỉnh ma trận trộn hoặc bảng ánh xạ kênh PWM mà không cần thay đổi toàn bộ thuật toán điều khiển cấp cao.

Tóm lại, cơ sở lý thuyết của khối điều khiển chuyển động ROV trong đồ án dựa trên ba bước chính: biểu diễn lệnh chuyển động bằng vector điều khiển cấp cao,

phân bổ lệnh xuống từng động cơ bằng ma trận trộn, và chuyển đổi lệnh động cơ chuẩn hóa thành tín hiệu PWM gửi tới ESC. Chuỗi xử lý này có thể được tóm tắt như sau:

$$\mathbf{u} = \begin{bmatrix} u_f \\ u_l \\ u_z \\ u_\psi \end{bmatrix} \quad (6.31)$$

$$\mathbf{t} = M\mathbf{u} \quad (6.32)$$

$$PWM_i = PWM_{neutral} + clip(t_i, -1, 1)G_{ESC}P_{max} \quad (6.33)$$

Đây là nền tảng để triển khai các chế độ điều khiển cao hơn của ROV, bao gồm điều khiển thủ công, ổn định tư thế, giữ độ sâu và bám mục tiêu tự động.

6.1.3 Thuật toán Optical Flow

Trong bài toán bám mục tiêu của ROV, hệ thống cần xác định sự thay đổi vị trí của đối tượng trong chuỗi ảnh thu được từ camera. Sau khi người vận hành chọn một mục tiêu trên khung hình, thuật toán thị giác cần tiếp tục theo dõi mục tiêu đó trong các frame kế tiếp và ước lượng sự dịch chuyển của mục tiêu trong ảnh. Thông tin dịch chuyển này sau đó được chuyển thành sai số thị giác, làm đầu vào cho bộ điều khiển chuyển động của ROV.

Optical Flow là một phương pháp thị giác máy tính dùng để ước lượng chuyển động biểu kiến của các điểm ảnh giữa hai frame liên tiếp trong một chuỗi ảnh. Chuyển động biểu kiến ở đây là chuyển động quan sát được trên mặt phẳng ảnh, không nhất thiết là chuyển động thật trong không gian ba chiều. Nguyên nhân tạo ra Optical Flow có thể đến từ chuyển động của vật thể, chuyển động của camera, hoặc đồng thời cả hai. Đối với hệ thống ROV, camera được gắn trên phương tiện, vì vậy chuyển động trong ảnh có thể phản ánh cả chuyển động của mục tiêu và chuyển động tương đối giữa ROV với mục tiêu.

Trong đồ án này, Optical Flow được sử dụng nhằm giải quyết yêu cầu theo dõi mục tiêu đã chọn trong luồng hình ảnh thời gian thực. Thay vì nhận dạng lại vật thể ở mỗi frame bằng một mô hình học sâu, thuật toán theo dõi sự dịch chuyển của các điểm đặc trưng thuộc vùng mục tiêu giữa các frame liên tiếp. Cách tiếp cận này phù hợp với hệ thống nhúng và xử lý thời gian thực vì không yêu cầu tập dữ liệu

huấn luyện lớn, chi phí tính toán tương đối thấp và có thể áp dụng cho mục tiêu do người dùng chọn tùy ý.

a, Khái niệm chuyển động biểu kiến trong ảnh

Một camera ghi lại cảnh ba chiều của môi trường dưới dạng ảnh hai chiều. Khi vật thể trong môi trường chuyển động hoặc khi camera chuyển động, vị trí chiếu của các điểm vật lý lên mặt phẳng ảnh cũng thay đổi. Sự thay đổi vị trí này tạo thành trường chuyển động trong ảnh.

Giả sử một điểm vật lý P trên mục tiêu được camera quan sát tại thời điểm t . Điểm này được chiếu lên ảnh tại vị trí:

$$\mathbf{p}(t) = \begin{bmatrix} x(t) \\ y(t) \end{bmatrix} \quad (6.34)$$

Tại thời điểm kế tiếp $t + \Delta t$, do chuyển động tương đối giữa camera và mục tiêu, điểm P có thể xuất hiện tại vị trí mới:

$$\mathbf{p}(t + \Delta t) = \begin{bmatrix} x(t + \Delta t) \\ y(t + \Delta t) \end{bmatrix} \quad (6.35)$$

Vector dịch chuyển của điểm ảnh giữa hai frame là:

$$\Delta \mathbf{p} = \mathbf{p}(t + \Delta t) - \mathbf{p}(t) = \begin{bmatrix} \Delta x \\ \Delta y \end{bmatrix} \quad (6.36)$$

Trong đó:

$$\Delta x = x(t + \Delta t) - x(t) \quad (6.37)$$

$$\Delta y = y(t + \Delta t) - y(t) \quad (6.38)$$

Nếu chia độ dịch chuyển cho khoảng thời gian giữa hai frame, ta thu được vận tốc biểu kiến của điểm ảnh:

$$\mathbf{v} = \frac{\Delta \mathbf{p}}{\Delta t} = \begin{bmatrix} u \\ v \end{bmatrix} \quad (6.39)$$

Trong đó u là thành phần vận tốc theo phương ngang của ảnh và v là thành phần

vận tốc theo phương dọc của ảnh. Tập hợp các vector vận tốc của các điểm ảnh trong toàn bộ ảnh được gọi là trường Optical Flow.

b, Phân biệt chuyển động thật và Optical Flow

Cần phân biệt giữa chuyển động thật trong không gian ba chiều và Optical Flow trong ảnh. Chuyển động thật của vật thể diễn ra trong hệ tọa độ không gian, có thể được biểu diễn bằng các đại lượng như vị trí, vận tốc và gia tốc trong hệ tọa độ 3D. Trong khi đó, Optical Flow chỉ mô tả sự thay đổi vị trí của các điểm ảnh trên mặt phẳng ảnh 2D.

Một điểm quan trọng là cùng một chuyển động trên ảnh có thể được tạo ra bởi nhiều nguyên nhân khác nhau. Ví dụ, nếu mục tiêu dịch sang phải trong ảnh, nguyên nhân có thể là:

- mục tiêu thật sự di chuyển sang phải so với camera;
- ROV hoặc camera di chuyển sang trái;
- ROV quay yaw làm toàn bộ cảnh dịch chuyển trong ảnh;
- kết hợp của nhiều nguyên nhân trên.

Vì vậy, Optical Flow không trực tiếp cung cấp tọa độ 3D hay vận tốc thật của mục tiêu. Tuy nhiên, trong bài toán điều khiển bám mục tiêu, hệ thống không nhất thiết cần khôi phục đầy đủ chuyển động 3D. Mục tiêu điều khiển là làm cho đối tượng đã chọn duy trì vị trí mong muốn trong ảnh và duy trì kích thước tương đối phù hợp. Do đó, sai lệch ảnh do Optical Flow cung cấp là đủ để tạo phản hồi điều khiển cho ROV.

c, Giả thiết bảo toàn độ sáng

Cơ sở quan trọng nhất của Optical Flow là giả thiết bảo toàn độ sáng. Giả thiết này cho rằng cường độ sáng của cùng một điểm vật lý gần như không đổi giữa hai frame liên tiếp trong một khoảng thời gian rất ngắn.

Gọi cường độ sáng tại vị trí ảnh (x, y) và thời điểm t là:

$$I(x, y, t) \tag{6.40}$$

Nếu điểm ảnh tại (x, y) ở thời điểm t dịch chuyển đến vị trí $(x + \Delta x, y + \Delta y)$ ở thời điểm $t + \Delta t$, giả thiết bảo toàn độ sáng được viết như sau:

$$I(x, y, t) = I(x + \Delta x, y + \Delta y, t + \Delta t) \tag{6.41}$$

Ý nghĩa của phương trình này là: mặc dù vị trí của điểm ảnh thay đổi, giá trị cường độ sáng của điểm vật lý tương ứng được xem là gần như không đổi.

Giả thiết này thường hợp lý khi:

- khoảng thời gian giữa hai frame nhỏ;
- vật thể không thay đổi màu sắc đột ngột;
- điều kiện chiếu sáng không thay đổi quá nhanh;
- camera không bị rung hoặc mờ quá mạnh.

Tuy nhiên, trong môi trường dưới nước, giả thiết này có thể bị ảnh hưởng bởi ánh sáng yếu, nhiễu, phản xạ, tán xạ ánh sáng và độ đục của nước. Vì vậy, hệ thống cần kết hợp Optical Flow với các cơ chế kiểm tra độ tin cậy để tránh sử dụng các kết quả theo dõi sai.

d, Khai triển Taylor và phương trình ràng buộc Optical Flow

Để xây dựng phương trình cơ bản của Optical Flow, xét biểu thức:

$$I(x + \Delta x, y + \Delta y, t + \Delta t) \quad (6.42)$$

Khai triển Taylor bậc nhất quanh điểm (x, y, t) :

$$I(x + \Delta x, y + \Delta y, t + \Delta t) \approx I(x, y, t) + I_x \Delta x + I_y \Delta y + I_t \Delta t \quad (6.43)$$

Trong đó:

- $I_x = \frac{\partial I}{\partial x}$ là đạo hàm riêng của cường độ ảnh theo phương ngang;
- $I_y = \frac{\partial I}{\partial y}$ là đạo hàm riêng của cường độ ảnh theo phương dọc;
- $I_t = \frac{\partial I}{\partial t}$ là đạo hàm riêng của cường độ ảnh theo thời gian.

Từ giả thiết bảo toàn độ sáng:

$$I(x, y, t) = I(x + \Delta x, y + \Delta y, t + \Delta t) \quad (6.44)$$

Thay khai triển Taylor vào, ta có:

$$I(x, y, t) = I(x, y, t) + I_x \Delta x + I_y \Delta y + I_t \Delta t \quad (6.45)$$

Rút gọn hai vế:

$$I_x \Delta x + I_y \Delta y + I_t \Delta t = 0 \quad (6.46)$$

Chia cả hai vế cho Δt :

$$I_x \frac{\Delta x}{\Delta t} + I_y \frac{\Delta y}{\Delta t} + I_t = 0 \quad (6.47)$$

Đặt:

$$u = \frac{\Delta x}{\Delta t} \quad (6.48)$$

$$v = \frac{\Delta y}{\Delta t} \quad (6.49)$$

Ta thu được phương trình ràng buộc Optical Flow:

$$I_x u + I_y v + I_t = 0 \quad (6.50)$$

Phương trình này mô tả quan hệ giữa gradient không gian của ảnh, biến thiên thời gian của ảnh và vận tốc biểu kiến của điểm ảnh. Đây là phương trình nền tảng của nhiều thuật toán Optical Flow.

e, Ý nghĩa của các thành phần I_x, I_y, I_t

Trong phương trình:

$$I_x u + I_y v + I_t = 0 \quad (6.51)$$

mỗi thành phần có một ý nghĩa riêng.

Thành phần I_x cho biết cường độ ảnh thay đổi nhanh hay chậm theo phương ngang. Nếu ảnh có biên dọc rõ ràng, giá trị I_x thường lớn. Thành phần I_y cho biết cường độ ảnh thay đổi theo phương dọc. Nếu ảnh có biên ngang rõ ràng, giá trị I_y thường lớn. Thành phần I_t cho biết sự thay đổi cường độ ảnh giữa hai frame liên tiếp.

Nếu một vùng ảnh hoàn toàn phẳng, tức là cường độ gần như không đổi theo cả phương ngang và phương dọc, ta có:

$$I_x \approx 0, \quad I_y \approx 0 \quad (6.52)$$

Khi đó, rất khó xác định chuyển động của vùng ảnh vì không có đủ thông tin đặc trưng để theo dõi. Đây là lý do Optical Flow thường hoạt động tốt hơn trên các vùng có texture, góc, cạnh hoặc hoa văn rõ ràng.

f, Vấn đề aperture problem

Phương trình ràng buộc Optical Flow có dạng:

$$I_x u + I_y v + I_t = 0 \quad (6.53)$$

Trong phương trình này, hai ẩn số cần tìm là u và v , nhưng chỉ có một phương trình. Do đó, nếu chỉ xét một điểm ảnh riêng lẻ, không thể xác định duy nhất vector chuyển động hai chiều của điểm ảnh đó. Đây được gọi là vấn đề thiếu ràng buộc trong Optical Flow, hay aperture problem.

Aperture problem có thể hiểu trực quan như sau: nếu chỉ quan sát một đoạn cạnh thẳng thông qua một cửa sổ nhỏ, ta chỉ thấy rõ chuyển động vuông góc với cạnh, còn chuyển động dọc theo cạnh rất khó xác định. Vì vậy, để ước lượng chuyển động ổn định, thuật toán cần xét thêm thông tin từ các điểm ảnh lân cận hoặc áp dụng thêm giả thiết về tính trơn của trường chuyển động.

Các phương pháp Optical Flow thường giải quyết vấn đề này bằng một trong hai hướng:

- giả định các điểm lân cận có chuyển động giống nhau trong một cửa sổ nhỏ;
- giả định trường chuyển động thay đổi trơn trong toàn ảnh.

Phương pháp Lucas–Kanade thuộc nhóm thứ nhất, trong khi các phương pháp dense Optical Flow toàn cục như Horn–Schunck thuộc nhóm thứ hai.

g, Sparse Optical Flow và Dense Optical Flow

Optical Flow có thể được chia thành hai nhóm chính: sparse Optical Flow và dense Optical Flow.

Dense Optical Flow ước lượng vector chuyển động cho hầu hết hoặc toàn bộ các điểm ảnh trong ảnh. Kết quả là một trường vector dày đặc biểu diễn chuyển động tại mỗi pixel. Cách tiếp cận này cung cấp thông tin chuyển động chi tiết, nhưng thường có chi phí tính toán cao hơn.

Sparse Optical Flow chỉ ước lượng chuyển động cho một tập điểm đặc trưng được chọn trước. Các điểm này thường là góc, cạnh hoặc vùng có texture rõ ràng. Vì chỉ theo dõi một số lượng điểm nhất định, sparse Optical Flow có chi phí tính toán thấp hơn và phù hợp với các hệ thống cần xử lý thời gian thực.

Trong bài toán bám mục tiêu của ROV, sparse Optical Flow là lựa chọn phù hợp vì hệ thống chỉ cần theo dõi vùng mục tiêu đã chọn, không cần tính trường chuyển động cho toàn bộ ảnh. Việc tập trung vào các điểm đặc trưng trong bounding box giúp giảm tải tính toán và tăng khả năng chạy thời gian thực trên phần cứng nhúng.

h, Phương pháp Lucas–Kanade

Phương pháp Lucas–Kanade là một phương pháp phổ biến để tính sparse Optical Flow. Ý tưởng chính của phương pháp này là trong một vùng ảnh nhỏ, các điểm ảnh lân cận được giả định có cùng vector chuyển động. Nhờ đó, từ nhiều phương trình ràng buộc Optical Flow của các điểm trong vùng lân cận, ta có thể tìm được vector chuyển động chung theo nghĩa bình phương tối thiểu.

Giả sử xét một cửa sổ nhỏ gồm n điểm ảnh quanh điểm cần theo dõi. Với điểm ảnh thứ i , phương trình Optical Flow là:

$$I_{x_i}u + I_{y_i}v + I_{t_i} = 0 \quad (6.54)$$

Hay:

$$I_{x_i}u + I_{y_i}v = -I_{t_i} \quad (6.55)$$

Với n điểm ảnh, ta thu được hệ phương trình:

$$\begin{cases} I_{x_1}u + I_{y_1}v = -I_{t_1} \\ I_{x_2}u + I_{y_2}v = -I_{t_2} \\ \dots \\ I_{x_n}u + I_{y_n}v = -I_{t_n} \end{cases} \quad (6.56)$$

Viết dưới dạng ma trận:

$$Ad = b \quad (6.57)$$

Trong đó:

$$A = \begin{bmatrix} I_{x_1} & I_{y_1} \\ I_{x_2} & I_{y_2} \\ \vdots & \vdots \\ I_{x_n} & I_{y_n} \end{bmatrix} \quad (6.58)$$

$$\mathbf{d} = \begin{bmatrix} u \\ v \end{bmatrix} \quad (6.59)$$

$$\mathbf{b} = - \begin{bmatrix} I_{t_1} \\ I_{t_2} \\ \vdots \\ I_{t_n} \end{bmatrix} \quad (6.60)$$

Vì số phương trình thường lớn hơn số ẩn, hệ được giải theo nghĩa bình phương tối thiểu:

$$\mathbf{d} = (A^T A)^{-1} A^T \mathbf{b} \quad (6.61)$$

Trong đó:

$$A^T A = \begin{bmatrix} \sum I_{x_i}^2 & \sum I_{x_i} I_{y_i} \\ \sum I_{x_i} I_{y_i} & \sum I_{y_i}^2 \end{bmatrix} \quad (6.62)$$

Ma trận $A^T A$ còn được gọi là ma trận cấu trúc cục bộ của vùng ảnh. Để điểm đặc trưng có thể theo dõi tốt, ma trận này cần khả nghịch và có hai trị riêng đủ lớn. Điều đó có nghĩa là vùng ảnh cần có biến thiên cường độ theo cả hai phương, chẳng hạn như các điểm góc. Nếu vùng ảnh chỉ có một cạnh thẳng, một trị riêng có thể nhỏ, khiến việc ước lượng chuyển động theo một hướng trở nên không ổn định. Nếu vùng ảnh phẳng, cả hai trị riêng đều nhỏ và điểm đó không phù hợp để theo dõi.

i, Lựa chọn điểm đặc trưng

Trước khi áp dụng Lucas–Kanade, hệ thống cần chọn các điểm đặc trưng phù hợp trong vùng mục tiêu. Điểm đặc trưng tốt là điểm có thể được nhận ra và theo dõi ổn định qua các frame liên tiếp. Trong thực tế, các điểm góc thường được ưu tiên vì chúng có sự thay đổi cường độ rõ ràng theo cả hai phương ngang và dọc.

Một tiêu chí thường được sử dụng để đánh giá điểm đặc trưng là dựa trên ma trận cấu trúc:

$$G = \sum_{(x,y) \in W} \begin{bmatrix} I_x^2 & I_x I_y \\ I_x I_y & I_y^2 \end{bmatrix} \quad (6.63)$$

Trong đó W là cửa sổ lân cận quanh điểm đang xét. Gọi λ_1 và λ_2 là hai trị riêng của ma trận G . Một điểm được coi là dễ theo dõi nếu cả hai trị riêng đều đủ lớn:

$$\lambda_1 > \lambda_{min}, \quad \lambda_2 > \lambda_{min} \quad (6.64)$$

Nếu một trong hai trị riêng quá nhỏ, điểm đó có thể nằm trên một cạnh đơn hướng và dễ gặp aperture problem. Nếu cả hai trị riêng đều nhỏ, vùng ảnh gần như phẳng và không cung cấp đủ thông tin để theo dõi.

Trong bài toán ROV, điểm đặc trưng được chọn trong bounding box của mục tiêu. Việc chọn điểm bên trong vùng mục tiêu giúp thuật toán tập trung theo dõi đối tượng quan tâm, tránh bị ảnh hưởng bởi chuyển động nền hoặc các vật thể không liên quan.

j, Pyramidal Lucas–Kanade

Lucas–Kanade cơ bản giả định chuyển động giữa hai frame là nhỏ. Giả định này phù hợp khi tốc độ frame cao và chuyển động của đối tượng không quá nhanh. Tuy nhiên, trong thực tế, ROV và mục tiêu có thể chuyển động nhanh hơn, hoặc camera có thể rung, khiến độ dịch chuyển giữa hai frame khá lớn. Khi đó, Lucas–Kanade ở một độ phân giải duy nhất có thể không theo dõi ổn định.

Để xử lý chuyển động lớn hơn, phương pháp Pyramidal Lucas–Kanade sử dụng tháp ảnh đa tỉ lệ. Ý tưởng là xây dựng nhiều phiên bản của ảnh ở các độ phân giải khác nhau:

$$I^0, I^1, I^2, \dots, I^L \quad (6.65)$$

Trong đó:

- I^0 là ảnh gốc;
- I^1 là ảnh giảm kích thước từ I^0 ;
- I^2 là ảnh tiếp tục giảm kích thước từ I^1 ;
- I^L là mức ảnh có độ phân giải thấp nhất.

Ở mức ảnh thấp, chuyển động lớn trong ảnh gốc trở thành chuyển động nhỏ hơn tính theo số pixel. Thuật toán bắt đầu ước lượng chuyển động ở mức thô nhất, sau đó lan truyền kết quả xuống các mức ảnh có độ phân giải cao hơn để tinh chỉnh.

Quy trình tổng quát như sau:

1. Xây dựng tháp ảnh cho frame trước và frame hiện tại.

2. Bắt đầu từ mức ảnh có độ phân giải thấp nhất.
3. Ước lượng chuyển động của điểm đặc trưng ở mức này.
4. Phóng đại vector chuyển động lên mức ảnh cao hơn.
5. Dùng vector đã phóng đại làm khởi tạo để tìm chỉnh ở mức tiếp theo.
6. Lặp lại cho tới ảnh gốc.

Nhờ cơ chế coarse-to-fine này, Pyramidal Lucas–Kanade có thể theo dõi chuyển động lớn hơn so với Lucas–Kanade một mức ảnh. Đây là lý do phương pháp này thường được dùng trong các hệ thống tracking thời gian thực.

k, Theo dõi bounding box mục tiêu

Trong bài toán bám mục tiêu, người dùng thường chọn mục tiêu bằng một bounding box trên khung hình. Bounding box có thể được biểu diễn bởi tọa độ góc trái trên và kích thước:

$$B_t = (x_t, y_t, w_t, h_t) \quad (6.66)$$

Trong đó:

- (x_t, y_t) là tọa độ góc trái trên của bounding box tại thời điểm t ;
- w_t là chiều rộng bounding box;
- h_t là chiều cao bounding box.

Tâm của bounding box là:

$$c_t = \begin{bmatrix} x_t + \frac{w_t}{2} \\ y_t + \frac{h_t}{2} \end{bmatrix} \quad (6.67)$$

Khi người dùng chọn mục tiêu ban đầu, hệ thống lưu bounding box tham chiếu:

$$B_0 = (x_0, y_0, w_0, h_0) \quad (6.68)$$

và tâm ban đầu:

$$c_0 = \begin{bmatrix} x_0 + \frac{w_0}{2} \\ y_0 + \frac{h_0}{2} \end{bmatrix} \quad (6.69)$$

Trong vùng bounding box ban đầu, hệ thống chọn một tập các điểm đặc trưng:

$$P_0 = \{p_1^0, p_2^0, \dots, p_N^0\} \quad (6.70)$$

Với mỗi frame mới, Optical Flow ước lượng vị trí mới của các điểm này:

$$P_t = \{p_1^t, p_2^t, \dots, p_N^t\} \quad (6.71)$$

Độ dịch chuyển của từng điểm là:

$$\Delta p_i = p_i^t - p_i^{t-1} \quad (6.72)$$

Từ các vector dịch chuyển của nhiều điểm, hệ thống có thể ước lượng độ dịch chuyển tổng thể của mục tiêu. Một cách đơn giản là lấy trung bình:

$$\Delta c = \frac{1}{N} \sum_{i=1}^N \Delta p_i \quad (6.73)$$

Tuy nhiên, trung bình có thể bị ảnh hưởng bởi các điểm theo dõi sai. Do đó, trong thực tế thường dùng trung vị để tăng tính bền vững:

$$\Delta x = \text{median}(\Delta x_1, \Delta x_2, \dots, \Delta x_N) \quad (6.74)$$

$$\Delta y = \text{median}(\Delta y_1, \Delta y_2, \dots, \Delta y_N) \quad (6.75)$$

Sau khi có độ dịch chuyển tổng thể, tâm bounding box được cập nhật:

$$c_t = c_{t-1} + \begin{bmatrix} \Delta x \\ \Delta y \end{bmatrix} \quad (6.76)$$

Từ tâm mới và kích thước hiện tại của bounding box, hệ thống suy ra vị trí bounding box mới. Như vậy, Optical Flow không nhất thiết cần nhận dạng lại toàn bộ vật thể, mà chỉ cần theo dõi sự dịch chuyển của các điểm đặc trưng đại diện cho vật thể.

1, Ước lượng thay đổi tỉ lệ mục tiêu

Ngoài vị trí tâm, kích thước của mục tiêu trong ảnh cũng là thông tin quan trọng. Khi ROV tiến lại gần mục tiêu, mục tiêu thường xuất hiện lớn hơn trong ảnh. Khi

ROV lùi xa, mục tiêu thường nhỏ hơn. Vì vậy, sự thay đổi diện tích bounding box có thể được sử dụng như một tín hiệu tương đối về khoảng cách.

Diện tích bounding box tại thời điểm t là:

$$A_t = w_t h_t \quad (6.77)$$

Diện tích ban đầu là:

$$A_0 = w_0 h_0 \quad (6.78)$$

Tỉ lệ thay đổi diện tích có thể được tính bằng:

$$r_A = \frac{A_t}{A_0} \quad (6.79)$$

Sai lệch diện tích so với ban đầu:

$$e_A = \frac{A_t - A_0}{A_0} \quad (6.80)$$

Nếu $e_A > 0$, mục tiêu đang lớn hơn so với lúc chọn ban đầu. Điều này thường tương ứng với việc ROV đang gần mục tiêu hơn. Nếu $e_A < 0$, mục tiêu nhỏ hơn so với ban đầu, thường tương ứng với việc ROV đang xa mục tiêu hơn. Nếu $e_A \approx 0$, kích thước mục tiêu gần với kích thước tham chiếu.

Trong một số trường hợp, thay vì dùng trực tiếp diện tích, có thể dùng căn bậc hai của tỉ lệ diện tích để biểu diễn thay đổi tỉ lệ tuyến tính của kích thước:

$$s = \sqrt{\frac{A_t}{A_0}} \quad (6.81)$$

Sai lệch tỉ lệ khi đó có thể viết:

$$e_s = s - 1 \quad (6.82)$$

Cách biểu diễn này hợp lý vì diện tích tỉ lệ với bình phương kích thước tuyến tính. Nếu chiều rộng và chiều cao của mục tiêu cùng tăng theo một hệ số s , diện tích sẽ tăng theo s^2 .

m, Chuẩn hóa sai lệch ảnh

Sai lệch tính bằng pixel phụ thuộc vào độ phân giải ảnh. Một sai lệch 40 pixel trên ảnh rộng 320 pixel có ý nghĩa lớn hơn nhiều so với sai lệch 40 pixel trên ảnh rộng 1920 pixel. Vì vậy, để bộ điều khiển không phụ thuộc quá mạnh vào độ phân giải camera, sai lệch ảnh thường được chuẩn hóa.

Giả sử ảnh có chiều rộng W và chiều cao H . Nếu tâm ảnh là:

$$c_{img} = \begin{bmatrix} \frac{W}{2} \\ \frac{H}{2} \end{bmatrix} \quad (6.83)$$

và tâm mục tiêu hiện tại là:

$$c_t = \begin{bmatrix} x_c \\ y_c \end{bmatrix} \quad (6.84)$$

thì sai lệch so với tâm ảnh là:

$$\Delta x_c = x_c - \frac{W}{2} \quad (6.85)$$

$$\Delta y_c = y_c - \frac{H}{2} \quad (6.86)$$

Sai lệch chuẩn hóa có thể được tính:

$$e_x = \frac{\Delta x_c}{W/2} \quad (6.87)$$

$$e_y = \frac{\Delta y_c}{H/2} \quad (6.88)$$

Khi đó, nếu mục tiêu nằm gần biên phải của ảnh, e_x gần 1. Nếu mục tiêu nằm gần biên trái, e_x gần -1 . Nếu mục tiêu ở gần tâm ảnh, e_x gần 0. Tương tự, e_y biểu diễn sai lệch theo phương dọc.

Trong một số thiết kế, sai lệch không được tính so với tâm ảnh mà tính so với vị trí mục tiêu tại thời điểm người dùng chọn. Khi đó, hệ thống sẽ cố giữ mục tiêu tại vị trí được chọn ban đầu thay vì đưa mục tiêu về đúng tâm ảnh. Cách chọn điểm tham chiếu phụ thuộc vào yêu cầu điều khiển của hệ thống.

n, Sai lệch frame-to-frame và sai lệch so với điểm tham chiếu

Khi dùng Optical Flow cho tracking, cần phân biệt hai loại sai lệch:

- sai lệch giữa hai frame liên tiếp;
- sai lệch giữa trạng thái hiện tại và trạng thái tham chiếu.

Sai lệch giữa hai frame liên tiếp mô tả mục tiêu vừa dịch chuyển bao nhiêu trong khoảng thời gian rất ngắn:

$$\Delta p_{t,t-1} = p_t - p_{t-1} \quad (6.89)$$

Sai lệch này phù hợp để cập nhật vị trí tracker theo thời gian. Tuy nhiên, nếu chỉ dùng sai lệch frame-to-frame làm đầu vào điều khiển, bộ điều khiển có thể chỉ phản ứng với vận tốc tương đối trong ảnh, không phản ánh trực tiếp việc mục tiêu đang lệch bao xa so với vị trí mong muốn.

Trong khi đó, sai lệch so với điểm tham chiếu được tính:

$$\Delta p_{t,ref} = p_t - p_{ref} \quad (6.90)$$

Trong đó p_{ref} có thể là tâm ảnh hoặc vị trí mục tiêu tại thời điểm người dùng chọn. Sai lệch này phản ánh trực tiếp mục tiêu đang nằm lệch bao nhiêu so với vị trí mong muốn. Đối với điều khiển phản hồi, sai lệch so với điểm tham chiếu thường phù hợp hơn vì mục tiêu của bộ điều khiển là đưa sai lệch về 0.

Trong bài toán ROV, nếu yêu cầu là giữ mục tiêu tại đúng vị trí ban đầu khi người dùng chọn, điểm tham chiếu nên là vị trí mục tiêu lúc khởi tạo tracking. Nếu yêu cầu là luôn đưa mục tiêu vào tâm ảnh, điểm tham chiếu nên là tâm khung hình. Việc xác định rõ điểm tham chiếu rất quan trọng vì nó quyết định ý nghĩa của tín hiệu sai số gửi cho bộ điều khiển.

o, Đánh giá độ tin cậy của tracking

Optical Flow có thể bị sai khi ảnh bị mờ, thiếu texture, mục tiêu bị che khuất, ánh sáng thay đổi hoặc camera rung mạnh. Do đó, hệ thống cần đánh giá độ tin cậy của kết quả tracking trước khi sử dụng nó cho điều khiển ROV.

Một số tiêu chí đánh giá độ tin cậy gồm:

- số lượng điểm đặc trưng còn theo dõi được;
- tỉ lệ điểm theo dõi hợp lệ so với số điểm ban đầu;
- sai số theo dõi thuận–ngược;

- mức phân tán của các vector dịch chuyển;
- diện tích bounding box có nằm trong khoảng hợp lý hay không;
- tốc độ thay đổi vị trí hoặc kích thước bounding box có bất thường hay không.

Giả sử ban đầu có N_0 điểm đặc trưng và tại frame hiện tại còn N_t điểm theo dõi hợp lệ. Một chỉ số đơn giản về độ tin cậy là:

$$c_N = \frac{N_t}{N_0} \quad (6.91)$$

Nếu c_N thấp, điều đó cho thấy nhiều điểm đặc trưng đã bị mất và kết quả tracking có thể không đáng tin cậy.

Một cách khác là dùng sai số forward-backward. Thuật toán trước hết theo dõi điểm từ frame $t - 1$ sang frame t , sau đó theo dõi ngược từ frame t về frame $t - 1$. Nếu điểm được theo dõi chính xác, vị trí quay ngược lại phải gần với vị trí ban đầu. Sai số forward-backward của điểm thứ i là:

$$e_i^{fb} = \|p_i^{t-1} - \hat{p}_i^{t-1}\| \quad (6.92)$$

Trong đó p_i^{t-1} là vị trí ban đầu của điểm ở frame trước, còn $\hat{p}_i^{t-1}$ là vị trí thu được khi tracking ngược lại. Nếu e_i^{fb} vượt quá một ngưỡng, điểm đó có thể bị loại bỏ.

Độ tin cậy tổng thể có thể được biểu diễn bằng một giá trị:

$$0 \leq c \leq 1 \quad (6.93)$$

Trong đó c càng gần 1 thì kết quả tracking càng đáng tin cậy. Bộ điều khiển chỉ nên sử dụng dữ liệu Optical Flow khi c lớn hơn một ngưỡng nhất định:

$$c \geq c_{min} \quad (6.94)$$

Nếu $c < c_{min}$, hệ thống nên coi mục tiêu là mất hoặc chưa đủ tin cậy, từ đó không gửi lệnh điều khiển tự động dựa trên vision.

p, Kiểm tra diện tích bounding box

Ngoài confidence, diện tích bounding box cũng là một tiêu chí quan trọng để kiểm tra tính hợp lệ của mục tiêu. Nếu bounding box quá nhỏ, mục tiêu có thể đang ở quá xa, bị mất hoặc không còn đủ thông tin ảnh để tracking ổn định. Nếu

bounding box quá lớn, mục tiêu có thể quá gần camera, bị cắt khỏi khung hình hoặc tracker đã bị trôi sang vùng nền.

Diện tích bounding box chuẩn hóa theo diện tích ảnh được tính:

$$\rho_A = \frac{A_t}{WH} \quad (6.95)$$

Trong đó:

- $A_t = w_t h_t$ là diện tích bounding box;
- W và H là chiều rộng và chiều cao ảnh.

Bounding box được xem là hợp lệ nếu:

$$\rho_{min} \leq \rho_A \leq \rho_{max} \quad (6.96)$$

Việc đặt ngưỡng dưới và ngưỡng trên giúp hệ thống loại bỏ các trường hợp tracking không đáng tin cậy. Trong môi trường dưới nước, kiểm tra này đặc biệt cần thiết vì nhiễu, bọt nước, phản xạ ánh sáng hoặc các vật thể nền có thể làm tracker bị trôi.

q. Quy trình Optical Flow trong hệ thống bám mục tiêu

Quy trình sử dụng Optical Flow cho bám mục tiêu trong hệ thống ROV có thể được mô tả như sau:

1. Camera thu luồng ảnh liên tục từ môi trường phía trước ROV.
2. Người vận hành chọn mục tiêu cần bám trên giao diện, thường bằng một bounding box hoặc một điểm chọn trong ảnh.
3. Hệ thống lưu trạng thái tham chiếu ban đầu của mục tiêu, bao gồm vị trí, kích thước bounding box và tập điểm đặc trưng.
4. Tại mỗi frame mới, thuật toán Optical Flow ước lượng vị trí mới của các điểm đặc trưng.
5. Các điểm theo dõi không hợp lệ được loại bỏ dựa trên trạng thái tracking, sai số forward-backward hoặc các tiêu chí chất lượng khác.
6. Từ các điểm còn hợp lệ, hệ thống ước lượng độ dịch chuyển tổng thể của mục tiêu.
7. Bounding box được cập nhật theo độ dịch chuyển và thay đổi tỉ lệ ước lượng.
8. Hệ thống tính sai lệch vị trí mục tiêu và sai lệch kích thước so với trạng thái

tham chiếu.

9. Các giá trị sai lệch, độ tin cậy và trạng thái tracking được gửi tới bộ điều khiển ROV.

Có thể tóm tắt pipeline như sau:

$$Frame_{t-1}, Frame_t \rightarrow OpticalFlow \rightarrow \Delta x, \Delta y, e_s, c \rightarrow B \text{ Điều khiển ROV} \quad (6.97)$$

Trong đó Δx và Δy mô tả sai lệch vị trí của mục tiêu trong ảnh, e_s mô tả sai lệch tỉ lệ kích thước, còn c là độ tin cậy của quá trình theo dõi.

r, Biến đổi sai lệch ảnh thành sai số điều khiển

Optical Flow không trực tiếp sinh lệnh cho động cơ đẩy. Thay vào đó, nó cung cấp các đại lượng mô tả trạng thái mục tiêu trong ảnh. Các đại lượng này được chuyển thành sai số điều khiển cho ROV.

Giả sử sai lệch ngang của mục tiêu trong ảnh là Δx . Sai số điều khiển theo phương ngang có thể được viết:

$$e_{lat} = k_x \Delta x \quad (6.98)$$

Trong đó k_x là hệ số chuyển đổi từ pixel sang sai số điều khiển. Nếu dùng sai lệch chuẩn hóa, có thể viết:

$$e_{lat} = k_x \frac{\Delta x}{W/2} \quad (6.99)$$

Sai số này được dùng để sinh lệnh dịch ngang cho ROV. Nếu mục tiêu lệch sang phải, ROV cần dịch theo chiều làm giảm sai lệch đó. Dấu của lệnh điều khiển phụ thuộc vào quy ước hệ tọa độ ảnh, hướng lắp camera và quy ước chiều dương của trục lateral.

Tương tự, sai lệch tỉ lệ kích thước e_s có thể được dùng để tạo sai số điều khiển tiến/lùi:

$$e_{range} = k_s e_s \quad (6.100)$$

Nếu mục tiêu nhỏ hơn kích thước tham chiếu, ROV có thể cần tiến lại gần. Nếu mục tiêu lớn hơn kích thước tham chiếu, ROV có thể cần lùi ra xa. Dấu của lệnh

tiền/lùi cần được chọn sao cho bộ điều khiển làm giảm sai lệch kích thước về 0.

Như vậy, đầu ra của Optical Flow có thể được ánh xạ sang vector sai số điều khiển:

$$\mathbf{e}_{vision} = \begin{bmatrix} e_{range} \\ e_{lat} \end{bmatrix} \quad (6.101)$$

Trong đó:

- e_{range} liên quan đến chuyển động tiến/lùi;
- e_{lat} liên quan đến chuyển động dịch ngang.

Sau đó, bộ điều khiển sẽ biến sai số này thành lệnh chuyển động cấp cao của ROV.

s, Liên hệ với điều khiển phản hồi

Trong hệ thống điều khiển phản hồi, mục tiêu là làm cho sai số giữa trạng thái hiện tại và trạng thái mong muốn tiến về 0. Với tám mục tiêu bằng camera, trạng thái mong muốn không nhất thiết là tọa độ 3D của mục tiêu, mà có thể là vị trí và kích thước mong muốn của mục tiêu trong ảnh.

Có thể định nghĩa vector trạng thái ảnh của mục tiêu:

$$\mathbf{s}_t = \begin{bmatrix} x_c \\ y_c \\ A_t \end{bmatrix} \quad (6.102)$$

Trong đó:

- x_c là tọa độ ngang của tâm mục tiêu;
- y_c là tọa độ dọc của tâm mục tiêu;
- A_t là diện tích bounding box hiện tại.

Trạng thái mong muốn:

$$\mathbf{s}^* = \begin{bmatrix} x_c^* \\ y_c^* \\ A^* \end{bmatrix} \quad (6.103)$$

Sai số thị giác:

$$\mathbf{e}_t = \mathbf{s}_t - \mathbf{s}^* = \begin{bmatrix} x_c - x_c^* \\ y_c - y_c^* \\ A_t - A^* \end{bmatrix} \quad (6.104)$$

Trong bài toán điều khiển ROV, có thể chỉ sử dụng một số thành phần của vector sai số này. Ví dụ, sai lệch ngang $x_c - x_c^*$ dùng cho điều khiển lateral, còn sai lệch diện tích $A_t - A^*$ dùng cho điều khiển tiến/lùi. Thành phần sai lệch dọc $y_c - y_c^*$ có thể dùng cho điều khiển lên/xuống nếu hệ thống muốn bám mục tiêu theo cả phương dọc ảnh, hoặc có thể không dùng nếu độ sâu được giữ bởi một bộ điều khiển riêng.

t, Ưu điểm của Optical Flow trong hệ thống ROV

Optical Flow có một số ưu điểm phù hợp với bài toán Track and Catch ROV.

Thứ nhất, Optical Flow không yêu cầu mô hình nhận dạng vật thể được huấn luyện trước. Người vận hành có thể chọn bất kỳ mục tiêu nào trong khung hình, miễn là mục tiêu có đủ đặc trưng ảnh để theo dõi. Điều này giúp hệ thống linh hoạt hơn so với các phương pháp chỉ nhận diện được một số lớp vật thể cố định.

Thứ hai, Optical Flow có chi phí tính toán tương đối thấp nếu sử dụng dạng sparse. Hệ thống chỉ cần theo dõi một số điểm đặc trưng trong vùng mục tiêu, thay vì xử lý toàn bộ ảnh bằng mạng neural lớn. Điều này phù hợp với các nền tảng tính toán hạn chế trên ROV.

Thứ ba, Optical Flow tận dụng tính liên tục thời gian của video. Vì camera cung cấp chuỗi frame liên tiếp, vị trí mục tiêu ở frame hiện tại thường gần với vị trí ở frame trước. Thuật toán có thể khai thác đặc điểm này để tracking thời gian thực.

Thứ tư, đầu ra của Optical Flow có dạng sai lệch pixel và sai lệch kích thước, rất phù hợp để đưa vào bộ điều khiển phản hồi. Bộ điều khiển không cần biết mục tiêu là loại vật thể gì, mà chỉ cần biết mục tiêu đang lệch bao nhiêu so với vị trí mong muốn trong ảnh.

u, Giới hạn của Optical Flow trong môi trường dưới nước

Mặc dù phù hợp với bài toán bám mục tiêu thời gian thực, Optical Flow cũng có các giới hạn cần được phân tích trong đồ án.

Thứ nhất, Optical Flow phụ thuộc mạnh vào texture của mục tiêu. Nếu mục tiêu có bề mặt trơn, ít chi tiết hoặc màu sắc đồng nhất, thuật toán khó chọn được các điểm đặc trưng ổn định.

Thứ hai, Optical Flow nhạy với thay đổi ánh sáng. Trong môi trường dưới nước,

ánh sáng có thể thay đổi do phản xạ, tán xạ, đèn chiếu của ROV hoặc sự thay đổi hướng camera. Những thay đổi này có thể làm giả thiết bảo toàn độ sáng không còn đúng.

Thứ ba, ảnh dưới nước thường bị nhiễu, mờ và giảm tương phản. Điều này làm giảm chất lượng gradient ảnh, khiến việc tính I_x , I_y và I_t kém ổn định hơn.

Thứ tư, Optical Flow có thể mất mục tiêu khi vật thể chuyển động quá nhanh giữa hai frame. Nếu độ dịch chuyển quá lớn, điểm đặc trưng ở frame trước có thể không còn nằm gần vị trí tìm kiếm ở frame sau.

Thứ năm, thuật toán có thể bị drift. Drift là hiện tượng tracker dần dần trôi khỏi mục tiêu ban đầu do tích lũy sai số qua nhiều frame. Khi drift xảy ra, bounding box có thể bám sang nền hoặc vật thể khác.

Thứ sáu, Optical Flow chỉ cung cấp chuyển động biểu kiến trong ảnh, không trực tiếp cung cấp khoảng cách thật đến mục tiêu. Việc suy luận khoảng cách từ kích thước bounding box chỉ mang tính tương đối và phụ thuộc vào hình dạng, hướng nhìn và kích thước thật của vật thể.

Vì các lý do trên, hệ thống cần kết hợp Optical Flow với các cơ chế kiểm tra độ tin cậy, ngưỡng diện tích bounding box, giới hạn tốc độ thay đổi và cơ chế mất mục tiêu để đảm bảo an toàn khi đưa dữ liệu vision vào điều khiển ROV.

v, Các trường hợp lỗi thường gặp

Trong quá trình vận hành, một hệ thống tracking dựa trên Optical Flow có thể gặp một số trường hợp lỗi sau.

Trường hợp thứ nhất là mất đặc trưng. Khi số điểm đặc trưng còn lại quá ít, hệ thống không còn đủ thông tin để ước lượng chuyển động của mục tiêu. Khi đó, confidence cần giảm xuống và bộ điều khiển không nên tiếp tục dùng dữ liệu vision.

Trường hợp thứ hai là tracking nhầm nền. Điều này xảy ra khi các điểm đặc trưng ban đầu không nằm hoàn toàn trên mục tiêu, hoặc khi mục tiêu rời khỏi vùng ảnh nhưng các điểm trên nền vẫn được theo dõi. Kết quả là tracker vẫn báo có chuyển động, nhưng chuyển động đó không còn đại diện cho mục tiêu.

Trường hợp thứ ba là biến dạng bounding box. Nếu một số điểm theo dõi sai lệch quá lớn, kích thước bounding box có thể tăng hoặc giảm bất thường. Kiểm tra diện tích bounding box và giới hạn tỉ lệ thay đổi giúp phát hiện trường hợp này.

Trường hợp thứ tư là sai lệch do camera rung. Khi toàn bộ ảnh dịch chuyển vì camera rung hoặc ROV quay nhanh, Optical Flow có thể ghi nhận chuyển động

lớn dù mục tiêu không thật sự di chuyển tương đối theo cách mong muốn. Trong trường hợp này, cần phối hợp với dữ liệu IMU hoặc giới hạn tốc độ lệnh điều khiển để tránh phản ứng quá mạnh.

Trường hợp thứ năm là thay đổi ánh sáng đột ngột. Khi đèn ROV thay đổi công suất, hoặc mục tiêu đi qua vùng phản xạ mạnh, cường độ ảnh thay đổi làm vi phạm giả thiết bảo toàn độ sáng. Điều này có thể làm tracker mất ổn định.

w, Vai trò của tiền xử lý ảnh

Để Optical Flow hoạt động ổn định hơn, ảnh đầu vào thường được tiền xử lý trước khi tracking. Các bước tiền xử lý có thể gồm:

- chuyển ảnh màu sang ảnh xám;
- giảm nhiễu bằng bộ lọc làm mượt nhẹ;
- cân bằng sáng hoặc tăng tương phản;
- giới hạn vùng xử lý trong bounding box;
- loại bỏ các frame lỗi hoặc quá mờ.

Việc chuyển sang ảnh xám giúp giảm số kênh xử lý và phù hợp với công thức Optical Flow dựa trên cường độ ảnh $I(x, y, t)$. Làm mượt nhẹ giúp giảm nhiễu cao tần, từ đó làm gradient ảnh ổn định hơn. Tuy nhiên, nếu làm mượt quá mạnh, các chi tiết và góc quan trọng có thể bị mất, làm giảm số điểm đặc trưng tốt.

Trong môi trường dưới nước, tăng tương phản có thể cải thiện khả năng phát hiện đặc trưng, nhưng cũng cần tránh khuếch đại nhiễu quá mức. Do đó, tiền xử lý ảnh cần được lựa chọn cân bằng giữa giảm nhiễu và giữ lại chi tiết mục tiêu.

x, Tái khởi tạo điểm đặc trưng

Trong quá trình tracking dài, số lượng điểm đặc trưng có thể giảm dần do mất tracking, che khuất hoặc mục tiêu thay đổi hình dạng. Nếu không bổ sung điểm mới, hệ thống có thể mất khả năng theo dõi sau một thời gian.

Do đó, một cơ chế thường được sử dụng là tái khởi tạo điểm đặc trưng trong bounding box hiện tại. Khi số điểm hợp lệ giảm dưới một ngưỡng:

$$N_t < N_{min} \quad (6.105)$$

hệ thống có thể phát hiện lại các điểm đặc trưng mới trong vùng bounding box hiện tại. Điều này giúp duy trì đủ số điểm để tracking. Tuy nhiên, cần cẩn thận vì nếu bounding box đã bị drift sang nền, việc tái khởi tạo sẽ củng cố lỗi tracking. Vì vậy, tái khởi tạo nên được thực hiện khi confidence vẫn đủ cao hoặc khi có thêm

tiêu chí xác nhận mục tiêu.

y, Sử dụng thống kê bền vững để giảm ảnh hưởng của outlier

Không phải mọi điểm được Optical Flow theo dõi đều chính xác. Một số điểm có thể trôi sang nền, bị nhiễu hoặc bị che khuất. Những điểm này được gọi là outlier. Nếu dùng trung bình cộng đơn giản để tính độ dịch chuyển tổng thể, outlier có thể làm kết quả sai lệch.

Để giảm ảnh hưởng của outlier, hệ thống có thể dùng các thống kê bền vững như trung vị:

$$\Delta x = \text{median}(\Delta x_1, \Delta x_2, \dots, \Delta x_N) \quad (6.106)$$

$$\Delta y = \text{median}(\Delta y_1, \Delta y_2, \dots, \Delta y_N) \quad (6.107)$$

Ngoài ra, có thể loại bỏ các điểm có vector dịch chuyển quá khác biệt so với phần lớn các điểm còn lại. Ví dụ, tính độ lệch của từng vector so với vector trung vị:

$$r_i = \|\Delta p_i - \text{median}(\Delta p)\| \quad (6.108)$$

Nếu:

$$r_i > r_{max} \quad (6.109)$$

điểm thứ i có thể được xem là outlier và bị loại bỏ.

Cách xử lý này giúp bounding box cập nhật ổn định hơn, đặc biệt trong trường hợp một phần mục tiêu bị che khuất hoặc một số điểm đặc trưng chuyển sang nền.

z, Mối quan hệ giữa Optical Flow và điều khiển Track and Catch

Mục tiêu của bài toán Track and Catch không chỉ là phát hiện mục tiêu trong ảnh, mà còn điều khiển ROV di chuyển để tiếp cận và duy trì mục tiêu ở vị trí phù hợp. Vì vậy, Optical Flow là khối cảm nhận, còn bộ điều khiển ROV là khối hành động.

Có thể mô tả quan hệ này như sau:

Camera → Optical Flow → Sai số thị giác → Bộ điều khiển → Thruster → Chuyển Động ROV
(6.110)

Trong chuỗi này, Optical Flow cung cấp phản hồi thị giác. Nếu mục tiêu lệch sang phải trong ảnh, sai số ngang sẽ khác 0. Bộ điều khiển dùng sai số này để tạo lệnh dịch ngang làm giảm sai lệch. Nếu mục tiêu nhỏ hơn kích thước mong muốn, sai số tỉ lệ cho biết ROV cần tiến lại gần. Khi ROV di chuyển, hình ảnh camera thay đổi, Optical Flow tiếp tục cập nhật sai số mới. Đây là một vòng điều khiển kín dựa trên thị giác.

Về bản chất, hệ thống đang thực hiện điều khiển servo thị giác đơn giản, trong đó đặc trưng ảnh của mục tiêu được dùng làm tín hiệu phản hồi. Trạng thái cần điều khiển không phải là vị trí 3D tuyệt đối của mục tiêu, mà là vị trí và kích thước của mục tiêu trong mặt phẳng ảnh.

CHƯƠNG 7. QUẢN LÝ THÔNG TIN VÀ PHÂN TÍCH DỮ LIỆU

Như đã trình bày trong Chương 3, hệ thống phần mềm ROV được thiết kế thành ba thành phần độc lập: Firmware nhúng trên ROV, phần mềm điều khiển mặt đất (GCS) và **Hệ thống lưu trữ và đồng bộ dữ liệu sau nhiệm vụ**. Trong khi các chương trước đã trình bày chi tiết việc xây dựng Firmware nhúng và phần mềm GCS, chương này tập trung vào thành phần thứ ba — phục vụ giai đoạn hậu kỳ: sau khi ROV trở về bờ, toàn bộ dữ liệu thu thập được tải lên, tổ chức và phân tích tập trung trên nền tảng web đa người dùng.

7.1 Tổng quan phân hệ

7.1.1 Bài toán đặt ra

Trong thực tế vận hành ROV, quá trình phân tích dữ liệu sau chuyến đi đang tồn tại nhiều bất cập. Phần cứng trên ROV ghi lại dữ liệu cảm biến (nhiệt độ, áp suất, độ sâu) dưới dạng tệp CSV, đồng thời lưu video vào bộ nhớ cục bộ. Do môi trường thực địa không có kết nối mạng ổn định, toàn bộ quá trình upload diễn ra sau khi đoàn về bờ. Công tác phân tích hậu kỳ hiện tại diễn ra hoàn toàn rời rạc: người phân tích phải mở trình phát video, sau đó đối chiếu thủ công từng dòng trong tệp CSV để xác định trạng thái môi trường tại một khoảnh khắc cụ thể trong video. Quy trình này tốn nhiều thời gian, dễ sai sót, và gây khó khăn khi chia sẻ kết quả giữa nhiều thành viên trong đội.

Khảo sát các phần mềm hiện có cho thấy chưa có giải pháp nào đáp ứng đầy đủ nhu cầu quản lý dữ liệu hậu kỳ cho ROV trên nền tảng web đa người dùng. Bảng 7.1 tóm tắt so sánh với các giải pháp phổ biến.

Bảng 7.1: So sánh các giải pháp quản lý dữ liệu ROV hiện có

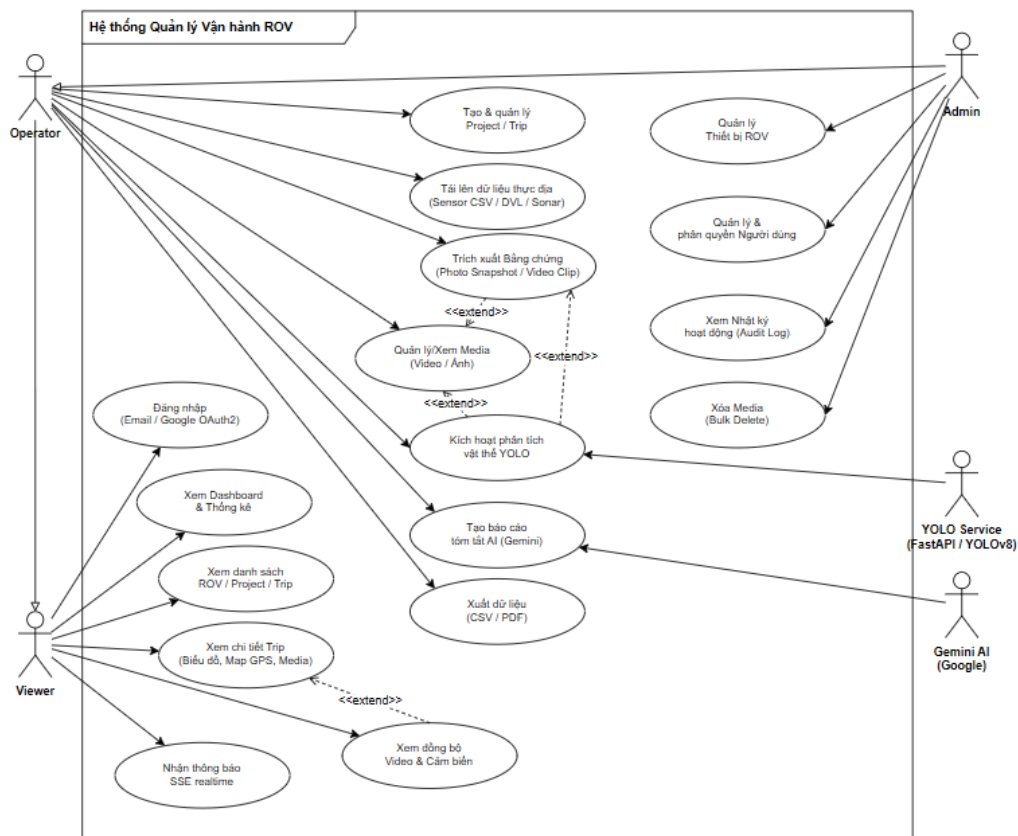
Tiêu chí	QGroundControl	BlueOS	Google Drive	Đề xuất
Phân tích dữ liệu hậu kỳ	Hạn chế	—	—	✓
Đồng bộ Video – Cảm biến	—	—	—	✓
Nhận diện vật thể AI (YOLO)	—	—	—	✓
Tóm tắt nhiệm vụ bằng LLM	—	—	—	✓
Nền tảng Web đa người dùng	—	Hạn chế	✓	✓
Phân quyền theo vai trò (RBAC)	—	—	—	✓

Từ phân tích trên, phân hệ “*Quản lý Thông tin Nhiệm vụ ROV*” được xây dựng với bốn tính năng cốt lõi: (i) quản lý tập trung các Chuyến khảo sát (Projects) và Lướt lặn (Trips) theo mô hình phân cấp; (ii) ánh xạ và đồng bộ thanh thời gian video với biểu đồ dữ liệu cảm biến theo thời gian thực; (iii) tích hợp học sâu (YOLOv8)

để tự động nhận diện vật thể và trích xuất bằng chứng từ video; (iv) ứng dụng Mô hình ngôn ngữ lớn (Gemini) để tổng hợp báo cáo nhiệm vụ bằng ngôn ngữ tự nhiên.

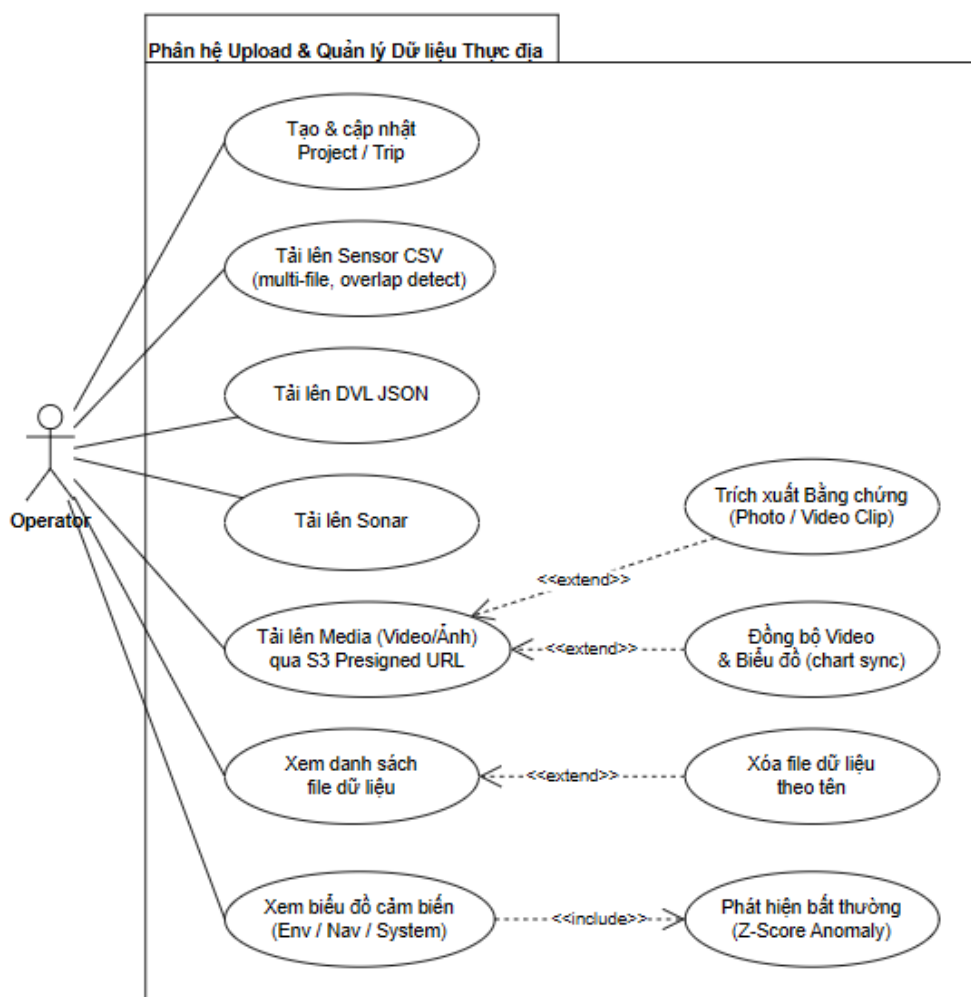
7.1.2 Biểu đồ use case tổng quát

Biểu đồ Use Case tổng quát (Hình 7.1) mô tả sự tương tác giữa ba nhóm người dùng (Viewer, Operator, Admin) và hai tác nhân hệ thống bên ngoài (YOLO Service và Gemini AI). Quyền hạn được tổ chức theo mô hình kế thừa tăng dần: Viewer chỉ xem, Operator thêm quyền tải lên và phân tích, Admin có toàn quyền bao gồm xóa tài nguyên và quản lý người dùng.

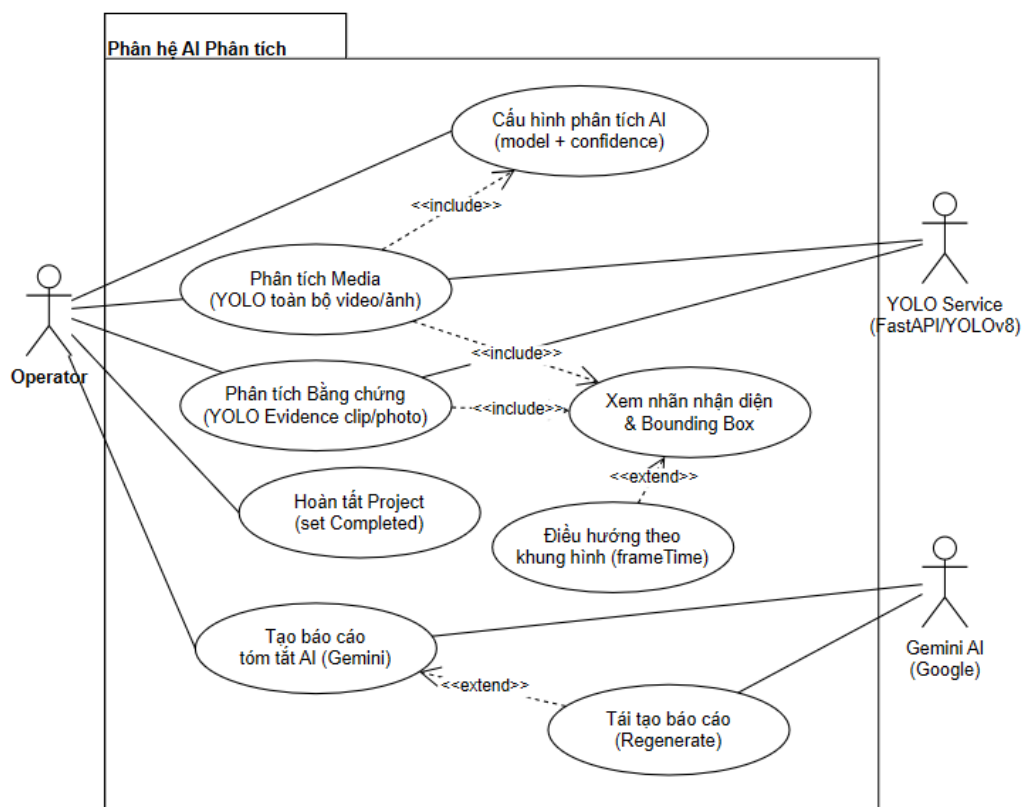


Hình 7.1: Biểu đồ Use Case tổng quát phân hệ quản lý thông tin ROV

Hình 7.2 và Hình 7.3 làm rõ hai phân hệ quan trọng nhất. Phân hệ Dữ liệu Thực địa mô tả luồng upload đa loại tệp (CSV, DVL, Sonar, Media), quản lý theo tên file nguồn (`sourceFile`), và kích hoạt chế độ đồng bộ Video–Biểu đồ khi video có trường `recordedAt`. Phân hệ AI Phân tích cho thấy Operator cấu hình mô hình và ngưỡng confidence trước khi chạy phân tích bất đồng bộ qua Bull Queue, kết quả được đẩy về qua SSE.

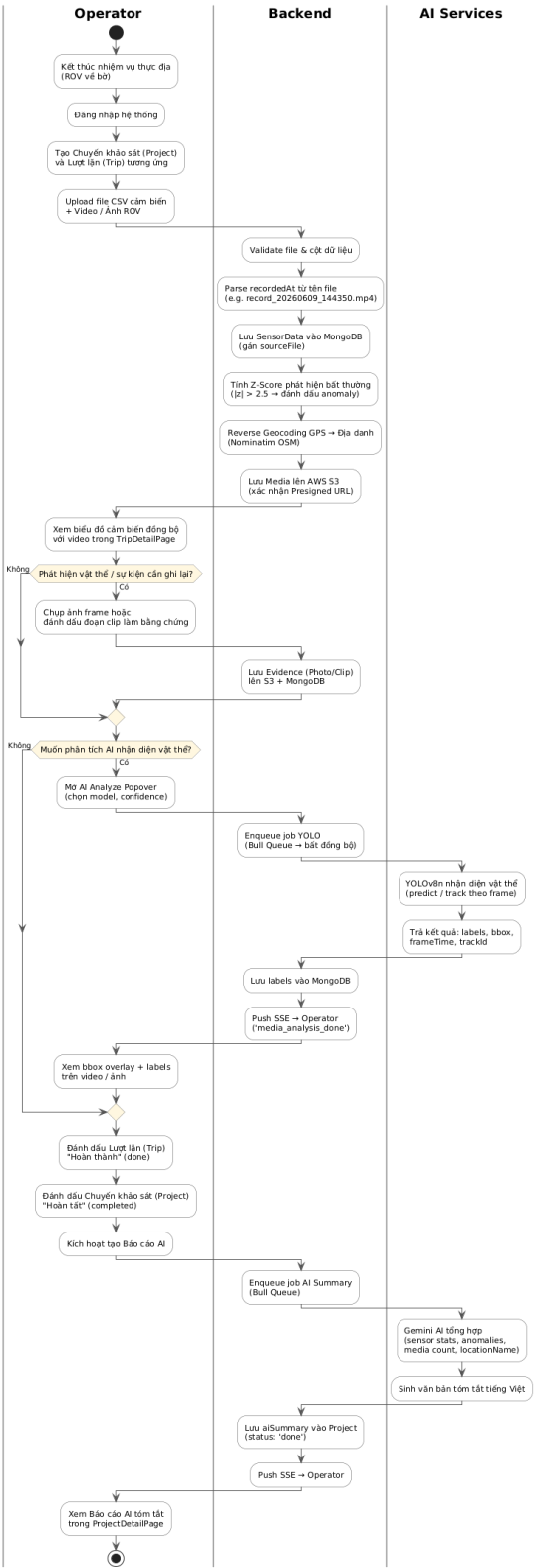


Hình 7.2: Biểu đồ Use Case phân rã – Phân hệ Upload và Quản lý Dữ liệu Thực địa



Hình 7.3: Biểu đồ Use Case phân rã – Phân hệ AI Phân tích

Hình 7.4 tóm tắt luồng nghiệp vụ đầu cuối từ khi Operator trở về từ thực địa đến khi có báo cáo tổng kết, gồm năm giai đoạn: đăng nhập và tạo mới Project/Trip; tải lên dữ liệu; xem lại video và trích xuất bằng chứng; đánh dấu hoàn tất Trip; kích hoạt AI tóm tắt.



Hình 7.4: Biểu đồ hoạt động – Quy trình phân tích dữ liệu nhiệm vụ ROV

7.1.3 Yêu cầu phi chức năng

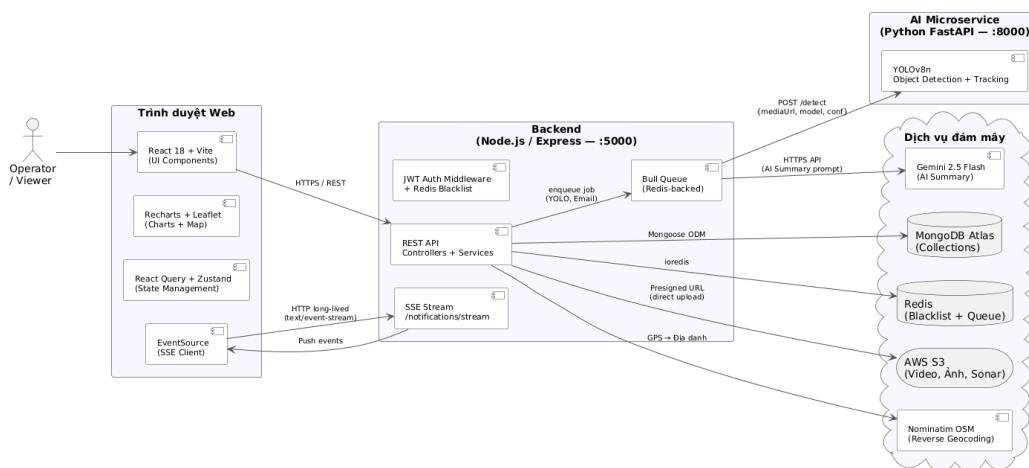
Để đảm bảo hệ thống hoạt động ổn định và mang lại trải nghiệm tối ưu trong môi trường vận hành thực tế, phân hệ web phải thỏa mãn các yêu cầu phi chức năng sau:

- **Hiệu năng và phản hồi:** Giao diện cockpit theo dõi video và cảm biến không được giật lag. Cơ chế đồng bộ thanh thời gian video–biểu đồ phải duy trì độ trễ dưới 100 ms. Thông báo SSE phải hiển thị tức thì khi có thay đổi trạng thái.
- **Xử lý bất đồng bộ:** Các tác vụ nặng như phân tích hàng nghìn khung hình video bằng YOLOv8 hoặc truy vấn LLM (Gemini) không được chặn luồng giao diện người dùng. Tất cả tác vụ này phải được xếp vào hàng đợi (Bull Queue) và xử lý ngầm, kết quả trả về qua SSE.
- **Khả năng lưu trữ và mở rộng:** Tập dữ liệu thực địa (video, ảnh, sonar) thường có kích thước lớn (Gigabytes). Hệ thống không lưu tệp trực tiếp trong cơ sở dữ liệu mà sử dụng nền tảng lưu trữ đám mây (AWS S3) với cơ chế Presigned URL, giảm tải băng thông cho máy chủ chính.
- **Bảo mật và phân quyền:** Hệ thống sử dụng JWT có vòng đời ngắn (15 phút) kết hợp Refresh Token (7 ngày) và danh sách đen (Blacklist) trên Redis để vô hiệu hóa tức thì tài khoản đăng xuất hoặc bị chặn. Mọi truy cập tài nguyên đều kiểm tra quyền tại lớp middleware theo ma trận RBAC ba cấp.
- **Khả năng sử dụng:** Giao diện hỗ trợ cả chế độ sáng và tối, tương thích trên màn hình từ 375 px (mobile) đến 1440 px (desktop). Tất cả danh sách có trạng thái trống (empty state) thân thiện; các thao tác tải lên có phản hồi tiến trình rõ ràng.

7.2 Kiến trúc và hệ sinh thái công nghệ

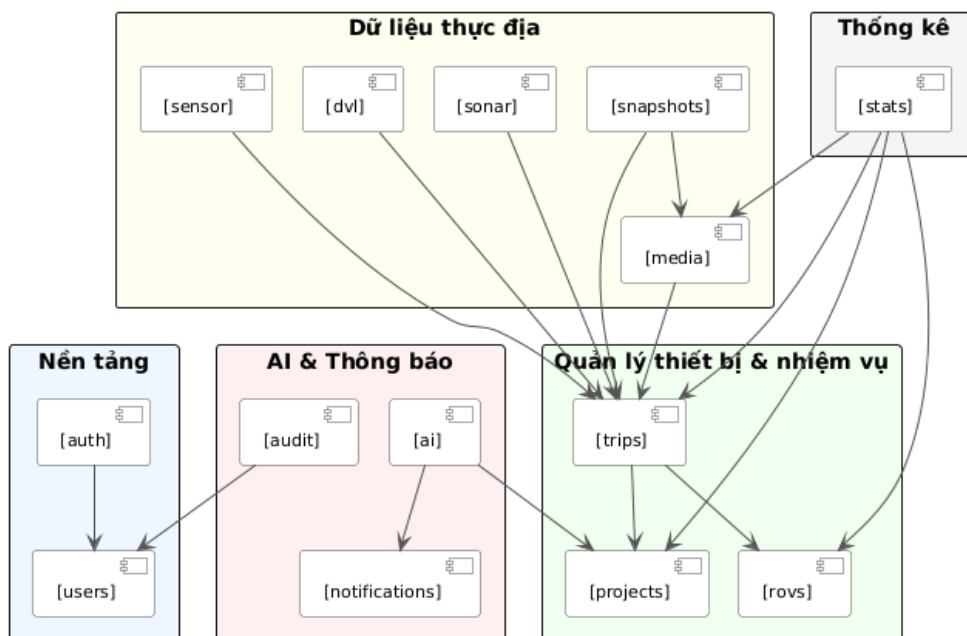
7.2.1 Kiến trúc tổng quan

Phân hệ được thiết kế theo kiến trúc **Client-Server đa tầng** kết hợp mô hình **Microservice thu nhỏ** để tách biệt bộ phận nhận diện hình ảnh nặng nề (YOLOv8) khỏi máy chủ chính. Ba tầng gồm: tầng trình bày (React SPA), tầng nghiệp vụ (Node.js) và tầng dữ liệu (MongoDB Atlas).



Hình 7.5: Kiến trúc tổng quan phân hệ quản lý thông tin ROV

Như Hình 7.5, Frontend được phân phối qua Vercel CDN và giao tiếp với Backend Node.js thông qua REST API. Backend đảm nhận toàn bộ nghiệp vụ: xác thực JWT, quản lý hàng đợi Bull, xử lý file AWS S3 và điều phối các dịch vụ ngoài (Gemini API, YOLOv8 Microservice, Nominatim). Hình 7.6 thể hiện sự phân chia module phía Backend theo nguyên tắc *high cohesion, low coupling*: mỗi thực thể nghiệp vụ (Projects, Trips, Media, Sensor, DVL, Sonar, Snapshots, Notifications, AI, Audit) có bộ tệp độc lập gồm `routes.js` → `controller.js` → `service.js` → `model.js`.



Hình 7.6: Sơ đồ phụ thuộc module phía Backend

7.2.2 Hệ sinh thái công nghệ

Bảng 7.2 trình bày hệ sinh thái công nghệ của phân hệ web, được lựa chọn để đáp ứng các yêu cầu phi chức năng: đồng bộ video–biểu đồ dưới 100ms, xử lý tác vụ AI nặng không block giao diện, và hỗ trợ dark mode từ 375px đến 1440px.

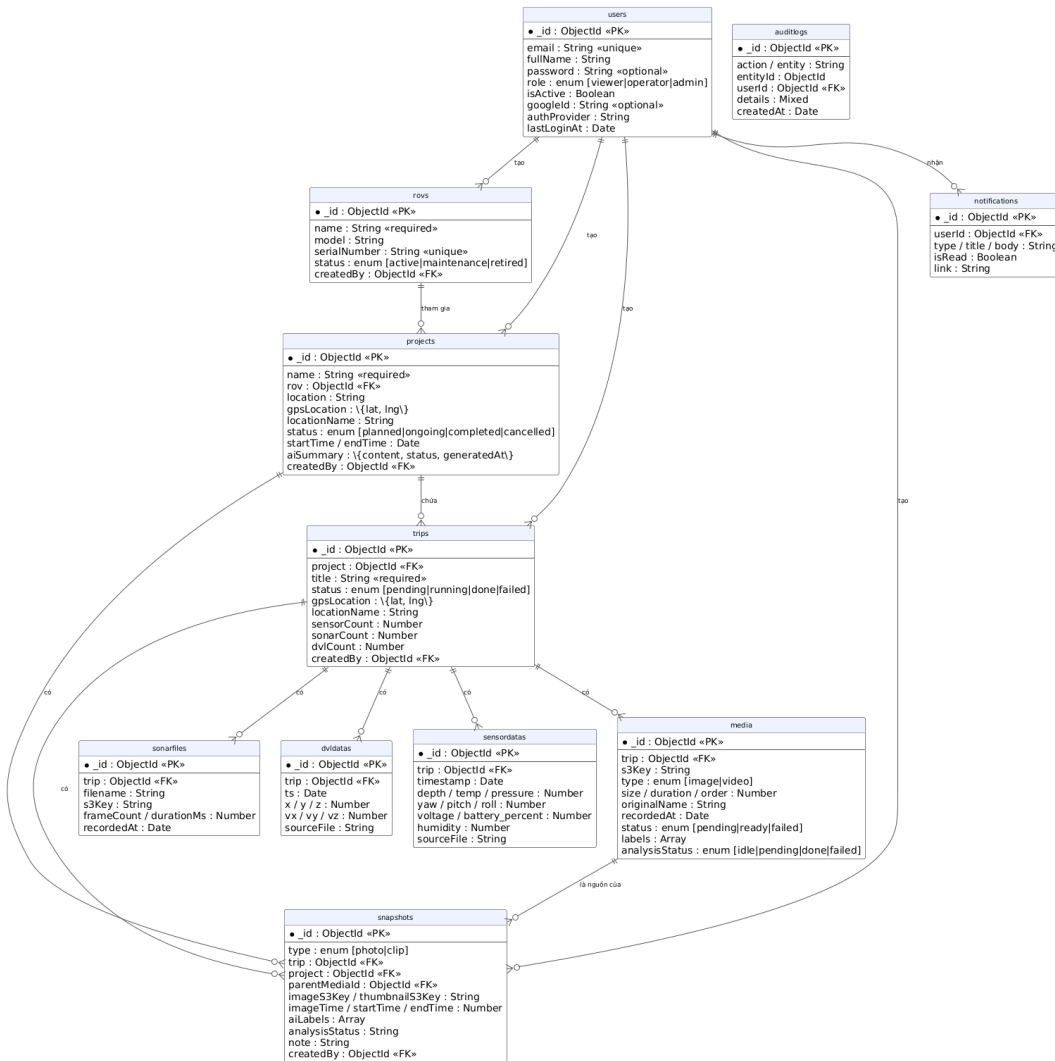
Bảng 7.2: Hệ sinh thái công nghệ phân hệ quản lý thông tin

Tầng / Vai trò	Công nghệ	Ghi chú
Giao diện người dùng	React 18.3, Vite 5.3, Tailwind CSS 3.4	SPA, dark/light mode, 375–1440px
Quản lý trạng thái	Zustand 4.5, React Query 5.4	Client state + server cache
Biểu đồ	Recharts 3.8	AreaChart, LineChart, Brush
Bản đồ	Leaflet.js 1.9 (raw API)	OpenStreetMap, không cần API key
Máy chủ	Node.js 20 LTS, Express.js 4.19	REST API, SSE stream
ORM	Mongoose 8.4	Schema, validation, compound index
Cơ sở dữ liệu	MongoDB Atlas (M0 Free)	NoSQL document, 512 MB
Cache & hàng đợi	Redis + Bull 4.16	JWT blacklist + 4 async job queues
Lưu trữ tệp	AWS S3 + Presigned URL	Upload thẳng từ trình duyệt, max 500 MB/file
Xác thực	JWT (HS256) + Passport.js	Access 15 phút, Refresh 7 ngày
Giao tiếp realtime	Server-Sent Events (SSE)	Một chiều: server → client
Nhận diện vật thể	YOLOv8n + FastAPI, Python 3.11	Microservice độc lập, cổng 8000
LLM tóm tắt	Gemini 2.5 Flash	@google/generative-ai
Phát hiện bất thường	Z-Score (tự cài đặt)	Ngưỡng $ z > 2,5$
Reverse Geocoding	OpenStreetMap Nominatim	Miễn phí, không cần API key

Hai cơ chế kỹ thuật trọng tâm là **Bull Queue** và **SSE**. Bull Queue đảm bảo “at-least-once delivery” cho các tác vụ nặng (phân tích YOLO, gọi Gemini API) không được phép chặn Event Loop của Node.js. Hệ thống khởi tạo bốn hàng đợi độc lập: `media-analysis` (timeout 25 phút, 1 lần thử), `snapshot-analysis` (5 phút, 1 lần thử), `ai-summary` (60 giây, 3 lần thử backoff) và `email` (60 giây, 3 lần thử). SSE qua HTTP/1.1 được chọn thay vì WebSocket vì chỉ cần giao tiếp một chiều (server → client); kết nối SSE được lưu trong cấu trúc `Map<userId, res>` in-memory, Worker sau khi xử lý xong tìm đúng connection của người dùng và ghi trực tiếp vào response stream.

7.2.3 Thiết kế cơ sở dữ liệu

Hệ thống sử dụng MongoDB với 11 collection chính, có cấu trúc phân cấp trung tâm: **Users** → **Projects** → **Trips** → dữ liệu con (SensorData, DVLDData, SonarFile, Media, Snapshots). Hình 7.7 mô tả quan hệ giữa các collection dưới dạng sơ đồ ER.



Hình 7.7: Sơ đồ ER cơ sở dữ liệu phân hệ quản lý thông tin ROV

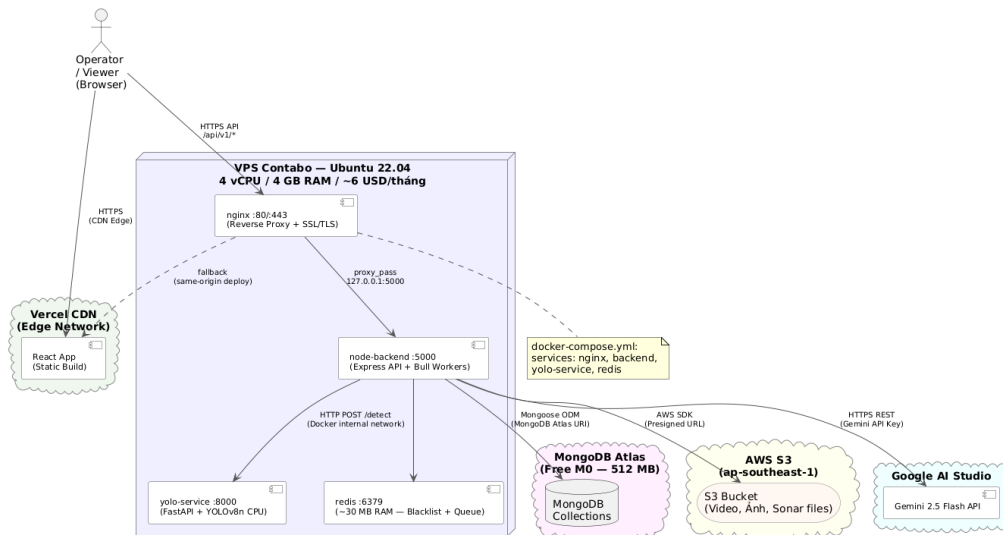
Bảng 7.3: Mô tả các collection chính trong MongoDB

Collection	Trường quan trọng	Mô tả
users	email, role, googleId, is-Active	Người dùng; role: viewer/operator/admin
rovs	name, serialNumber, status	Thiết bị ROV; status: active/maintenance/retired
projects	rov, status, aiSummary	Chuyến khảo sát; status: planned/ongoing/completed/cancelled
trips	project, status, gpsLocation, sensorCount	Lần lặn; status: pending/running/done/failed
sensordatas	trip, timestamp, depth, temp, sourceFile	Chuỗi thời gian cảm biến; index {trip, timestamp}
dvldatas	trip, ts, x, y, z, sourceFile	Quỹ đạo DVL theo thời gian
sonarfiles	trip, filename, frameCount, durationMs	Metadata file sonar nhị phân trên S3
media	trip, type, labels, analysisStatus	Video/ảnh; labels từ YOLO; analysisStatus: idle/pending/done/failed
snapshots	trip, type, imageS3Key, aiLabels	Ảnh frame hoặc clip bằng chứng
notification	userId, type, isRead, link	Thông báo in-app qua SSE
auditlogs	action, entity, userId, details	Nhật ký hoạt động admin

Index phức hợp được đặt tại các collection truy vấn thường xuyên: `SensorData{trip, timestamp, -1}`, `Media{trip, order}`, `Notification{userId, isRead, createdAt}` nhằm đảm bảo hiệu năng khi dataset tăng trưởng.

7.2.4 Mô hình triển khai đề xuất

Mô hình triển khai đề xuất (Hình 7.8) sử dụng Docker Compose trên một VPS duy nhất để đóng gói bốn dịch vụ: `nginx` (reverse proxy + SSL), `node-backend` (Express API + Bull Workers), `yolo-service` (FastAPI + YOLOv8n), `redis` (JWT blacklist + hàng đợi). Frontend triển khai riêng trên Vercel CDN; cơ sở dữ liệu trên MongoDB Atlas; tệp media trên AWS S3. Ước tính chi phí hạ tầng khoảng \$7/tháng (VPS \$6 + S3 <\$1, MongoDB Atlas và Gemini API ở mức free tier).



Hình 7.8: Mô hình triển khai đề xuất trên VPS sử dụng Docker Compose

7.3 Phân quyền và xác thực

7.3.1 Mô hình phân quyền RBAC

Hệ thống phục vụ ba nhóm người dùng với quyền hạn phân chia theo cơ chế kiểm soát truy cập dựa trên vai trò (RBAC): **Viewer** chỉ xem dữ liệu; **Operator** kế thừa quyền Viewer và thêm quyền tải lên, phân tích, trích xuất bằng chứng; **Admin** kế thừa toàn bộ quyền Operator và bổ sung đặc quyền xóa tài nguyên và quản lý người dùng. Phân quyền được thực thi tại middleware `authorize(roles)` trên mọi route Backend; Frontend ẩn các nút không phù hợp với role nhưng Backend vẫn kiểm tra độc lập để tránh bypass qua API trực tiếp.

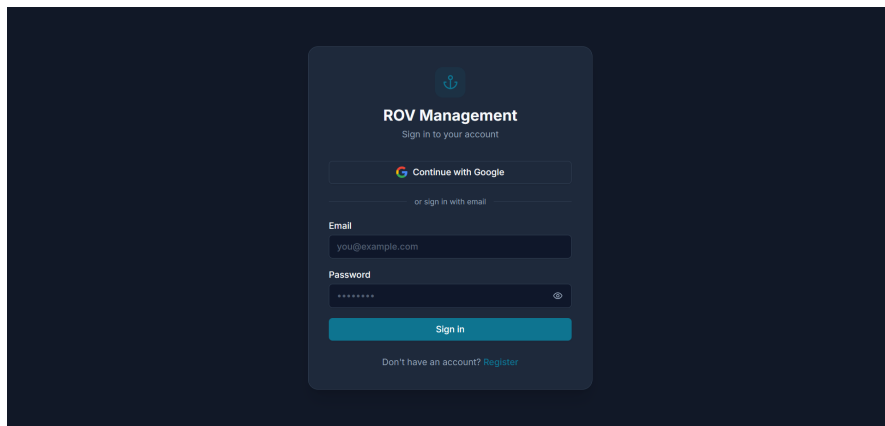
Bảng 7.4: Ma trận phân quyền RBAC của phân hệ quản lý thông tin

Tính năng	Admin	Operator	Viewer
Xem Dashboard, ROV, Project, Trip	✓	✓	✓
Nhận thông báo realtime (SSE)	✓	✓	✓
Tạo và chỉnh sửa Project / Trip	✓	✓	—
Tạo và chỉnh sửa Thiết bị ROV	✓	✓	—
Tải lên dữ liệu cảm biến (CSV / DVL / Sonar)	✓	✓	—
Xóa dữ liệu cảm biến / DVL / Sonar	✓	✓	—
Tải lên Media (Video / Ảnh)	✓	✓	—
Trích xuất Bằng chứng (Photo / Clip)	✓	✓	—
Xóa Bằng chứng (Photo / Clip)	✓	✓	—
Kích hoạt phân tích YOLO	✓	✓	—
Tạo báo cáo tóm tắt AI	✓	✓	—
Xuất dữ liệu (CSV / PDF)	✓	✓	—
Xóa Media (Ảnh / Video)	✓	—	—
Xóa Project / Trip	✓	—	—
Xóa Thiết bị ROV	✓	—	—
Quản lý và phân quyền Người dùng	✓	—	—
Xem Nhật ký hoạt động (Audit Log)	✓	—	—

7.3.2 Cơ chế xác thực JWT và Google OAuth

Hệ thống sử dụng cặp token có vòng đời bất đối xứng: **Access Token** (HS256, TTL 15 phút) gửi kèm mọi request dưới dạng `Authorization: Bearer; Refresh Token` (TTL 7 ngày) lưu trong MongoDB. Khi Access Token hết hạn, Axios interceptor tự động gọi `POST /auth/refresh` và thực hiện lại request ban đầu mà không làm gián đoạn trải nghiệm người dùng. Khi đăng xuất, token được thêm vào Redis Blacklist với key `blacklist:{token}` và TTL bằng thời gian sống còn lại — middleware `authenticate` kiểm tra Redis trước khi chấp nhận mọi request, vô hiệu hóa tức thì token bị thu hồi.

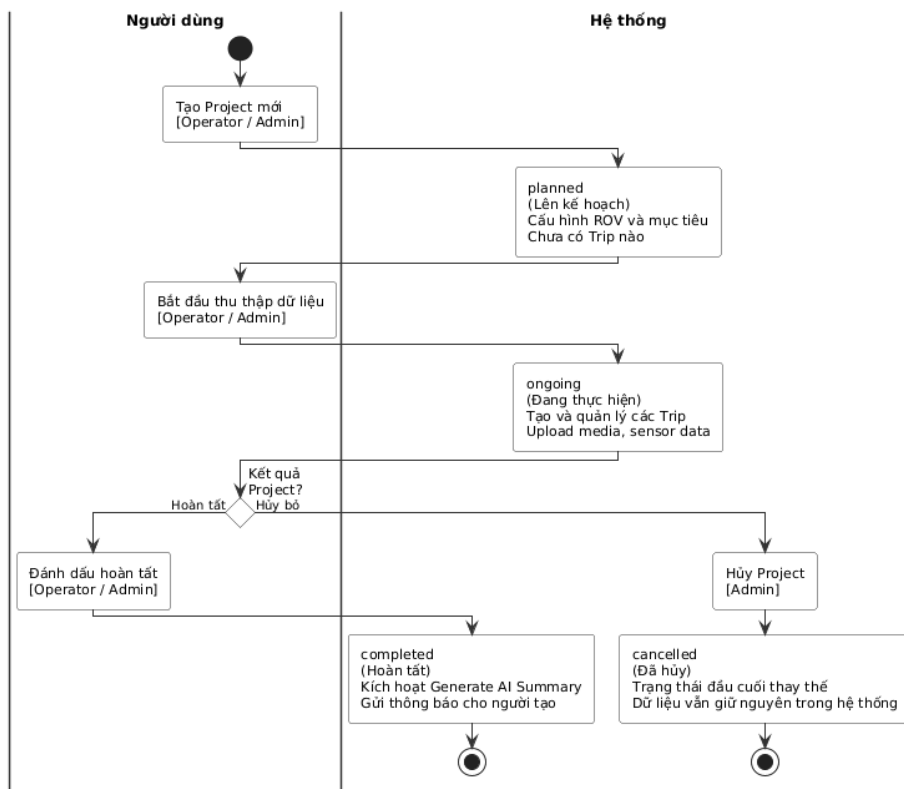
Ngoài đăng nhập email/mật khẩu, hệ thống hỗ trợ **Google OAuth 2.0** thông qua Passport.js: khi người dùng nhấn “Đăng nhập bằng Google”, Passport GoogleStrategy tra cứu email trong MongoDB — nếu tài khoản đã tồn tại thì gán thêm `googleId`; nếu chưa tồn tại thì tạo tài khoản mới với `authProvider = 'google'` và role mặc định `viewer`. Màn hình đăng nhập (Hình 7.9) hỗ trợ đồng thời cả hai phương thức.



Hình 7.9: Màn hình đăng nhập – hỗ trợ cả Email/Password và Google OAuth 2.0

7.3.3 Vòng đời Project và Trip

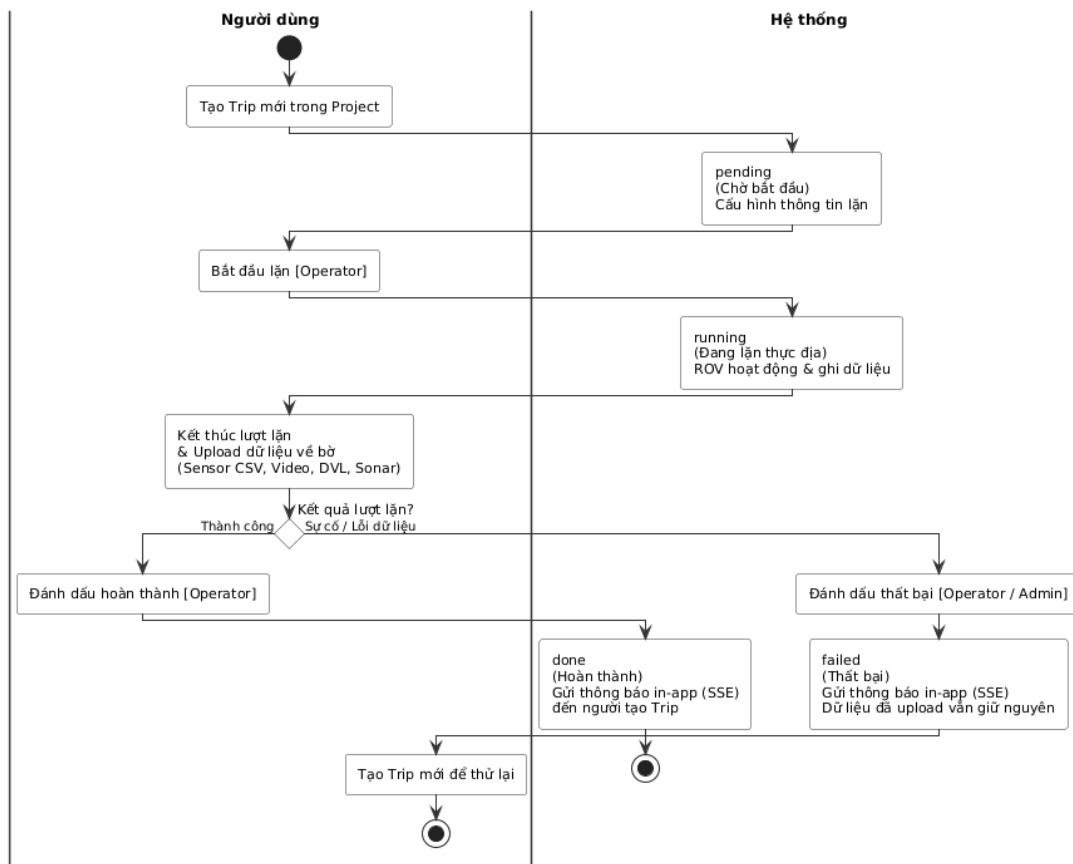
Hệ thống quản lý hai thực thể trung tâm có vòng đời nhiều giai đoạn. **Chuyến khảo sát (Project)** có bốn trạng thái: *planned* → *ongoing* → *completed* / *cancelled*. Hành vi duy nhất ràng buộc theo trạng thái là: chỉ Project ở trạng thái *completed* mới được phép kích hoạt tóm tắt AI (Hình 7.10).



Hình 7.10: Sơ đồ trạng thái vòng đời Chuyến khảo sát (Project)

Lượt lặn (Trip) có bốn trạng thái: *pending* → *running* → *done* / *failed*. Khi Trip chuyển sang *done* hoặc *failed*, hệ thống tự động đẩy thông báo SSE đến người tạo

Trip. Các tính năng phân tích YOLO và đồng bộ video–biểu đồ được Operator kích hoạt thủ công, không ràng buộc theo trạng thái (Hình 7.11).



Hình 7.11: Sơ đồ trạng thái vòng đời Lượt lặn (Trip)

7.3.4 Vòng đời phân tích Media

Mỗi tệp phương tiện (video hoặc ảnh) trong hệ thống có trường `analysisStatus` riêng, độc lập với trạng thái của Lượt lặn. Vòng đời này phản ánh tiến trình của tác vụ phân tích YOLO bất đồng bộ và được tóm tắt trong Bảng 7.5.

Bảng 7.5: Các trạng thái vòng đời phân tích Media (`analysisStatus`)

Trạng thái	Ý nghĩa	Điều kiện chuyển đổi
idle	Chưa phân tích	Trạng thái mặc định sau khi tệp được xác nhận tải lên thành công (<i>status = ready</i>)
pending	Đang chờ trong hàng đợi	Operator nhấn “ <i>Chạy phân tích</i> ” → job vào Bull Queue, labels cũ bị xóa

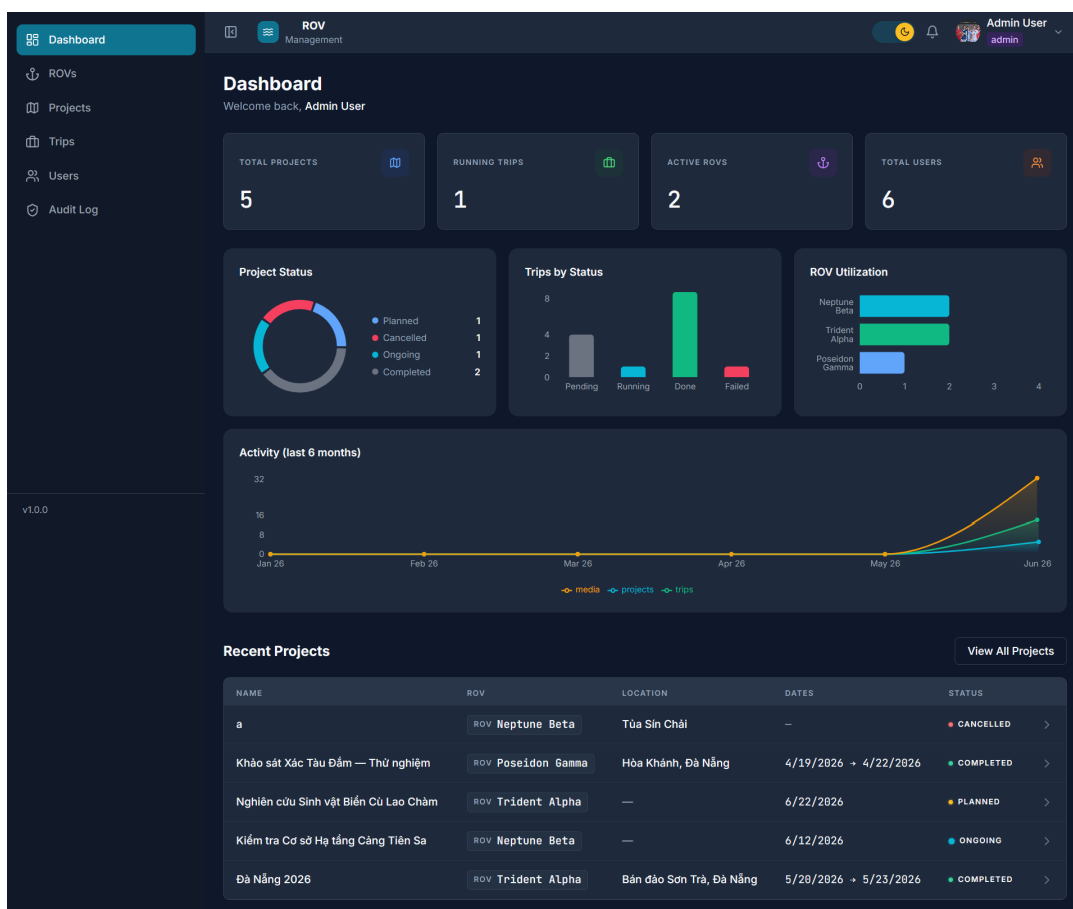
done	Phân tích hoàn tất	Worker nhận kết quả từ YOLO Service, lưu labels vào MongoDB, đẩy SSE
failed	Phân tích thất bại	YOLO Service timeout (25 phút) hoặc lỗi kết nối; Operator có thể kích hoạt lại

Thiết kế tách biệt `analysisStatus` khỏi `status` tải lên của tệp cho phép Operator xem và sử dụng video/ảnh bình thường trong khi quá trình phân tích AI vẫn đang diễn ra ngầm — đảm bảo luồng công việc không bị gián đoạn.

7.4 Hiện thực các chức năng cốt lõi

7.4.1 Dashboard và thống kê tổng quan

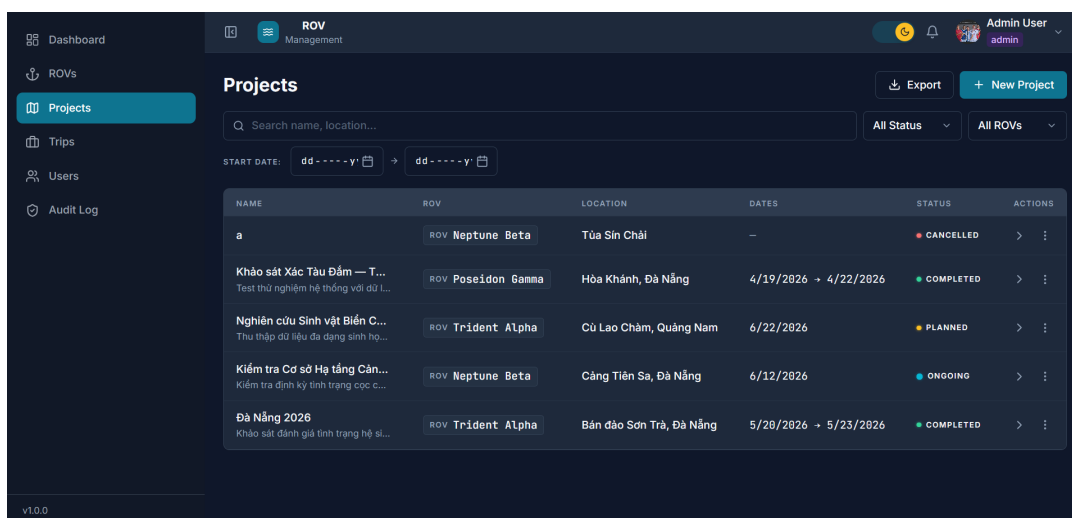
Dashboard (Hình 7.12) hiển thị bức tranh tổng quan hệ thống qua bốn thẻ KPI (tổng số Project, số Trip đang chạy, số ROV hoạt động, tổng người dùng) và ba biểu đồ Recharts, được tải song song qua bảy MongoDB aggregation. Trang có tốc độ tải xuất sắc với điểm Lighthouse Performance 98 (FCP 0,8 s, LCP 0,9 s).



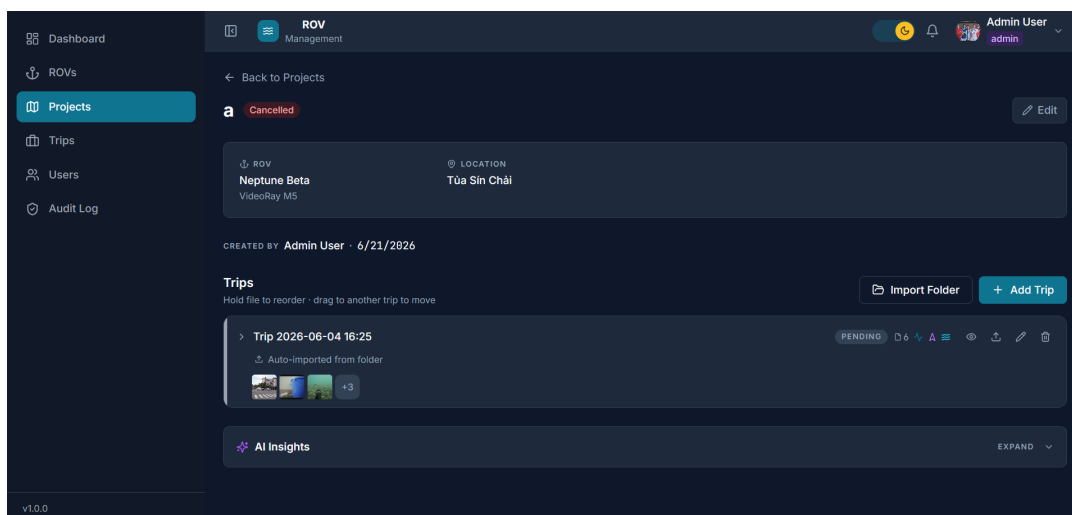
Hình 7.12: Dashboard – KPI cards và biểu đồ thống kê tổng quan (dark mode)

7.4.2 Quản lý Project và Trip

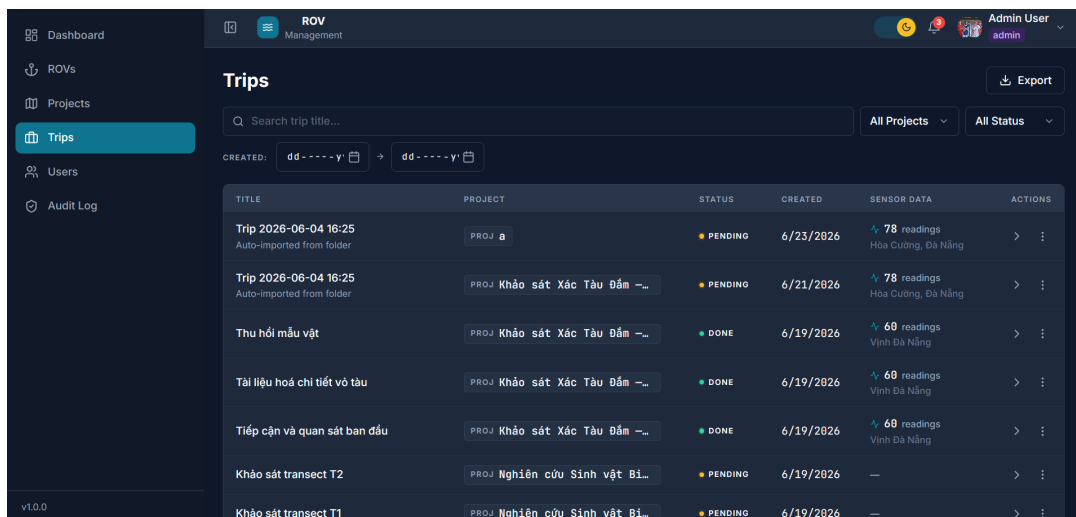
Trang Projects (Hình 7.13) hỗ trợ tìm kiếm theo tên, lọc theo trạng thái và khoảng thời gian, phân trang và xuất CSV/PDF. ProjectDetailPage (Hình 7.14) hiển thị toàn bộ thông tin chuyến đi, danh sách Trip, khu vực DATA FILES liệt kê tên file cụ thể (sensor CSV, DVL JSON, sonar) với nút xóa từng file riêng lẻ, và section AI Summary song ngữ. Trang Trips (Hình 7.15) cho phép lọc toàn bộ lượt lặn theo trạng thái và ngày, độc lập với Project.



Hình 7.13: Trang danh sách Projects – filter, search và export



Hình 7.14: ProjectDetailPage – TripCard mở rộng với DATA FILES và AI Summary

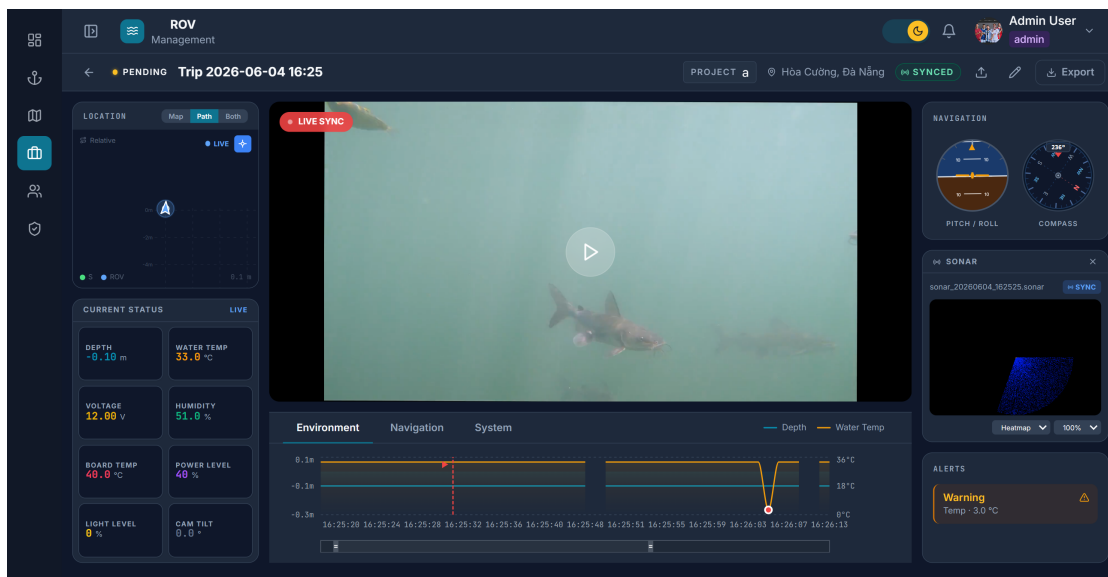


TITLE	PROJECT	STATUS	CREATED	SENSOR DATA	ACTIONS
Trip 2026-06-04 16:25 Auto-imported from folder	PROJ a	PENDING	6/23/2026	78 readings Hòa Cường, Đà Nẵng	> ⋮
Trip 2026-06-04 16:25 Auto-imported from folder	PROJ Khảo sát Xác Tàu Đắm ...	PENDING	6/21/2026	78 readings Hòa Cường, Đà Nẵng	> ⋮
Thu hồi mẫu vật	PROJ Khảo sát Xác Tàu Đắm ...	DONE	6/19/2026	69 readings Vịnh Đà Nẵng	> ⋮
Tài liệu hoá chi tiết vỏ tàu	PROJ Khảo sát Xác Tàu Đắm ...	DONE	6/19/2026	69 readings Vịnh Đà Nẵng	> ⋮
Tiếp cận và quan sát ban đầu	PROJ Khảo sát Xác Tàu Đắm ...	DONE	6/19/2026	69 readings Vịnh Đà Nẵng	> ⋮
Khảo sát transect T2	PROJ Nghiên cứu Sinh vật Bì...	PENDING	6/19/2026	—	> ⋮
Khảo sát transect T1	PROJ Nghiên cứu Sinh vật Bì...	PENDING	6/19/2026	—	> ⋮

Hình 7.15: Trang danh sách Trips – filter theo status và date range

7.4.3 Giao diện Cockpit TripDetailPage

TripDetailPage là tính năng cốt lõi nhất của phân hệ, được thiết kế theo phong cách “cockpit” (Hình 7.16): layout 3 cột không cuộn trang, tối ưu cho màn hình 1080p, hỗ trợ đầy đủ dark mode và responsive từ 375px đến 1440px.



Hình 7.16: TripDetailPage Cockpit – layout 3 cột, đồng bộ video–biểu đồ thời gian thực

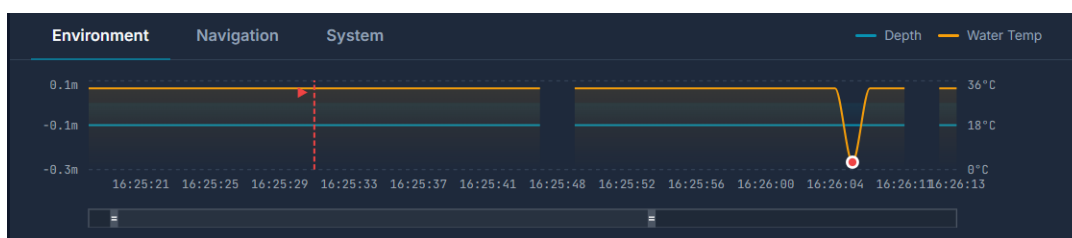
Cột trái hiển thị bản đồ Leaflet với Marker GPS lần lặn, ba thẻ KPI (độ sâu trung bình, nhiệt độ, áp suất từ thống kê cảm biến) và mục DATA FILES liệt kê từng file dữ liệu kèm nút xóa.

Cột giữa là trình phát video nền đen. Phía trên có thanh gradient với tên file và các nút điều khiển AI: “Detect/Hide” bật/tắt bounding box YOLO; “Analyze” mở popover chọn model và ngưỡng confidence; “” chụp frame ảnh; “” ghi clip;

“Evidence N” mở bảng bằng chứng. Khi video có `recordedAt`, badge “LIVE SYNC” xuất hiện và thanh đỏ `ReferenceLine` trên biểu đồ bên dưới di chuyển theo từng giây phát.

Cột phải có đồng hồ `ArtificialHorizon` và `CompassRose` dạng SVG kiểu hàng không, animate theo dữ liệu yaw/pitch/roll từ CSV. Phần Alerts hiển thị card cảnh báo màu hổ phách cho từng điểm bất thường Z-Score.

Dưới cùng là ba tab biểu đồ Recharts: Environment (depth/temp/pressure, Hình 7.17), Navigation (yaw/pitch/roll), System (voltage/battery/humidity), kèm Brush zoom, legend toggle và nút mở rộng.



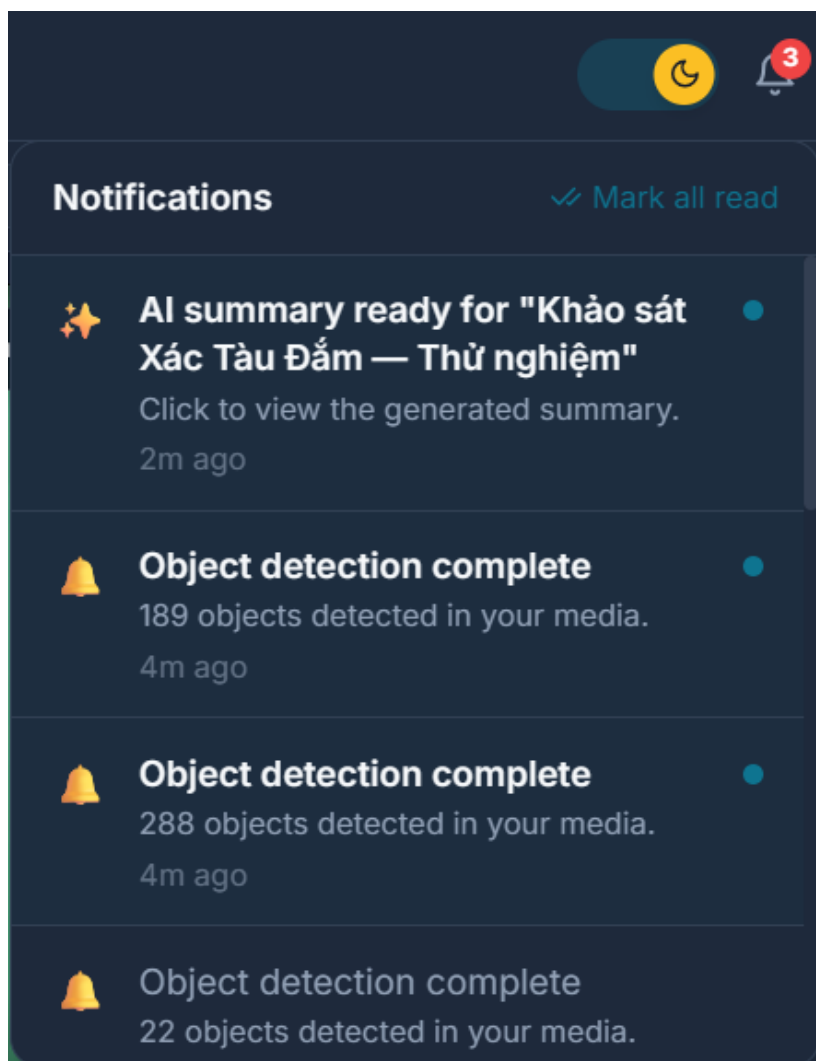
Hình 7.17: Tab biểu đồ Environment – dot đỏ đánh dấu điểm bất thường Z-Score ($|z| > 2,5$)

7.4.4 Upload đa nguồn dữ liệu thực địa

Hệ thống hỗ trợ bốn loại dữ liệu thực địa: tệp CSV cảm biến (depth, temp, pressure, yaw/pitch/roll, voltage, humidity), tệp JSON quỹ đạo DVL, tệp Sonar nhị phân và Media (video/ảnh) qua cơ chế S3 Presigned URL. Điểm nổi bật là mô hình **multi-file append**: thay vì overwrite, hệ thống lưu từng tệp theo tên nguồn (`sourceFile`) và cho phép xóa từng file riêng lẻ. Tên file có định dạng `_YYYYMM-MDD_HHMMSS` được tự động parse để lấy `recordedAt`, cho phép đồng bộ Video-Biểu đồ mà không cần nhập tay. Hệ thống tự động phát hiện chồng lấp thời gian và cảnh báo bằng toast màu vàng (`overlap_trimmed`) khi phần timestamp của tệp mới trùng với data đã có.

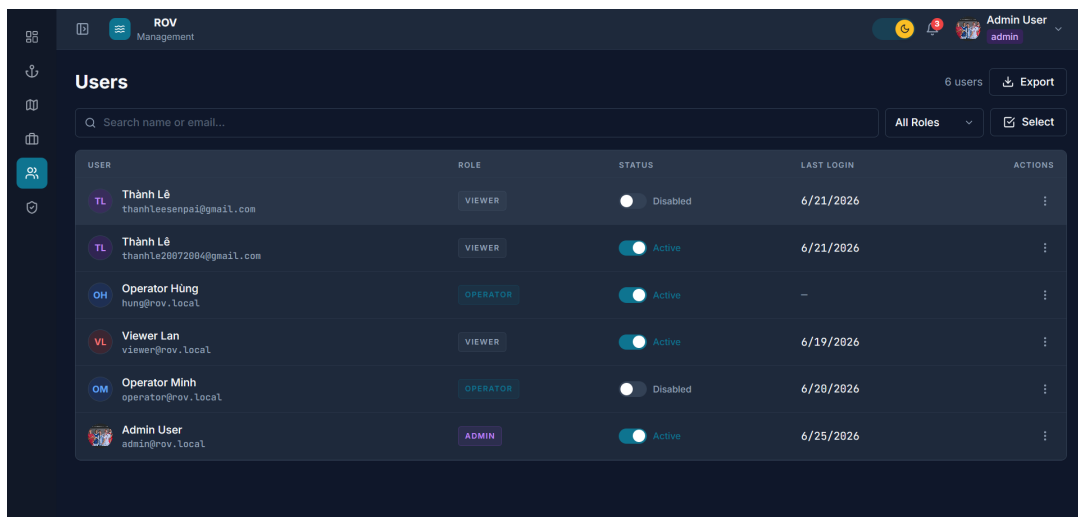
7.4.5 Thông báo realtime và Audit Log

Hệ thống đẩy thông báo realtime qua SSE mỗi khi có sự kiện quan trọng: Trip chuyển sang `done/failed`, AI Summary hoàn tất, YOLO phân tích xong. Bell icon trên navbar hiển thị badge số chưa đọc; click mở dropdown 15 thông báo gần nhất (Hình 7.18).

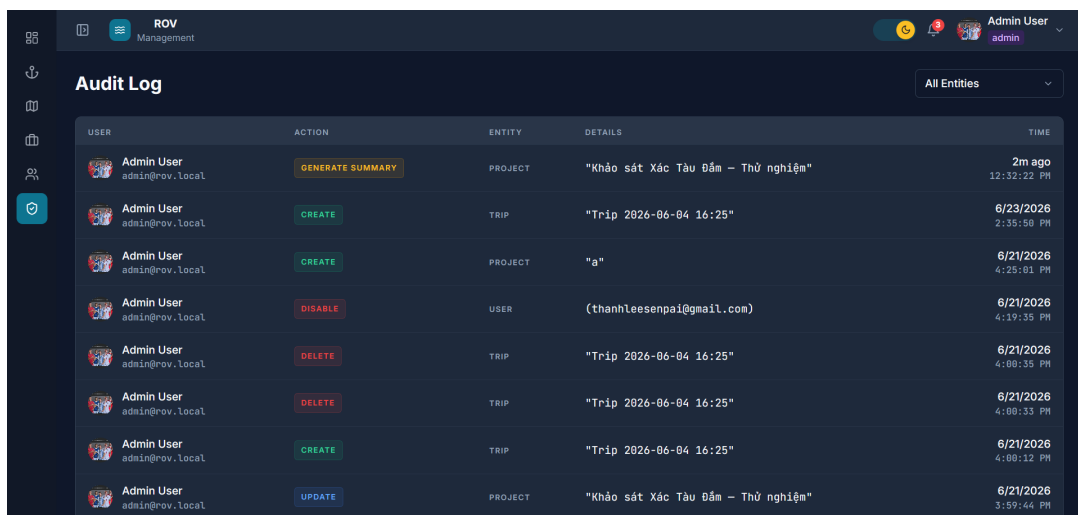


Hình 7.18: Notification bell dropdown – thông báo realtime qua SSE

Trang quản lý người dùng (Hình 7.19, chỉ hiện với Admin) cung cấp tìm kiếm, lọc theo role, đổi role đơn lẻ hoặc bulk, kích hoạt/vô hiệu hóa tài khoản và xuất CSV. Mọi thao tác quan trọng của Admin được ghi vào Audit Log (Hình 7.20) với đầy đủ thông tin: người thực hiện, hành động, đối tượng và thời gian.



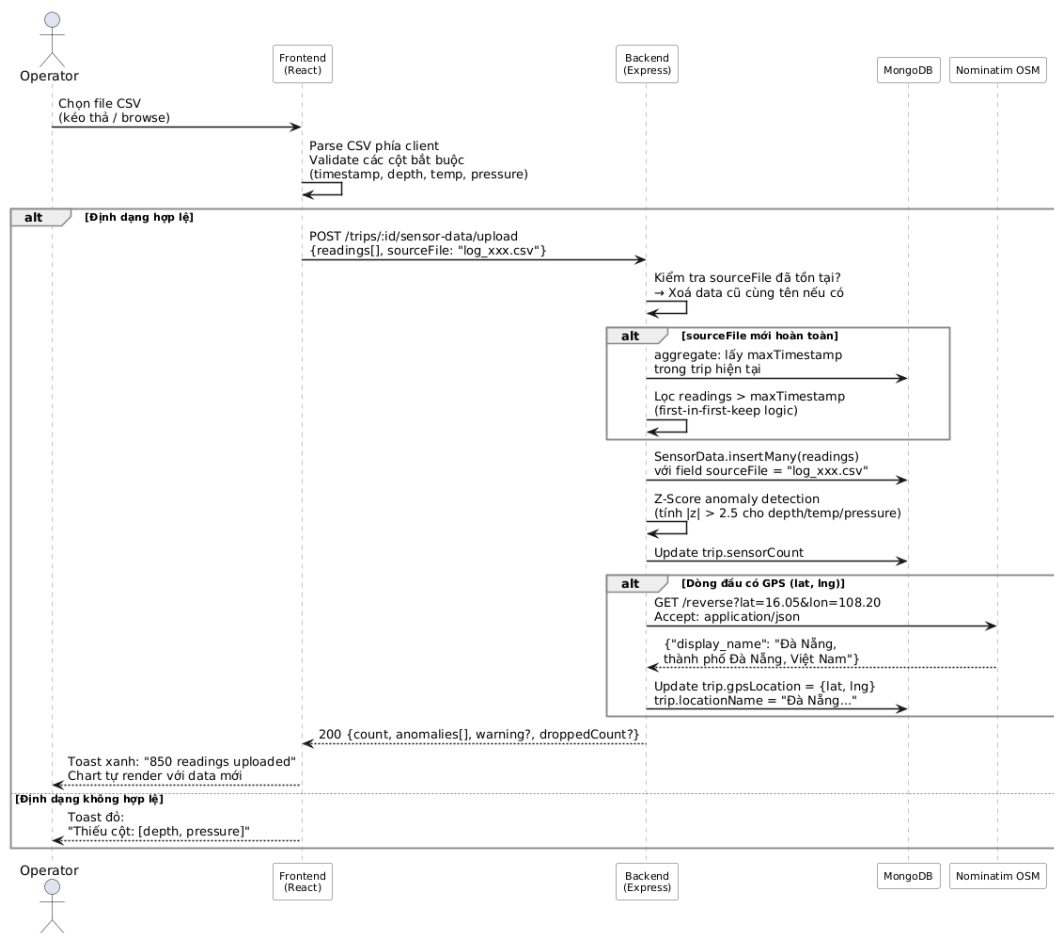
Hình 7.19: Trang quản lý người dùng (Admin) – bulk operations và role management



Hình 7.20: Audit Log – nhật ký hoạt động admin theo timeline

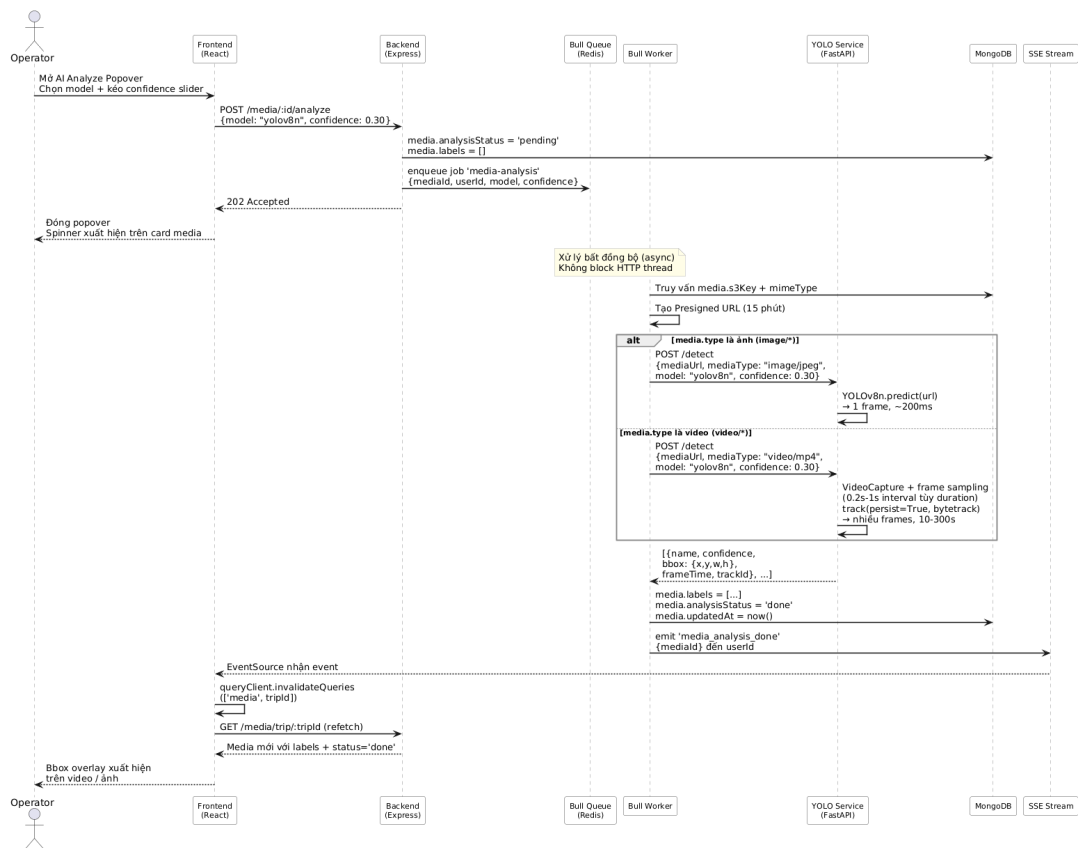
7.4.6 Sơ đồ tuần tự các luồng nghiệp vụ chính

Hình 7.21 mô tả luồng tải lên dữ liệu cảm biến CSV gồm sáu bước: Frontend parse và validate file phía client; gửi mảng readings kèm `sourceFile` lên Backend; kiểm tra và xử lý overlap; lưu batch vào MongoDB; gọi Nominatim reverse geocoding nếu có GPS; trả về kết quả kèm cảnh báo.



Hình 7.21: Sơ đồ tuần tự – Luồng tải lên file CSV cảm biến

Hình 7.22 mô tả luồng phân tích YOLO bất đồng bộ gồm bảy bước: Operator chọn cấu hình AI; Backend nhận request, cập nhật `analysisStatus = 'pending'` và enqueue job; trả về HTTP 202 ngay lập tức; Bull Worker lấy job, gọi YOLO Microservice với presigned URL; YOLO chạy inference và trả labels; Worker lưu kết quả vào MongoDB; đẩy SSE về Frontend để invalidate query và cập nhật giao diện mà không cần tải lại trang.



Hình 7.22: Sơ đồ tuần tự – Luồng phân tích YOLO bất đồng bộ qua Bull Queue

7.5 Giải pháp đồng bộ hóa Video và Dữ liệu cảm biến

7.5.1 Bài toán đặt ra

Một trong những khó khăn lớn nhất của đội ngũ vận hành ROV là phải đối chiếu thủ công giữa đoạn video ghi hình và tệp nhật ký thông số cảm biến (CSV). Camera ghi hình và bo mạch cảm biến thường hoạt động độc lập với hệ quy chiếu thời gian bị lệch nhau. Bài toán đặt ra là: khi video đang phát, thanh đánh dấu trên biểu đồ cảm biến phải lập tức nhảy đến đúng thời điểm đo đạc tương ứng với khung hình đang phát.

7.5.2 Giải pháp hiện thực

Giải pháp sử dụng thuật toán **Ánh xạ thời gian dựa trên độ lệch gốc (Time Offset Mapping)** thực hiện qua bốn bước:

- **Bước 1 – Chuẩn hóa đầu vào:** Video tải lên được gán trường `recordedAt` (thời điểm bắt đầu ghi hình theo ISO 8601). Nếu tên file có định dạng `_YYYYMM-MDD_HHMMSS`, hệ thống tự động parse mà không cần nhập tay. Tệp CSV bắt buộc có cột `timestamp` mỗi dòng.
- **Bước 2 – Tính toán timestamp đồng bộ:** Tại Frontend, sự kiện `timeupdate` của phần tử HTML5 `<video>` được lắng nghe liên tục khi video phát. Giá trị

`currentTime` (giây) được chuyển thành timestamp tuyệt đối:

$$t_{\text{sync}} = t_{\text{recordedAt}} + t_{\text{video}} \times 1000 \quad (\text{mili-giây UTC})$$

- **Bước 3 – Cập nhật ReferenceLine:** Giá trị t_{sync} được truyền vào thành phần `<ReferenceLine x={t_sync}>` của `Recharts` — thanh đồ di chuyển mượt mà trên biểu đồ đồng bộ với video đang phát mà không gây re-render toàn trang nhờ kiến trúc Virtual DOM của React.
- **Bước 4 – Tự động frame Brush:** Khi người dùng chọn video khác trong playlist, vùng zoom Brush tự động nhảy đến khoảng thời gian tương ứng của video đó (tính từ `recordedAt` và `duration`), giúp người dùng quan sát đúng đoạn dữ liệu mà không cần kéo thủ công.

7.5.3 Kết quả đạt được

Giải pháp tạo ra giao diện Cockpit đồng bộ mượt mà theo chiều video → biểu đồ. Khi video phát, badge “LIVE SYNC” xuất hiện tại góc dưới trái và thanh đồ `ReferenceLine` di chuyển liên tục trên biểu đồ cảm biến, cho phép Operator quan sát ngay giá trị nhiệt độ, áp suất, độ sâu tại đúng khoảnh khắc đang diễn ra mà không cần đổi chiều thủ công. Cơ chế này loại bỏ hoàn toàn việc tra cứu giữa video và CSV.

7.6 Kiến trúc xử lý bất đồng bộ tích hợp YOLOv8

7.6.1 Bài toán đặt ra

Nhận diện vật thể trong video ROV (cá, chướng ngại vật, vết nứt đường ống ngầm) đòi hỏi chạy suy luận mô hình học sâu YOLOv8 có thể mất từ vài phút đến hàng giờ tùy độ dài video. Nếu triển khai đồng bộ, vòng lặp sự kiện Node.js sẽ bị chặn và toàn hệ thống web “đóng băng” với người dùng khác trong suốt thời gian đó.

7.6.2 Giải pháp hiện thực

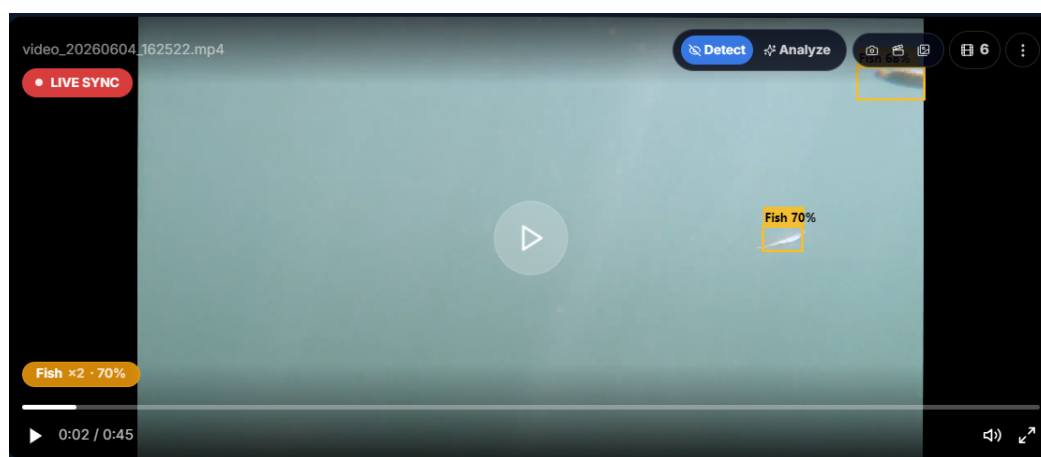
Giải pháp xây dựng kiến trúc ba thành phần phối hợp:

- **Node.js HTTP endpoint:** `POST /media/:id/analyze` nhận cấu hình `{model, confidence}`, đặt `analysisStatus = 'pending'`, enqueue job vào `mediaAnalysisQueue` (lưu trên Redis), và trả về **HTTP 202 ngay lập tức** mà không chờ xử lý.
- **Node.js Bull Worker (xử lý ngầm):** Chạy song song với Event Loop chính. Khi có job, Worker tạo presigned GET URL cho file trên S3, gọi `POST yolo-`

`service:8000/detect` với tham số `model` và `confidence`. Sau khi nhận kết quả, Worker lưu labels vào MongoDB, cập nhật `analysisStatus = 'done'` và đẩy SSE thông báo về Frontend.

- **Python FastAPI YOLO Microservice:** Dịch vụ HTTP độc lập, không kết nối Redis hay MongoDB. Nhận POST `/detect`, tải file từ presigned URL, chạy YOLOv8 với **chiến lược lấy mẫu thích nghi**: < 30 s lấy mẫu 0,2 s/frame; 30 s–3 phút lấy mẫu 0,5 s/frame; > 3 phút lấy mẫu 1,0 s/frame. Mỗi frame áp dụng ByteTrack để gán `trackId` ổn định và Cross-class NMS (IoU = 0,3). Trả về mảng `{name, confidence, bbox, frameTime, trackId}`.

Khi video đang phát, bounding box SVG cập nhật theo frame hiện tại dựa trên `frameTime`; Operator nhấp vào nhãn để lọc và chỉ hiển thị bounding box của lớp đó (Hình 7.23).



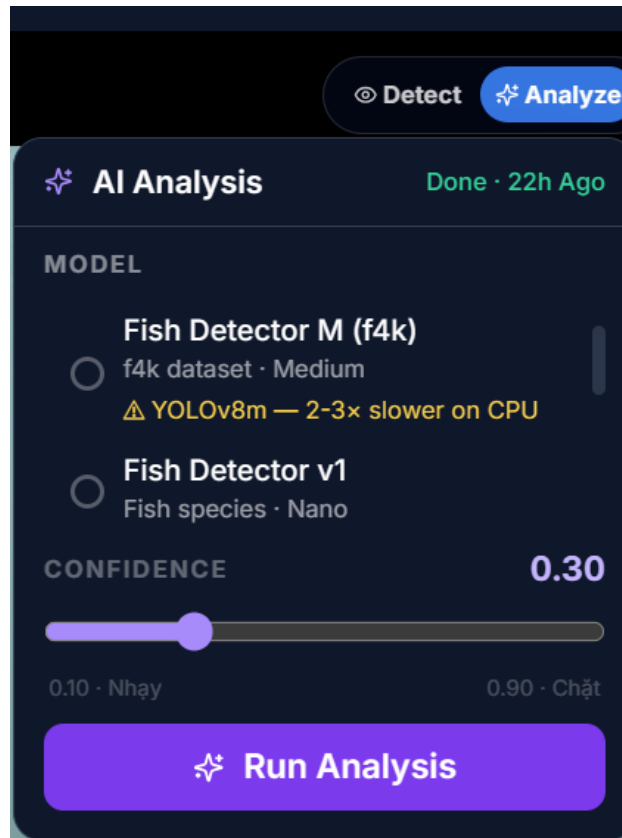
Hình 7.23: Bounding box YOLO overlay trên video ROV – đồng bộ per-frame theo `frameTime`

7.6.3 AI Analyze Popover – Cấu hình phân tích

Trước khi kích hoạt phân tích, Operator có thể tùy chỉnh tham số qua popover nhỏ trực tiếp trên giao diện Cockpit (Hình 7.24). Popover cho phép chọn mô hình AI (ví dụ: YOLOv8n General, Fish Detector v1, v2 hay các mô hình tùy chỉnh khác nếu có) và kéo thanh trượt để đặt ngưỡng `confidence` trong khoảng `[0,1 ; 0,9]`. Model nặng hơn (YOLOv8m) hiển thị cảnh báo màu vàng về thời gian xử lý dài hơn. Sau khi nhấn “Run Analysis”, popover đóng ngay lập tức và hệ thống phản hồi HTTP 202 — Operator có thể tiếp tục làm việc trong khi phân tích diễn ra ngầm.

Việc cho phép điều chỉnh ngưỡng `confidence` giải quyết thực tế của video ROV dưới nước: môi trường nước đục hoặc ánh sáng yếu khiến độ tin cậy nhận diện thấp hơn so với điều kiện lý tưởng. Hạ ngưỡng từ 0,3 xuống 0,15–0,2 giúp phát hiện

được vật thể trong điều kiện này, trong khi tăng lên 0,5+ giúp loại bỏ false positive khi môi trường tốt.



Hình 7.24: AI Analyze Popover – chọn model và điều chỉnh ngưỡng confidence trước khi phân tích

7.6.4 Kết quả đạt được

Kiến trúc này đảm bảo tính khả dụng tuyệt đối cho hệ thống web chính: người dùng có thể đẩy nhiều video vào hàng đợi phân tích ngầm và thoải mái làm việc trên các trang khác. Nếu YOLO Microservice không phản hồi, Worker ghi `analysisStatus = 'failed'` mà không crash hàng đợi; Operator có thể kích hoạt lại bất kỳ lúc nào. Kiến trúc microservice tách rời cho phép thay thế hoặc nâng cấp mô hình AI mà không ảnh hưởng đến Backend Node.js.

7.7 Hệ thống trích xuất bằng chứng (Evidence System)

7.7.1 Bài toán đặt ra

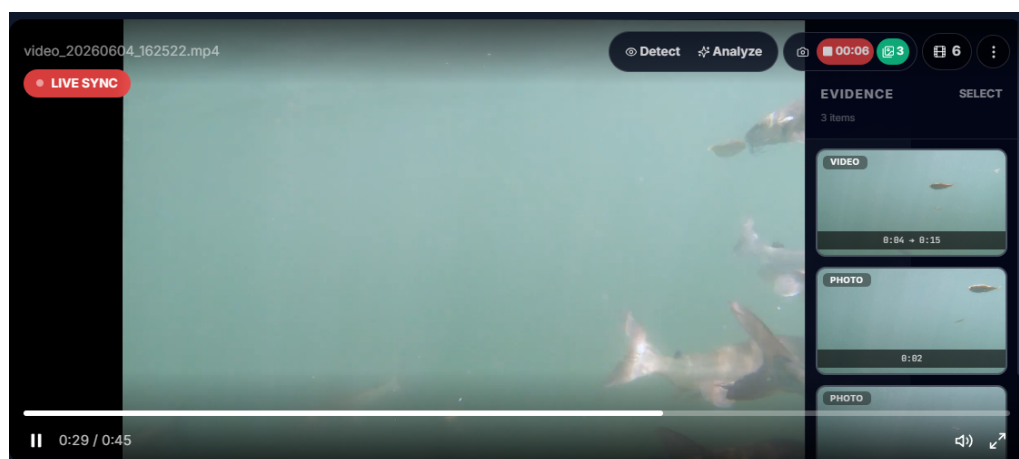
Khi Operator phát hiện vật thể quan trọng trong video, không có công cụ nào cho phép đánh dấu và lưu trữ phát hiện đó ngay lập tức mà không gián đoạn luồng xem. Cách xử lý thủ công là ghi lại dấu thời gian rồi cắt video bằng phần mềm bên ngoài — tốn thời gian và làm gián đoạn quy trình phân tích.

7.7.2 Giải pháp hiện thực

Hệ thống bằng chứng cung cấp hai loại artefact độc lập:

- **Photo (Ảnh khung hình):** Nhấn “” trong khi video phát, Canvas API trích xuất khung hình hiện tại thành PNG. Nếu bounding box đang bật, tọa độ được ghi đè trực tiếp vào ảnh (*canvas burn-in*) trước khi tải lên AWS S3, lưu vào trường `imageS3Key`.
- **Clip (Đoạn video):** Nhấn “” bắt đầu → dừng, hệ thống chỉ ghi `startTime` và `endTime` (giây trong video gốc) — **không cắt hay sao chép file video vật lý**. Lựa chọn này tiết kiệm hoàn toàn băng thông và dung lượng S3.

Mỗi bằng chứng lưu vào collection `snapshots` riêng với `parentMediaId` liên kết đến video gốc, không lẫn vào playlist `media`. Hệ thống cho phép phân tích YOLO riêng từng bằng chứng qua `POST /snapshots/:id/analyze`: với Clip, YOLO Microservice nhận thêm `startTime/endTime` và chỉ xử lý frames trong khoảng đó, giảm đáng kể thời gian so với phân tích toàn bộ video. EvidenceViewer (Hình 7.25) hiển thị dưới dạng overlay inline — bảo toàn ngữ cảnh xem của Operator; với Clip, trình phát bị giới hạn trong $[t_{start}, t_{end}]$ và tự seek về `startTime` khi đến `endTime`.



Hình 7.25: Evidence Panel và EvidenceViewer – PHOTO/CLIP thumbnails với bounding box overlay

7.7.3 Kết quả đạt được

Operator ghi lại phát hiện trong thời gian thực mà không rời giao diện xem video. Chi phí lưu trữ S3 chỉ tăng theo số ảnh PNG, không phát sinh từ clip dài. Vòng lặp kiểm tra nhanh được tạo ra: *xem video* → *phát hiện* → *trích xuất* → *phân tích YOLO* → *xác nhận* trong một giao diện duy nhất.

7.8 Phát hiện bất thường cảm biến tự động (Z-Score)

7.8.1 Bài toán đặt ra

Mỗi lượt lặn ROV tạo ra hàng nghìn dòng dữ liệu cảm biến liên tục. Các điểm bất thường (thiết bị lặn quá sâu, rò rỉ điện, va chạm làm thay đổi áp suất đột ngột) bị chôn vùi trong dữ liệu bình thường và không thể tìm kiếm thủ công. Bài toán đặt ra là tự động phát hiện và gắn cờ bất thường ngay tại Backend mà không cần dữ liệu huấn luyện.

7.8.2 Giải pháp hiện thực

Thuật toán **Z-Score** — phương pháp thống kê phát hiện outlier không cần nhãn (unsupervised) — được triển khai. Với mỗi metric, Z-Score của điểm đo x_i tính:

$$z_i = \left| \frac{x_i - \mu}{\sigma} \right| \quad (7.1)$$

trong đó μ là giá trị trung bình và σ là độ lệch chuẩn của toàn chuỗi. Điểm đo được đánh dấu bất thường khi $z_i > 2,5$. Ngưỡng 2,5 được chọn thay vì giá trị thông thường 3,0 để tăng độ nhạy phù hợp với đặc thù dữ liệu cảm biến môi trường thay đổi chậm — ngay cả spike nhỏ về điện áp hay nhiệt độ cũng có thể báo hiệu sự cố thiết bị. Thuật toán áp dụng song song cho 7 metric: depth, temp, temperature, pressure, voltage, battery_percent và humidity. Kết quả trả về cùng mỗi response sensor data: anomalies: [{index, metric, value, zScore, timestamp}]. So sánh với các lựa chọn thay thế: IQR kém nhạy với spike nhọn ngắn hạn; Isolation Forest đòi hỏi dữ liệu gán nhãn không có sẵn — Z-Score được chọn vì không cần huấn luyện, tính toán đủ nhanh để chạy inline tại API endpoint.

7.8.3 Kết quả đạt được

Thuật toán phát hiện thành công các spike bất thường trong dữ liệu kiểm thử và hiển thị đúng bằng dot đỏ trên biểu đồ AreaChart kèm panel “Anomalies Detected” liệt kê từng điểm cùng giá trị và Z-Score tương ứng. Giải pháp chuyển quy trình từ “xem toàn bộ → phát hiện thủ công” sang “mở trang → các điểm nghi vấn đã đánh dấu sẵn”, giúp Operator tập trung ngay vào những khoảnh khắc quan trọng nhất trong hàng giờ dữ liệu.

7.9 Tóm tắt nhiệm vụ bằng Mô hình ngôn ngữ lớn (Gemini)

7.9.1 Bài toán đặt ra

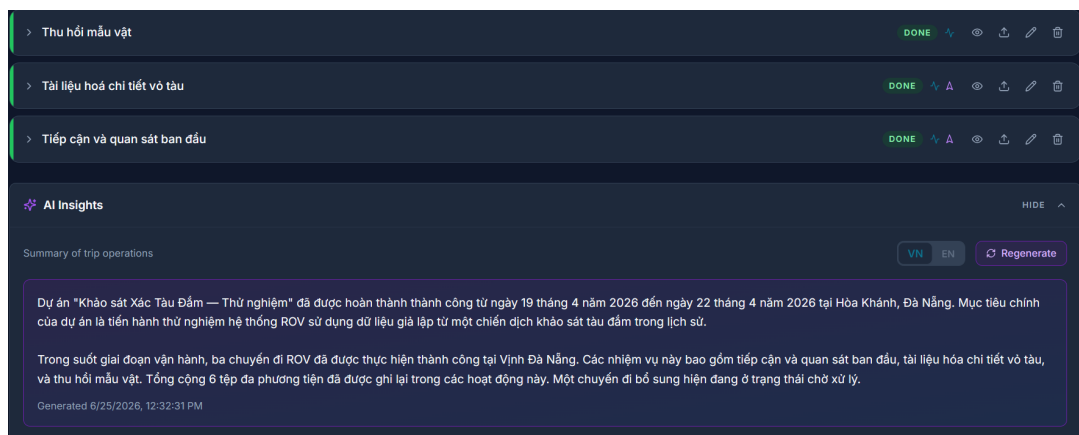
Kết thúc Project với nhiều Trip, ban quản lý thường yêu cầu báo cáo tóm tắt tình hình tổng thể. Việc thủ công tổng hợp số liệu (số Trip, trạng thái từng Trip, tọa độ

địa lý, các điểm bất thường cảm biến) ra văn bản tồn hàng giờ và dễ thiếu sót.

7.9.2 Giải pháp hiện thực

Đồ án ứng dụng **Gemini 2.5 Flash** (Google AI API) để sinh văn bản tự động qua ba bước:

- **Tổng hợp siêu dữ liệu:** Khi Operator nhấn “Generate Summary”, Backend gom toàn bộ dữ liệu từ MongoDB: tên Project, địa điểm (`locationName` từ Nominatim), thời gian, danh sách Trip kèm trạng thái, số lượng media và các điểm bất thường cảm biến ($Z\text{-Score } |z| > 2,5$).
- **Thiết kế Prompt:** Dữ liệu thô được đưa vào mẫu prompt tiếng Việt chuẩn mực, yêu cầu Gemini sinh báo cáo chuyên nghiệp cho ban quản đốc, bao gồm tóm tắt tình hình, kết quả khảo sát và khuyến nghị an toàn.
- **Xử lý bất đồng bộ:** Yêu cầu được đưa vào hàng đợi `ai-summary` và trả về HTTP 202 tức thì. Bull Worker gọi Gemini API, nhận phản hồi văn bản song ngữ và lưu vào `project.aiSummary.vi` và `project.aiSummary.en`. Khi hoàn tất, SSE đẩy thông báo về Frontend và summary xuất hiện tức thì trên giao diện (Hình 7.26).



Hình 7.26: AI Summary song ngữ – được tổng hợp bởi Gemini 2.5 Flash sau khi Project hoàn tất

7.9.3 Kết quả đạt được

Giải pháp biến hàng nghìn dòng dữ liệu thô thành văn bản báo cáo liền mạch chỉ trong 5–15 giây. Kết quả được lưu song ngữ Việt/Anh, phù hợp với yêu cầu báo cáo đa ngôn ngữ trong môi trường vận hành quốc tế. Tính năng khẳng định sự chuyển dịch từ số hóa dữ liệu (*Digitization*) sang thấu hiểu dữ liệu (*Data Insights*) — giá trị gia tăng lớn cho quy trình nghiệp vụ thực địa.

7.10 Kiểm thử hệ thống

7.10.1 Kiểm thử chức năng (Functional Test)

Script `functional-test.js` tự động hóa 37 ca kiểm thử bao phủ sáu nhóm: xác thực (7 ca), phân quyền RBAC (6 ca), validation đầu vào (5 ca), CRUD Project và Trip (11 ca), phân trang (6 ca) và kiểm thử hồi quy (2 ca).

```

PS D:\Code\rov-management\backend> node src/scripts/functional-test.js
Functional Test - ROV Management API
=====
-
  AUTH
  ✓ Login với đúng credentials → 200
  ✓ Response có accessToken
  ✓ Login sai password → 401
  ✓ Login email không tồn tại → 401

  ✓ Login thiếu password → 400/401
  ✓ GET /auth/me với token hợp lệ → 200
  ✓ /auth/me trả về đúng email
  ✓ GET /auth/me không có token → 401
  ✓ GET /auth/me token không hợp lệ → 401

  RBAC - Phân quyền
  ✓ Viewer login OK
  ✓ Viewer POST /projects → 403
  ✓ Viewer DELETE /trips/:id → 403
  ✓ Viewer GET /projects → 200
  ✓ Viewer batch upload → 403
  ✓ Không có token GET /projects → 401

  VALIDATION - Kiểm tra đầu vào
  ✓ POST /projects thiếu name → 400/422
  ✓ POST /projects với rovid sai format → 400/422/500
  ✓ POST /projects/:id/trips với id không tồn tại → 404
  ✓ Upload sensor-data readings không phải array → 400
  ✓ Upload sensor-data array rỗng → 400

  CRUD - Projects & Trips
  ✓ GET /rovs → 200
  ✓ POST /projects → 201
  ✓ GET /projects/:id → 200
  ✓ GET /projects/:id trả đúng name
  ✓ PATCH /projects/:id → 200
  ✓ PATCH cập nhật status đúng
  ✓ POST /projects/:id/trips → 201
  ✓ GET /trips/:id → 200
  ✓ DELETE /trips/:id → 200
  ✓ DELETE /projects/:id → 200
  ✓ GET project đã xóa → 404

  PAGINATION & FILTER
  ✓ GET /projects?limit=2 → 200
  ✓ Trả về đúng cấu trúc page
  ✓ Số item ≤ limit=2
  ✓ GET /projects?status=completed → 200
  ✓ Tất cả kết quả có status=completed
  ✓ GET /projects?page=999 → 200 với data rỗng

=====
  Kết quả: 37/37 passed (100%)
  ✓ TẤT CẢ PASS - API hoạt động đúng
PS D:\Code\rov-management\backend>

```

Hình 7.27: Kết quả kiểm thử chức năng – 37/37 ca đạt (100%)

Kết quả Hình 7.27 cho thấy 37/37 ca PASSED (100%). Trong quá trình viết

test, phát hiện và sửa một lỗi thực tế: endpoint `POST /projects/:id/trips` không kiểm tra project tồn tại, trả về HTTP 201 thay vì HTTP 404 khi `projectId` không hợp lệ.

7.10.2 Kiểm thử tải (Load Test)

Script `load-test.js` bắn 20 request đồng thời $\times 3$ vòng = 60 request/endpoint. Kết quả tổng hợp trong Bảng 7.6.

Bảng 7.6: Kết quả kiểm thử tải – 20 concurrent $\times$ 3 rounds (60 requests/endpoint)

Endpoint	Avg	P50	P95	Errors
POST /auth/login	1312 ms	8 ms	4101 ms	41/60
GET /auth/me	132 ms	82 ms	492 ms	0/60
GET /projects	474 ms	456 ms	934 ms	0/60
GET /trips	458 ms	252 ms	988 ms	0/60
GET /rovs	205 ms	168 ms	540 ms	0/60
GET /stats/overview	825 ms	887 ms	1830 ms	0/60
GET /trips/:id/sensor-data	327 ms	261 ms	523 ms	0/60
GET /trips/:id/dvl	737 ms	810 ms	885 ms	0/60
GET /media/trip/:id	377 ms	241 ms	759 ms	0/60
GET /notifications	525 ms	645 ms	855 ms	0/60

41/60 lỗi tại `POST /auth/login` là hoạt động đúng thiết kế: Redis rate limiter chặn quá 20 request/15 phút để phòng chống brute-force, các request vượt ngưỡng nhận HTTP 429 Too Many Requests. Endpoint `GET /stats/overview` chậm nhất (avg 825 ms) do thực hiện 7 aggregation song song trên MongoDB Atlas free tier (độ trễ mạng 50 ms Singapore–Việt Nam); trong môi trường production với VPS gần database, thời gian này giảm đáng kể. Tất cả endpoint đọc dữ liệu hoạt động ổn định ở tải 20 concurrent mà không có lỗi.

7.10.3 Kiểm thử hiệu năng Lighthouse

Công cụ Lighthouse (Chrome DevTools, chế độ Desktop) được chạy trên ba trang đại diện (Bảng 7.7).

Bảng 7.7: Kết quả kiểm thử Lighthouse – Desktop mode

Trang	Perf.	A11y	BP	SEO	FCP	LCP
Dashboard	98	94	96	100	0,8 s	0,9 s
ProjectsPage	95	81	100	100	0,7 s	1,4 s
TripDetailPage	88	89	100	100	0,7 s	1,9 s

- **Dashboard** đạt Performance 98 — xuất sắc. FCP 0,8 s và LCP 0,9 s cho thấy trang tải rất nhanh nhờ kiến trúc React SPA với Vite build tối ưu. TBT chỉ 90 ms chứng tỏ main thread ít bị chặn.

- **ProjectsPage** đạt Performance 95. Accessibility 81 do một số button thiếu accessible name và input date không có label rõ ràng — đây là điểm cần cải thiện trong phiên bản tiếp theo.
- **TripDetailPage** đạt Performance 88 do LCP 1,9 s: ảnh/video phải tải từ AWS S3 (Singapore) mất thêm 40–50 ms RTT so với localhost. Đây là giới hạn về hạ tầng, không phải vấn đề tối ưu code. TBT 100 ms vẫn trong ngưỡng tốt.
- **Best Practices đạt 100** trên hai trang kỹ thuật phản ánh code không có lỗi console, không dùng API deprecated và không có third-party cookies không kiểm soát.
- **SEO đạt 100** trên tất cả trang nhờ thẻ `<title>`, `<meta description>` hợp lệ và tất cả link có thể crawl được.

CHƯƠNG 8. KẾT QUẢ THỰC NGHIỆM

Tự viết

CHƯƠNG 9. KẾT LUẬN

9.1 Kết luận

Sinh viên so sánh kết quả nghiên cứu hoặc sản phẩm của mình với các nghiên cứu hoặc sản phẩm tương tự.

Sinh viên phân tích trong suốt quá trình thực hiện ĐATN, mình đã làm được gì, chưa làm được gì, các đóng góp nổi bật là gì, và tổng hợp những bài học kinh nghiệm rút ra nếu có.

9.2 Hướng phát triển

Trong phần này, sinh viên trình bày định hướng công việc trong tương lai để hoàn thiện sản phẩm hoặc nghiên cứu của mình.

Trước tiên, sinh viên trình bày các công việc cần thiết để hoàn thiện các chức năng/nhiệm vụ đã làm. Sau đó sinh viên phân tích các hướng đi mới cho phép cải thiện và nâng cấp các chức năng/nhiệm vụ đã làm.

MỘT SỐ LƯU Ý VỀ TÀI LIỆU THAM KHẢO

Lưu ý: Sinh viên không được đưa bài giảng/slide, các trang Wikipedia, hoặc các trang web thông thường làm tài liệu tham khảo.

Một trang web được phép dùng làm tài liệu tham khảo **chỉ khi** nó là công bố chính thống của cá nhân hoặc tổ chức nào đó. Ví dụ, trang web đặc tả ngôn ngữ XML của tổ chức W3C <https://www.w3.org/TR/2008/REC-xml-20081126/> là TLTK hợp lệ.

Có năm loại tài liệu tham khảo mà sinh viên phải tuân thủ đúng quy định về cách thức liệt kê thông tin như sau. Lưu ý: các phần văn bản trong cặp dấu < > dưới đây chỉ là hướng dẫn khai báo cho từng loại tài liệu tham khảo; sinh viên cần xóa các phần văn bản này trong ĐATN của mình.

<**Bài báo đăng trên tạp chí khoa học:** Tên tác giả, tên bài báo, tên tạp chí, volume, từ trang đến trang (nếu có), nhà xuất bản, năm xuất bản >

hovy1993automated E. H. Hovy, "Automated discourse generation using discourse structure relations," *Artificial intelligence*, vol. 63, no. 1-2, pp. 341–385, 1993

<**Sách:** Tên tác giả, tên sách, volume (nếu có), lần tái bản (nếu có), nhà xuất bản, năm xuất bản>

peterson2007computer L. L. Peterson and B. S. Davie, *Computer networks: a systems approach*. Elsevier, 2007.

NguyenThucHai N. T. Hải, *Mạng máy tính và các hệ thống mở*. Nhà xuất bản giáo dục, 1999.

<**Tập san Báo cáo Hội nghị Khoa học:** Tên tác giả, tên báo cáo, tên hội nghị, ngày (nếu có), địa điểm hội nghị, năm xuất bản>

poesio2001discourse M. Poesio and B. Di Eugenio, "Discourse structure and anaphoric accessibility," in *ESSLLI workshop on information structure, discourse structure and discourse semantics*, Copenhagen, Denmark, 2001, pp. 129–143.

<**Đồ án tốt nghiệp, Luận văn Thạc sĩ, Tiến sĩ:** Tên tác giả, tên đồ án/luận văn, loại đồ án/luận văn, tên trường, địa điểm, năm xuất bản>

knott1996data A. Knott, "A data-driven methodology for motivating a set of coherence relations," Ph.D. dissertation, The University of Edinburgh, UK, 1996.

<**Tài liệu tham khảo từ Internet:** Tên tác giả (nếu có), tựa đề, cơ quan (nếu

có), địa chỉ trang web, thời gian lần cuối truy cập trang web>

BernersTim T. Berners-Lee, *Hypertext transfer protocol (HTTP)*. [Online]. Available: `ftp://info.cern.ch/pub/www/doc/http-spec.txt.Z` (visited on 09/30/2010).

LectureA Princeton University, *Wordnet*. [Online]. Available: `http://www.cogsci.princeton.edu/~wn/index.shtml` (visited on 09/30/2010).

PHỤ LỤC

A. HƯỚNG DẪN VIẾT ĐỒ ÁN TỐT NGHIỆP

Quy định chung

Dưới đây là một số quy định và hướng dẫn viết đồ án tốt nghiệp mà bắt buộc sinh viên phải đọc kỹ và tuân thủ nghiêm ngặt.

Sinh viên cần đảm bảo tính thống nhất toàn báo cáo (font chữ, căn dòng hai bên, hình ảnh, bảng, margin trang, đánh số trang, v.v.). Để làm được như vậy, sinh viên chỉ cần sử dụng các định dạng theo đúng template ĐATN này. Khi paste nội dung văn bản từ tài liệu khác của mình, sinh viên cần chọn kiểu Copy là “Text Only” để định dạng văn bản của template không bị phá vỡ/vi phạm.

Tuyệt đối cấm sinh viên đạo văn. Sinh viên cần ghi rõ nguồn cho tất cả những gì không tự mình viết/vẽ lên, bao gồm các câu trích dẫn, các hình ảnh, bảng biểu, v.v. Khi bị phát hiện, sinh viên sẽ không được phép bảo vệ ĐATN.

Tất cả các hình vẽ, bảng biểu, công thức, và tài liệu tham khảo trong ĐATN nhất thiết phải được SV giải thích và tham chiếu tới ít nhất một lần. Không chấp nhận các trường hợp sinh viên đưa ra hình ảnh, bảng biểu tùy hứng và không có lời mô tả/giải thích nào.

Sinh viên tuyệt đối không trình bày ĐATN theo kiểu viết ý hoặc gạch đầu dòng. ĐATN không phải là một slide thuyết trình; khi người đọc không hiểu sẽ không có ai giải thích hộ. Sinh viên cần viết thành các đoạn văn và phân tích, diễn giải đầy đủ, rõ ràng. Câu văn cần đúng ngữ pháp, đầy đủ chủ ngữ, vị ngữ và các thành phần câu. Khi thực sự cần liệt kê, sinh viên nên liệt kê theo phong cách khoa học với các ký tự La Mã. Ví dụ, nhiều sinh viên luôn cảm thấy hối hận vì (i) chưa cố gắng hết mình, (ii) chưa sắp xếp thời gian học/chơi một cách hợp lý, (iii) chưa tìm được người yêu để chia sẻ quãng đời sinh viên vất vả, và (iv) viết ĐATN một cách cẩu thả.

Trong một số trường hợp nhất thiết phải dùng các bullet để liệt kê, sinh viên cần thống nhất Style cho toàn bộ các bullet các cấp mà mình sử dụng đến trong báo cáo. Nếu dùng bullet cấp 1 là hình tròn đen, toàn bộ báo cáo cần thống nhất cách dùng như vậy; ví dụ như sau:

- Đây là mục 1 – Thực sự không còn cách nào khác tôi mới dùng đến việc bullet trong báo cáo.
- Đây là mục 2 – Nghĩ lại thì tôi có thể không cần dùng bullet cũng được. Nên tôi sẽ xóa bullet và tổ chức lại hai mục này trong báo cáo của mình cho khoa học hơn. Tôi muốn thầy cô và người đọc cảm nhận được tâm huyết của tôi

trong từng trang báo cáo ĐATN.

A.1 Ngành học

Sinh viên lưu ý viết đúng ngành/chuyên ngành trên bìa và trên gáy theo đúng quy định của Trường. Ngành học hay chuyên ngành học phụ thuộc vào ngành học mà sinh viên đăng ký. Sinh viên có thể đăng nhập trên trang quản lý học tập của mình để xem lại chính xác ngành học của mình.

Một số ví dụ sinh viên có thể tham khảo dưới đây, trong trường hợp có chuyên ngành thì sinh viên không cần ghi chuyên ngành:

Đối với kỹ sư chính quy: Từ K61 trở về trước: Ngành Kỹ thuật phần mềm Từ K62 trở về sau: Ngành Khoa học máy tính Đối với cử nhân: Ngành Công nghệ thông tin Đối với chương trình EliteTech: Chương trình Việt Nhật/KSTN: Ngành Công nghệ thông tin Chương trình ICT Global: Ngành Information Technology Chương trình DS&AI: Ngành Khoa học dữ liệu

A.2 Đánh dấu (bullet) và đánh số (numering)

Việc sử dụng danh sách trong LaTeX khá đơn giản và không yêu cầu sinh viên phải thêm bất kỳ gói bổ sung nào. LaTeX cung cấp hai môi trường liệt kê đó là:

- Đánh dấu (bullet) là kiểu liệt kê không có thứ tự. Để sử dụng kiểu liệt kê đánh dấu, chúng ta khai báo như sau

```
\begin{itemize}
\item Nội dung thứ nhất được viết ở đây.
\item Nội dung thứ hai được viết ở đây.
\item ...
\end{itemize}
```

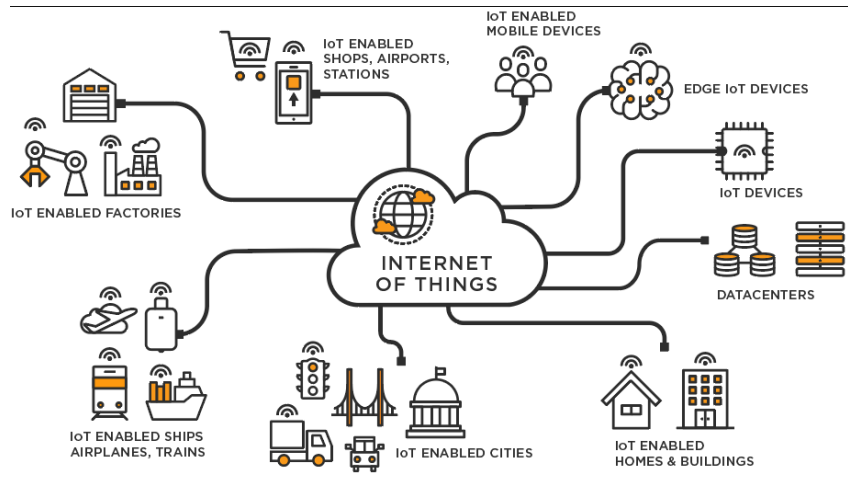
- Đánh số (numering) là kiểu liệt kê có thứ tự. Để sử dụng kiểu liệt kê đánh số, chúng ta khai báo như sau

```
\begin{enumerate}
\item Nội dung thứ nhất được viết ở đây.
\item Nội dung thứ hai được viết ở đây.
\item ...
\end{enumerate}
```

Chú ý các nội dung trình bày trong cả hai môi trường liệt kê theo sau lệnh `\item`. Ngoài ra LaTeX còn cung cấp một số kiểu liệt kê khác, sinh viên có thể tham khảo tại <https://www.overleaf.com/learn/latex/Lists>

Col1	Col2	Col2	Col3
1	6	87837	787
2	7	78	5415
3	545	778	7507
4	545	18744	7560
5	88	788	6344

Bảng A.1: Table to test captions and labels.



Hình A.1: Internet vạn vật

A.3 Cách thêm bảng

Bảng A.1 là ví dụ về cách tạo bảng. Tất cả các bảng biểu phải được đề cập đến trong phần nội dung và phải được phân tích và bình luận. Chú ý: Tạo bảng trong Latex khá phức tạp và mất thời gian, vì vậy sinh viên có thể sử dụng các công cụ hỗ trợ tạo bảng (Ví dụ: <https://www.tablesgenerator.com/>). Sinh viên có thể tìm hiểu sâu hơn về cách chèn ảnh trong Latex tại link <https://www.overleaf.com/learn/latex/Tables>.

A.4 Chèn hình ảnh

Hình A.1 là ví dụ về cách chèn ảnh. Lưu ý chú thích của hình vẽ được đặt ngay dưới hình vẽ. Sinh viên có thể tìm hiểu sâu hơn về cách chèn ảnh trong Latex tại https://www.overleaf.com/learn/latex/Inserting_Images.

Chú ý, tất cả các hình vẽ phải được đề cập đến trong phần nội dung và phải được phân tích và bình luận.

A.5 Tài liệu tham khảo

Cách liệt kê

Áp dụng cách liệt kê theo quy định của IEEE. Ví dụ của việc trích dẫn như sau

scott2013sdn. Cụ thể, sinh viên sử dụng lệnh `\cite{}` như sau **ashton2009internet**. Chỉ những tài liệu được trích dẫn thì mới xuất hiện trong phần Tài liệu tham khảo. Tài liệu tham khảo cần có nguồn gốc rõ ràng và phải từ nguồn đáng tin cậy. Hạn chế trích dẫn tài liệu tham khảo từ các website, từ wikipedia.

Các loại tài liệu tham khảo

Các nguồn tài liệu tham khảo chính là sách, bài báo trong các tạp chí, bài báo trong các hội nghị khoa học và các tài liệu tham khảo khác trên internet.

A.6 Cách viết phương trình và công thức toán học

Các gói `amsmath`, `amssymb`, `amsfonts` hỗ trợ viết phương trình/công thức toán học đã được bổ sung sẵn ở phần đầu của file `main.tex`. Một ví dụ về tạo phương trình (A.1) như sau

$$F(x) = \int_b^a \frac{1}{3}x^3 \quad (\text{A.1})$$

Phương trình A.1 là ví dụ về phương trình tích phân. Một phương trình khác không được đánh số thứ tự (gán nhãn)

$$x[t_n] = \frac{1}{\sqrt{N}} \sum_{k=0}^{N-1} X[f_k] e^{j2\pi nk/N}$$

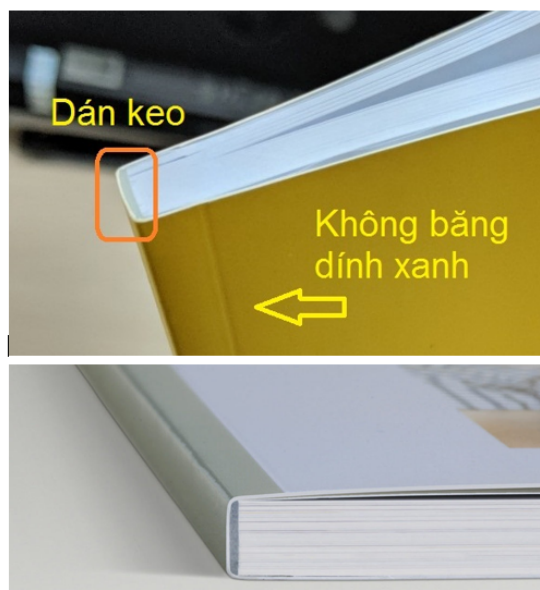
Phương trình này thể hiện phép biến đổi Fourier rời rạc ngược (IDFT).

A.7 Qui cách đóng quyển

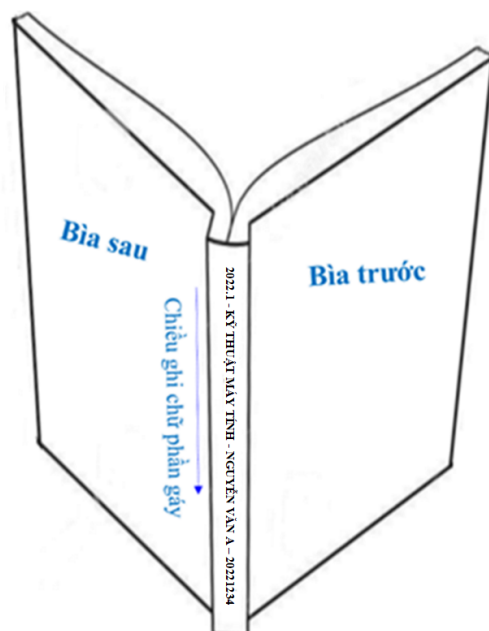
Phần bìa trước chế bản theo qui định; bìa trước và bìa sau là giấy liền khổ. Sử dụng keo nhiệt để dán gáy khi đóng quyển thay vì sử dụng băng dính và dập ghim như mô tả ở Hình A.3 Phần gáy ĐATN cần ghi các thông tin tóm tắt sau: Kỳ làm ĐATN - Ngành đào tạo - Họ và tên sinh viên - Mã số sinh viên. Ví dụ:

2022.1 - KỸ THUẬT MÁY TÍNH - NGUYỄN VĂN A - 20221234

Qui cách ghi chữ phần gáy như hình dưới đây:



Hình A.2: Qui cách đóng quyển đồ án



Hình A.3: Qui cách đóng quyển đồ án

B. ĐẶC TẢ USE CASE

Phụ lục này trình bày đặc tả chi tiết bốn Use Case cốt lõi của phân hệ Quản lý Thông tin Nhiệm vụ ROV.

B.1 UC-01: Tải lên và đồng bộ dữ liệu cảm biến

Bảng B.1: Đặc tả Use Case UC-01: Tải lên và đồng bộ dữ liệu cảm biến

Mã Use Case	UC-01
Tên	Tải lên và đồng bộ dữ liệu cảm biến (Sensor Data Sync)
Tác nhân	Operator
Tiền điều kiện	Tác nhân đã đăng nhập và đang ở trang chi tiết Lượt lặn (TripDetailPage).
Luồng chính	(1) Tác nhân kéo thả hoặc chọn tệp CSV/DVL/Sonar vào khu vực tải lên. (2) Hệ thống xác thực định dạng tệp (CSV phải chứa hai cột bắt buộc: timestamp và depth; các cột còn lại như temp, pressure, yaw, voltage,... là tùy chọn). (3) Hệ thống bóc tách tọa độ GPS từ dòng đầu tiên, gọi API Nominatim để chuyển thành tên địa danh (<i>Reverse Geocoding</i>). (4) Hệ thống lưu dữ liệu vào MongoDB, tính toán phát hiện bất thường bằng Z-Score ($ z > 2,5$). (5) Giao diện hiển thị biểu đồ môi trường; khi video phát, thanh đánh dấu dọc di chuyển đồng bộ theo dấu thời gian.
Luồng phát sinh	Định dạng tệp không hợp lệ → hệ thống thông báo lỗi chi tiết và yêu cầu tải lại. Tệp trùng phạm vi thời gian → hệ thống cảnh báo số dòng bị bỏ qua (overlap_trimmed).
Hậu điều kiện	Dữ liệu cảm biến được liên kết thành công vào lượt lặn; biểu đồ và bản đồ GPS cập nhật tức thì.

B.2 UC-02: Kích hoạt phân tích nhận diện vật thể bằng YOLOv8

Bảng B.2: Đặc tả Use Case UC-02: Phân tích nhận diện vật thể bằng AI

Mã Use Case	UC-02
Tên	Kích hoạt phân tích nhận diện vật thể (AI Object Detection)
Tác nhân	Operator; YOLO Service (hệ thống bên ngoài)
Tiền điều kiện	Tệp video hoặc ảnh đã được tải lên thành công (trạng thái <i>Ready</i>).
Luồng chính	(1) Tác nhân nhấn nút Cài đặt AI trên thanh công cụ video; cửa sổ thiết lập hiện ra cho phép chọn mô hình (YOLOv8n/tùy chỉnh) và ngưỡng độ tin cậy. (2) Tác nhân nhấn “<i>Chạy phân tích</i>”; hệ thống đưa tác vụ vào hàng đợi Bull Queue và trả về xác nhận tiếp nhận (HTTP 202). (3) Worker nền gọi Python Microservice, truyền URL presigned của tệp phương tiện, mô hình và ngưỡng. (4) YOLO Service xử lý từng khung hình (video: lấy mẫu theo tần suất thích nghi), trả về danh sách nhãn kèm tọa độ bounding box và thời điểm khung hình (frameTime). (5) Hệ thống lưu kết quả vào MongoDB, đẩy thông báo SSE đến giao diện tác nhân. (6) Bounding box xuất hiện trên video đồng bộ theo từng khung hình đang phát.
Luồng phát sinh	YOLO Service không phản hồi hoặc quá tải → job thất bại sau 1 lần thử (timeout 25 phút); Worker ghi <code>analysisStatus = 'failed'</code> và thông báo lỗi qua SSE.
Hậu điều kiện	Nhãn nhận diện (Labels, Bbox, FrameTime, TrackId) được lưu và hiển thị theo thời gian thực trên giao diện.

B.3 UC-03: Trích xuất và phân tích bằng chứng

Bảng B.3: Đặc tả Use Case UC-03: Trích xuất bằng chứng (Evidence System)

Mã Use Case	UC-03
Tên	Trích xuất và phân tích bằng chứng (Evidence System)
Tác nhân	Operator
Tiền điều kiện	Video của lượt lặn đang được phát trên TripDetailPage.

Luồng chính	(1) Tác nhân phát hiện vật thể nghi vấn, nhấn <i>Photo</i> hoặc <i>Clip</i> trên thanh công cụ. (2a) <i>Photo</i>: Hệ thống trích xuất khung hình hiện tại thành tệp PNG, tải lên AWS S3. Nếu bounding box đang hiển thị, tọa độ được ghi vào ảnh (canvas burn-in). (2b) <i>Clip</i>: Tác nhân nhấn bắt đầu → dừng; hệ thống ghi nhận t_{start} và t_{end} mà không cắt tệp vật lý. (3) Bằng chứng được lưu vào collection <i>Snapshots</i>; tác nhân có thể ghi chú thêm. (4) Tác nhân tùy chọn kích hoạt YOLO riêng cho bằng chứng này (mở rộng từ UC-02).
Luồng phát sinh	Không có video đang active → nút bị vô hiệu hóa, không thể thao tác.
Hậu điều kiện	Bằng chứng được lưu thành công và hiển thị trong bảng Evidence; tệp video gốc không bị thay đổi.

B.4 UC-04: Tạo báo cáo tóm tắt bằng ngôn ngữ tự nhiên

Bảng B.4: Đặc tả Use Case UC-04: Tóm tắt nhiệm vụ bằng AI (AI Summary)

Mã Use Case	UC-04
Tên	Tạo báo cáo tóm tắt nhiệm vụ bằng AI (AI Summary)
Tác nhân	Operator; Gemini AI (hệ thống bên ngoài)
Tiền điều kiện	Trạng thái Chuyển khảo sát (Project) đã chuyển sang <i>Completed</i> .
Luồng chính	(1) Tác nhân truy cập trang ProjectDetailPage, nhấn nút “<i>Tạo Tóm tắt</i>”. (2) Hệ thống thu thập thông kê chuyển đi: địa điểm (locationName), thời gian, số lượt lặn, tổng số ảnh/video, số cảnh báo cảm biến (Z-Score $z > 2,5$). (3) Dữ liệu được đưa vào mẫu câu (Prompt) và gửi tới Gemini API qua Bull Queue. (4) Hệ thống nhận phản hồi, lưu văn bản tóm tắt vào trường project.aiSummary. (5) Thông báo SSE được đẩy đến người kích hoạt; tóm tắt hiện ra tức thì trên giao diện.

Luồng phát sinh	Gemini API không phản hồi → job thất bại, trạng thái chuyển về <i>failed</i> ; tác nhân có thể thử lại.
Hậu điều kiện	Văn bản tóm tắt xuất hiện trên ProjectDetailPage; ban quản lý nắm bắt nhanh kết quả chuyển đi.